VECTOR

Safety Manual

CBD2300384 D00

ID	SPECDOC124319
Status	Draft
Generated on	2023-04-17 14:14

Contents

Safety Manual **9**

1 General Part **11**

1.1 Introduction 11

1.1.1 Purpose 11

1.1.2 Scope 11

1.1.3 Definitions 11

1.1.4 References 12

1.1.5 Overview 12

1.2 Safety Concept 13

1.2.1 Technical Solution 13

1.2.2 Tool Confidence 15

1.3 Assumed Safety Requirements of MICROSAR Safe 15

1.3.1 Initialization 16

1.3.2 Data Consistency 16

1.3.3 Non-volatile Memory 16

1.3.4 Hard Real-time Scheduling 17

1.3.5 Memory Protection 18

1.3.6 Communication 18

1.3.6.1 Inter ECU Communication 18

1.3.6.2 Intra ECU Communication 19

1.3.7 Monitoring of Software 20

1.3.8 Operating Modes 21

1.3.9 Peripheral In- and output 21

1.3.10 Time Synchronization 22

1.4 Assumptions of Use 22

1.4.1 Safety Concept 22

1.4.2 Integration of MICROSAR Safe Components 24
1.4.3 Partitioning 27
1.4.4 Resources 28
1.4.5 Process 28

2 Safety Manual ComXf 32
2.1 Assumed safety requirements 32
2.2 Configuration constraints 32
2.3 Additional verification measures 32
2.3.1 Guided integration testing 33
2.4 Safety requirements required from other components 34
2.5 Dependencies to hardware 34

3 Safety Manual Crc 35
3.1 Assumed safety requirements 35
3.2 Configuration constraints 35
3.3 Additional verification measures 35
3.4 Safety requirements required from other components 35
3.5 Dependencies to hardware 36

4 Safety Manual DiagXf 37
4.1 Assumed safety requirements 37
4.2 Configuration constraints 37
4.3 Additional verification measures 37
4.3.1 Guided integration testing 37
4.4 Safety requirements required from other components 38
4.5 Dependencies to hardware 38

5 Safety Manual E2E 39
5.1 Assumed safety requirements 39
5.2 Configuration constraints 39
5.3 Additional verification measures 39

5.4 Safety requirements required from other components 40
5.5 Dependencies to hardware 40

6 Safety Manual E2E Protection Wrapper 41
6.1 Safety features 41
6.2 Configuration constraints 41
6.3 Additional verification measures 42
6.3.1 Additional verification of E2EConfig file 42
6.3.2 Additional verification of generator execution 46
6.3.3 Additional verification for application 46
6.4 Dependencies to other components 47
6.4.1 Safety features required from other components 47
6.4.2 Coexistence with other components 47
6.5 Dependencies to hardware 48

7 Safety Manual E2EXf 49
7.1 Assumed safety requirements 49
7.2 Configuration constraints 49
7.3 Additional verification measures 49
7.3.1 Guided integration testing 49
7.4 Safety requirements required from other components 51
7.5 Dependencies to hardware 52

8 Safety Manual EcuM 53
8.1 Assumed safety requirements 53
8.2 Configuration constraints 53
8.3 Additional verification measures 54
8.4 Safety requirements required from other components 54
8.5 Dependencies to hardware 54

9 Safety Manual Os 55
9.1 Safety requirements 55

9.2 Configuration constraints 65
9.3 Additional verification measures 67
9.3.1 Interrupt handling 68
9.3.2 Memory mapping and linking 72
9.3.3 Stack 73
9.3.4 Multicore systems with mixed diagnostic coverage capability 74
9.3.5 (Non-)Trusted Functions 75
9.3.6 Miscellaneous 75
9.3.7 Tracing 78
1.3.8 Barriers 78
9.3.9 Inter OS Application Communication (IOC) 79
9.4 Safety requirements required from other components 87
9.5 Dependencies to hardware 88

10 Safety Manual Os (ARM CortexM) 89
10.1 Assumed safety requirements 89
10.2 Configuration constraints 89
10.3 Additional verification measures 89
10.3.1 General 89
10.3.2 ATSAMV7x 89
10.3.3 S32K14x 89
10.3.4 S32G274A, S32K3XX and S32R45 90
10.3.5 Traveo II - CYTx 90
10.4 Safety requirements required from other components 90
10.5 Dependencies to hardware 90
10.5.1 S32K14x 90
10.5.2 DRA7x 91
10.5.3 STA1385 91
10.5.4 IMX8x 91

10.5.5 S32G274A 91
10.5.6 CYT2Bx 92
10.5.7 CYT3Dx/CYT4Dx 92
10.5.8 TDA3x 92
10.5.9 S32K3XX 92
10.5.10 S32R45 93
10.5.11 CYT3Bx/CYT4Bx 93
10.5.12 BCM89107 93
10.5.13 CYT2Cx 93

11 Safety Manual Rte 95
11.1 Assumed safety requirements 95
11.2 Configuration constraints 97
11.3 Additional verification measures 98
11.3.1 Guided integration testing 99
11.3.1.1 BSW configuration 100
11.3.1.2 Executable Entity Scheduling 101
11.3.1.3 SWC Communication 104
11.3.1.4 Usage of RTE Headers 106
11.3.1.5 Usage of RTE APIs 107
11.3.1.6 Configuration of RTE APIs 109
11.4 Safety requirements required from other components 111
11.5 Dependencies to hardware 112

12 Safety Manual VStdLib 113
12.1 Assumed safety requirements 113
12.2 Configuration constraints 115
12.3 Additional verification measures 115
12.4 Safety requirements required from other components 115
12.5 Dependencies to hardware 115

13 Safety Manual Wdg 116
13.1 Safety requirements 116
13.2 Configuration constraints 116
13.3 Additional verification measures 116
13.4 Safety requirements required from other components 117
13.4.1 Safety features required from other components 117
13.4.2 Coexistence with other components 117
13.5 Dependencies to hardware 117

14 Safety Manual Wdg (DrvWd_XTle4278gEAsr) 118
14.1 Safety features 118
14.2 Configuration constraints 118
14.3 Additional verification measures 118
14.4 Dependencies to other components 118
14.4.1 Safety features required from other components 118
14.4.2 Coexistence with other components 118
14.5 Dependencies to hardware 118

15 Safety Manual WdgIf 119
15.1 Assumed safety requirements 119
15.2 Configuration constraints 119
15.3 Additional verification measures 119
15.4 Safety requirements required from other components 123
15.5 Dependencies to hardware 123

16 Safety Manual WdgM 124
16.1 Assumed safety requirements 124
16.2 Configuration constraints 125
16.3 Additional verification measures 126
16.3.1 Additional verification using WdgM Verifier 127
16.3.2 Additional verification of generator execution 130

16.4 Safety requirements required from other components 133
16.5 Dependencies to hardware 133

17 Glossary and Abbreviations 134
17.1 Glossary 134
17.2 Abbreviations 134

18 Contact 136

Safety Manual

Version	Date	Author	Remarks
1.00.00	2015-11-13	Jonas Wolf	Initial creation
1.01.00	2015-12-18	Jonas Wolf	Information about ASIL added.
1.02.00	2016-01-29	Jonas Wolf	Improvements in formulation.
1.03.00	2016-02-25	Jonas Wolf	Improvements in formulation.
1.03.01	2016-03-30	Jonas Wolf	Review findings incorporated.
1.03.02	2016-05-13	Jonas Wolf	Added hint on hardware-software integration (SMI-4).
1.03.03	2016-07-20	Hartmut Hoerner	Added SMI related to interrupt handling.
1.04.00	2016-09-02	Jonas Wolf	Added TSR-101876 for data consistency.
1.04.01	2016-09-19	Jonas Wolf	Clarifications on SMI-100 and SMI-19.
1.04.02	2016-12-02	Jonas Wolf	Version of referenced document fixed.
1.05.00	2017-02-24	Jonas Wolf	Modified SMI-18: checks need to be enabled per component as well.
1.06.00	2017-05-05	Jonas Wolf	Added SMI-36039: Pointer validity passed to call-outs. Added SMI-36041: Information on Beta-ESCANs. Update of versions of referenced documents. Section 1.1.5 clarified.
1.07.00	2017-06-20	Jonas Wolf	Update of TSRs from TSC (update is backward compatible) Added info on this version history. Fixes of typos.
1.07.01	2017-08-18	Jonas Wolf	Added information on TCL for MSSV and RTE Analyzer.
1.08.01	2017-11-10	Jonas Wolf	Added information on TCL for post-built tool chain. Added information that OS timing protection is only a partial replacement for a watchdog.
1.09.00	2018-01-11	Jonas Wolf	Added details to SMI-16 for interfaces between MICROSAR components.
1.09.01	2018-01-30	Jonas Wolf	Fixed typo in SMI-16.
1.09.02	2018-05-03	Jonas Wolf	Added information on SMI-12.
1.10.00	2018-11-16	Jonas Wolf	Update of technical safety concept. More info on SMI-99.

			Added SMI-246328.
1.11.00	2019-06-25	Jonas Wolf	Update of technical safety concept.
1.12.00	2020-03-16	Jonas Wolf	Adding SMI-428133 for VCA. Adding clarification for SMI-8.
1.13.00	2020-06-03	Jonas Wolf	Update of technical safety concept. Adding SMI-494148 and SMI-246328 and clarifications for systems with safety-related availability requirements.
1.14.00	2020-09-08	Jonas Wolf	Make history more precise and improve wording of SMI-494148.
1.15.00	2021-02-18	Jonas Wolf	Make document structure more precise. Improve details on SMI-14 and SMI-4.
1.16.00	2021-03-08	Jonas Wolf	Update of information on timing guarantees and TSR-6.
1.16.01	2021-03-12	Jonas Wolf	Update with review comments.
1.16.02	2022-04-20	Vibhu Layal	Added information to SMI-9.

1 General Part

1.1 Introduction

1.1.1 Purpose

This document describes the assumptions made by Vector during the development of MICROSAR Safe as Software Safety Element out of Context (SEooC). This document provides information on how to integrate MICROSAR Safe into a safety-related project.

This document is intended for the user of MICROSAR Safe. It shall be read by project managers, safety managers, and engineers to allow proper integration of MICROSAR Safe.

1.1.2 Scope

This document adds additional information to the components that are marked with an ASIL in the delivery description provided by Vector. Neither QM Vector components, nor components by other vendors are in the scope of this document.

Vector assumes that hardware and compiler manuals are correct and complete.
Vector uses the hardware reference manuals and compiler manuals for the development of MICROSAR Safe. Vector has no means to verify correctness or completeness of the hardware and compiler manuals.
Example information that may be critical from these manuals is the register assignment by compiler. This information is used to built up the context that is saved and restored by the operating system.
The compiler manual from the compiler version specified for the project is considered. The considered hardware manuals are documented in the Technical Reference of the hardware-specific component.

A general description of Vector's approach to ISO 26262 is described in [2]. This document is available on request.

1.1.3 Definitions

The words *shall*, *shall not*, *should*, *can* in this document are to be interpreted as described here:

1. *Shall* means that the definition is an absolute requirement of the specification.
2. *Shall not* means that the definition is an absolute prohibition of the specification.
3. *Should* means that there may exist valid reasons in particular circumstances to ignore a particular definition, but the full implications must be understood and carefully weighed before choosing a different course.
4. *Can* means that a definition is truly optional.

Each requirement in this specification has a unique identifier beginning with SMI. This identifier is semantically the same even for different Safety Manuals provided by Vector.

The user of MICROSAR Safe can deviate from all constraints and requirements in this Safety Manual in the responsibility of the user of MICROSAR Safe, if equivalent measures are used.
If a measure is equivalent can be decided in the responsibility of the user of MICROSAR Safe.

1.1.4 References

No.	Source	Title	Version
[1]	ISO	ISO 26262 Road vehicles — Functional safety (all parts)	2011/2012
[2]	Vector	ISO 26262 Compliance Document	1.3.2
[3]	Vector	MICROSAR Safe Product Information	As provided with the quote
[4]	Vector	MICROSAR Safe Silence Verifier Technical Reference	As provided with the delivery
[5]	Vector	Vector Code Analyzer Technical Reference	As provided with the delivery

1.1.5 Overview

This document is automatically generated for a delivery. The content of this document depends on the components (incl. their version) and used hardware (e.g. microcontroller or external hardware) in the delivery. This document is thus valid only for the delivery from Vector that it is included in. The version history above only reflects changes in the component independent part of the Safety Manual.

The structure of this document comprises:

- a general section that covers all assumptions and constraints that are always applicable, and
- at least one section for each ASIL component that covers its constraints and necessary verification steps. The section for a component is identified by its module abbreviation.

Vector's assumptions on the environment of the MICROSAR Safe components as well as the integration process are described.

Vector developed MICROSAR Safe as Safety Element out of Context for projects demanding ASIL D software. All requirements in this document apply independently from the actual highest ASIL of the project (unless otherwise stated).

MICROSAR Safe comprises a set of components developed according to ISO 26262. These components can be combined - together with other measures - to build a safe system according to ISO 26262.

Please read the Product Information MICROSAR Safe [3] first.

1.2 Safety Concept

It is assumed that the user of MICROSAR Safe is responsible for overall system safety. MICROSAR Safe can provide parts of safety mechanisms to its user. It is also assumed that the user of MICROSAR Safe verifies the robustness and effectiveness of the safety mechanisms within its system based on the configuration of MICROSAR Safe.

1.2.1 Technical Solution

Possible hardware faults (transient or permanent) are detected and handled by the application (user of MICROSAR Safe) according to the required ASIL necessary for the project.
This also implies protection against Single Event Upsets (SEUs) in volatile and non-volatile memory, as well as faults in registers and processing logic.

The hardware manual is assumed to be complete and correct.

MICROSAR Safe does not implement mechanisms to mitigate random hardware faults. Instead, MICROSAR Safe relies on hardware mechanisms, such as ECC memory and lock-step cores. For systems with safety-related availability requirements sufficiently reliable hardware or redundancy in hardware is assumed to cover hardware faults. Such hardware faults might e.g. occur during communication between the different parts of the redundant system.

Note that multi-core controllers with cores that provide different levels of diagnostic coverage (lock-step/non-lock-step cores) are considered during development of the MICROSAR Safe operating system.
However, they also require a detailed analysis of core interdependency by the user of MICROSAR Safe.

It is assumed that systematic hardware faults are handled on system or application level by the user of MICROSAR Safe.

The Safety Manual of the hardware MICROSAR Safe controls is used during development. The respective part of MICROSAR Safe Safety Manual describes if MICROSAR Safe implements safety mechanisms required by the Safety Manual of the hardware it controls.

Systematic faults in functionality that is allocated an assumed TSR in MICROSAR Safe are prevented by implementation of an ISO 26262-compliant development process.
It is assumed that development according to an ISO 26262-compliant development process implicitly ensures freedom from interference with respect to memory. I.e. MICROSAR Safe components only write within bounds of memory that they have ownership of.

For systems with safety-related availability requirements, it is assumed that systematic faults in software in the chain of events must be avoided. Thus, the complete software in the chain of events must implement safety requirements (see e.g. TSR-111108). It is

already ensured that no out-of-bounds memory write accesses are performed (see above). Invalid read accesses cannot interfere in memory, since they do not modify memory even if they trigger a trap.

However, the software in the chain of events must also perform its function within a bounded time. Therefore, MICROSAR Safe provides a real-time scheduler in the operating system and the corresponding Runtime Environment (Rte) ensures scheduling as specified and configured by the user of MICROSAR Safe (see TSR-6).

Moreover, components of MICROSAR Safe can provide a bounded execution time of their functionality. A finite execution time analysis (FETA) is performed for those components. This analysis ensures that components of MICROSAR Safe do not disturb execution flow of other modules by occupying CPU resources indefinitely, i.e. it ensures freedom from interference with respect to timing. This analysis is always a worst-case estimation. The information if a component provides this guarantee, can be found in the respective part of the Safety Manual of MICROSAR Safe. A Worst Case Execution Time (WCET) analysis still has to be performed by the user of MICROSAR Safe to evaluate if timing requirements in their system can be achieved.
Invalid read accesses to memory are not an issue in the first place, since the trap handler also is assumed to have a bounded runtime. If the upper bound of execution time of an algorithm in MICROSAR Safe components depends on the validity of read accesses this is analyzed during development of those components.
Invalid read accesses could also impact the functionality of MICROSAR Safe components. However, the correctness of the used data is ensured by the testing activities during the development of MICROSAR Safe and the used defensive programming.

MICROSAR Safe does not implement redundant (inverted) data storage to mitigate software faults.

It is assumed that software with a different integrity level is separated using a memory protection unit (MPU) (see also TSR-7).
The user of MICROSAR Safe has to ensure that write access to the same memory parts is only possible for software of the same or higher integrity level.

Depending on the requirements of the item, timing and program flow must be monitored. MICROSAR Safe provide mechanisms to implement this monitoring (see also TSR-13, TSR-14 and TSR-15).
MICROSAR Safe does not automatically monitor itself for intended timing and program flow.
For systems with safety-related availability requirements, it may be required to avoid interference in time and not only to detect it using e.g. a watchdog. It is assumed that the user of MICROSAR Safe adequately ensures freedom from interference with respect to time for this use-case by configuration of MICROSAR Safe.

MICROSAR Safe components without allocated safety requirements are developed according to an ISO 26262-compliant development process to provide an argument for coexistence.

If safety requirements are implemented in components of MICROSAR Safe, Vector ensures that they are locally non-complex.
For locally non-complex components no diverse implementation of algorithms is designed.

1.2.2 Tool Confidence

MICROSAR Safe must be configured using the DaVinci tools. MICROSAR Safe must be compiled by using a compiler for the target hardware.
These tools must be evaluated with respect to their impact on functional safety by the user.

It is assumed that the user of MICROSAR Safe evaluates and qualifies the defined compiler (incl. options) and linker according to ISO 26262 Part 8 Clause 11.

The DaVinci Developer and DaVinci Configurator are assumed to have an impact on functional safety. In the first place, only a tool error detection level of two can be expected. The resulting tool confidence level (TCL) would require a qualification of those tools.

To ensure freedom from interference with respect to memory for the BSW, Vector provides the MICROSAR Safe Silence Verifier (MSSV).

To ensure freedom from interference with respect to memory and to detect additional faults that may have been introduced by the DaVinci tools for the RTE, Vector provides the RTE Analyzer.

If MSSV and RTE Analyzer are used according to their Technical References and the Safety Manual, the tool confidence level for the DaVinci tools can be reduced to TCL1 by the user of MICROSAR Safe.

Tools by Vector do not implement mechanisms to handle hardware faults on the host development computer.

1.3 Assumed Safety Requirements of MICROSAR Safe

The items listed in this section are the assumed technical safety requirements on the Safety Element out of Context MICROSAR Safe.
These requirements are expected to match the requirements in the actual item development.

All technical safety requirements (TSRs) are assigned an ASIL D (unless otherwise stated) to service as many projects as possible.

No fault tolerant time intervals are given. Timing depends on the used hardware and its configuration.
It is assumed that the user configures MICROSAR Safe adequately for the intended use.

No safe state is defined since MICROSAR Safe allows the user to define the desired behavior in case of a detected fault.

The Safety Manual provided with a delivery shows the details on the support of a safety requirement on component level for a specific version.

1.3.1 Initialization

TSR-1 MICROSAR Safe shall provide a mechanism to initialize itself and its controlled hardware.

It is sensible for safety-related systems to start performing potentially hazardous actions only in a defined and intended state.
For example, adequate setup of clocks may be required to reach required fault tolerant times.

Components of MICROSAR Safe initialize their variables. For post build selectable and post build loadable configurations MICROSAR Safe components use a consistent and defined set of configuration data.

Hardware-specific components of MICROSAR Safe describe in their Technical References what hardware units are controlled.

The user of MICROSAR Safe shall ensure that initialization functions of MICROSAR Safe are called in the right order and at the right point in time.

The user of MICROSAR Safe is also responsible for the startup code and main function.

1.3.2 Data Consistency

TSR-101876 MICROSAR Safe shall provide mechanisms to ensure data consistency.

Concurrent access to resources (e.g. variables) from e.g. task and interrupt level, may lead to data inconsistencies.

MICROSAR Safe provides functions to enable or disable interrupts, spin-locks, resources or abstractions (i.e. exclusive areas) to enable the user of MICROSAR Safe to ensure data consistency.

MICROSAR Safe will not unintentionally enable or disable the mechanisms to ensure data consistency.

MICROSAR Safe also uses this functionality in its own components to ensure data consistency.

1.3.3 Non-volatile Memory

TSR-4 MICROSAR Safe shall provide a mechanism to store data in non-volatile memory.

Calibration or other application data may be safety-related, i.e. if there is data available from non-volatile memory it must not be corrupted by either software or hardware.

MICROSAR Safe implements an **end-to-end protection** in the Non-volatile RAM Manager (NvM) to ensure that data is neither corrupted nor masqueraded in either software or hardware.

Note that hardware may not even start storing data in non-volatile memory or loose it at any time.

Availability of data is, thus, not guaranteed. Availability may be increased by redundantly storing data in non-volatile memory.
The user of MICROSAR Safe must handle unavailability of data on application level.

TSR-5 MICROSAR Safe shall provide a mechanism to retrieve data from non-volatile memory.

Calibration or other application data may be safety-related, i.e. if there is data available from non-volatile memory it must not be corrupted by either software or hardware.

MICROSAR Safe implements an **end-to-end protection** in the Non-volatile RAM Manager (NvM) to ensure that data is neither corrupted nor masqueraded in either software or hardware.

Note that hardware may not even start storing data in non-volatile memory or loose it at any time.

Availability of data is, thus, not guaranteed. Availability may be increased by redundantly storing data in non-volatile memory.
The user of MICROSAR Safe must handle unavailability of data on application level.

Note that if data is written more than once and the latest data in the non-volatile memory gets corrupted, older (uncorrupted) data may be returned to the user of MICROSAR Safe.

1.3.4 Hard Real-time Scheduling

TSR-6 MICROSAR Safe shall provide hard real-time scheduling properties.

Hard real-time scheduling is not required for safety in the first place, since deadline violations can usually be detected and a safe state entered.
However, hard real-time scheduling may support argumentation in some cases.

For systems with safety-related availability requirements hard real-time scheduling properties are essential for the software that needs to stay operational.

The operating system of MICROSAR Safe implements fixed priority scheduling and the immediate priority ceiling protocol specified by AUTOSAR to guarantee hard real-time scheduling properties.

The runtime environment of MICROSAR Safe (SafeRTE) schedules runnables as configured by the user of MICRSOR Safe.

Note that safety-related availability requirements on a system usually require a second independent channel of control.

Note that safety-related availability requirements require analyses of the worst-case execution time (WCET) of the software that needs to stay operational. These analyses are expected from the user of MICROSAR Safe.

1.3.5 Memory Protection

TSR-7 MICROSAR Safe shall provide mechanisms to protect software applications from unspecified memory access.

Partitioning in software is often introduced because of different quality levels of software and different responsibilities of software development on one ECU.

Memory partitioning relies on the memory protection unit (MPU) in hardware for the effectiveness of the mechanism.

MICROSAR Safe configures exactly the configured memory access rights for each task and ISR at the hardware interface of the MPU.

Note that MICROSAR Safe does not necessarily control all available protection units (e.g. system MPUs or peripheral protection units).

The user of MICROSAR Safe is responsible for adequate configuration of the memory partitions.

1.3.6 Communication

1.3.6.1 Inter ECU Communication

TSR-10 MICROSAR Safe shall provide mechanisms to protect communication between ECUs.

Communication between ECUs may be corrupted, unintentionally replayed, lost or masked. To protect against these failure modes MICROSAR Safe provides the end-to-end (E2E) protection mechanism defined by AUTOSAR.

MICROSAR Safe detects repetition, loss, insertion, masking, reodering and corruption of messages between ECUs.

MICROSAR Safe allows the usage of cyclic redundancy checks (CRCs) and cryptographic algorithms to protect the integrity of messages between ECUs.
CRCs provide a certain hamming distance given a polynomial and maximum data block size.
Cryptographic hash functions or message authentication codes (MACs) provide a probabilistic statement on data corruption detection depending on the hash function or MAC, data block size and hash value size.

The user of MICROSAR Safe is responsible for the selection of the E2E profile or algorithm that is adequate to the requirements of the item development.

MICROSAR Safe does not reject messages with a valid CRC, hash or MAC. This is required for systems with safety-related availability requirements to claim reliability of a

single channel. MICROSAR Safe relies on the integrity of the underlying hardware for e.g. verification of a MAC if configured.

TSR-111108 MICROSAR Safe shall provide mechanisms to communicate between ECUs.

End-to-end protection ensures that repetition, loss, insertion, masking, reordering and corruption of messages between ECUs can be detected. These kinds of faults can be introduced by hardware or software.

For systems with safety-related availability requirements it is required to eliminate software as a source of the above-mentioned faults on sender and receiver side.

Thus, MICROSAR Safe does not unintentionally repeat, drop, insert, mask, reorder or corrupt messages. MICROSAR Safe might drop messages that have been corrupted by hardware faults, e.g. corruption by electromagnetic interference on the bus.

Usage of end-to-end protection is still required to ensure protection against hardware faults.

1.3.6.2 Intra ECU Communication

TSR-16 MICROSAR Safe shall provide mechanisms to communicate within its applications.

Software components need to communicate in order to fulfill their task.

MICROSAR Safe provides mechanisms to communicate within OS applications without end-to-end protection.

MICROSAR Safe does not unintentionally repeat, delay, insert, mask, corrupt or loose data communicated within OS applications.

MICROSAR Safe assumes protection of the memory against random hardware faults by the system, e.g. via ECC RAM and lock-step mode.

TSR-12 MICROSAR Safe shall provide mechanisms to communicate between its applications.

Mixed ASIL or multi-core systems need to exchange safety-related information between applications.

MICROSAR Safe provides mechanisms to communicate between OS applications without end-to-end protection.

MICROSAR Safe does not unintentionally repeat, delay, insert, mask, corrupt or loose data communicated between OS applications.

MICROSAR Safe assumes protection of the memory against random hardware faults by the system, e.g. via ECC RAM and lock-step mode.

1.3.7 Monitoring of Software

MICROSAR Safe provides monitoring mechanisms to supervise correct execution of software. These mechanisms must be configured by the user of MICROSAR Safe according to the requirements within the context of the item development.

TSR-13 MICROSAR Safe shall provide a mechanism to detect faults in program flow.

Program flow can be corrupted by random hardware faults or software faults.

The user of MICROSAR Safe can configure a graph of the logical program flow that is then supervised by MICROSAR Safe. A transition from one checkpoint to another that is not allowed is detected.

TSR-14 MICROSAR Safe shall provide a mechanism to detect stuck software.

Alive monitoring is used to reset the software or controller in case it is unresponsive.

The user of MICROSAR Safe can configure entities that need to regularly report their alive status that is then supervised by MICROSAR Safe. Omission of a regular report is detected.

TSR-15 MICROSAR Safe shall provide a mechanism to detect deadline violations.

Deadlines are important to reach a safe state within the defined fault tolerant time interval.

The user of MICROSAR Safe can configure a tolerable time interval for each transition in a logical program flow graph that is then supervised by MICROSAR Safe. Violation of a deadline is detected.

TSR-8 MICROSAR Safe shall provide a mechanism to detect time budget violations.

Budgets for task execution times, inter-arrival times and locking times allow to identify the originator of a deadline violation.

The user of MICROSAR Safe can configure budgets for execution times of tasks and category 2 ISRs, inter-arrival times and duration of locks (e.g. resource or interrupt locks) that are then supervised by MICROSAR Safe.
Exhaustion of budgets is detected.

TSR-9 MICROSAR Safe shall provide a mechanism to terminate software applications.

Terminating a complete OS application may be used as a safety mechanisms to mitigate a software fault within this OS application. This terminated OS application typically comprises QM software.

MICROSAR Safe terminates exactly the requested OS application and only upon request.

Note that MICROSAR Safe will not terminate OS applications automatically without configuration or request.

1.3.8 Operating Modes

TSR-100551 MICROSAR Safe shall provide a mechanism to switch between operating modes.

Fault detection and mitigation often result in a degraded or safe state.

MICROSAR Safe does not unintentionally switch between operating modes. MICROSAR Safe switches to exactly the requested operating mode.

1.3.9 Peripheral In- and output

TSR-17 MICROSAR Safe shall provide a mechanism to read input data from peripheral units.

Input of data from microcontroller peripheral units is required to implement almost any safety-related system.

MICROSAR Safe provides the data read from peripheral units without corruption or unintentional delay.

MICROSAR Safe does not unintentionally reorder nor masquerade data read from peripheral units.

Note that the peripheral hardware may not provide sufficient diagnostic coverage without redundant input.

For example, DIO and SPI drivers provided by Vector support the reading input data from peripheral units as safety requirement.

TSR-18 MICROSAR Safe shall provide a mechanism to write output data to peripheral units.

Output of data from microcontroller peripheral units is required to implement almost any safety-related system.

MICROSAR Safe writes the data provided at the software interfaces without corruption or unintentional delay.

MICROSAR Safe does not unintentionally trigger peripheral output units.

MICROSAR Safe does not unintentionally reorder nor masquerade data read from peripheral units.

Note that the peripheral hardware may require additional mechanisms on application or system level to ensure sufficient safety.

For example, DIO and SPI drivers provided by Vector support the output of data to peripheral units as safety requirement (e.g. to trigger external watchdogs or switch actuation paths).

1.3.10 Time Synchronization

TSR-111109 MICROSAR Safe shall provide a mechanism synchronize time between different ECUs.

For advanced driver assistance systems (ADAS) data from different sensor ECUs must be synchronized to perform sensor fusion. It is necessary that all participating ECUs must have a synchronized time for the sensor fusion to yield correct results. Moreover, it is required that the synchronized time correlates to the real time in the real world. If e.g. synchronized time and real time drift apart, sensor fusion will not yield correct results anymore.

MICROSAR Safe provides time synchronization using different communication busses like CAN, FlexRay and Ethernet. The used mechanisms for time synchronization differ between the different bus systems. This impacts the achievable accuracy of the synchronized time.

MICROSAR Safe can be used as time master and time slave.

MICROSAR Safe does not claim a synchronized state if the reported time is not synchronized to the master.

MICROSAR Safe relies on the integrity of the local clock source used for synchronization. MICROSAR Safe does not monitor the local clock source. Especially, drift and jumps are not monitored by MICROSAR Safe.

MICROSAR Safe does not detect a loss of accuracy resulting from faults in the local clock source (Ethernet) or unexpected interrupt latency (CAN and FlexRay).

1.4 Assumptions of Use

1.4.1 Safety Concept

SMI-14
The user of MICROSAR Safe shall be responsible for the functional safety concept.

The overall functional safety concept is in the responsibility of the user of MICROSAR Safe. MICROSAR Safe can only provide parts that can be used to implement the functional safety concept of the item.
It is also the responsibility of the user of MICROSAR Safe to configure MICROSAR Safe as intended by the user's safety concept.
The safety concept shall only rely on safety requirements explicitly described in this safety manual. If a component from MICROSAR Safe does not explicitly describe safety requirements in this safety manual, this component has been developed according to the methods for ASIL D to provide coexistence with other ASIL components.

- Example: NvM provides safety requirements for writing and reading of data, the lower layers, i.e. MemIf, Ea, Fee and drivers, only provide the ASIL for coexistence.

The safety concept shall **not** rely on functionality that is **not** explicitly described as safety requirement in this safety manual. This functionality may fail silently in case of a detected fault.

- Example: If a potential out-of-bounds memory access, e.g. due to invalid input or misconfiguration, is detected the requested function will not be performed. An error via DET is only reported if error reporting is enabled.

Vector recommends including the components from MICROSAR Safe in the software architecture of the project.
Vector assumes that safety requirements in the user's project are mapped to the corresponding components of MICROSAR Safe.
Vector assumes that the user checked that the assumed safety requirements provided by the component of MICROSAR Safe matches the allocated safety requirement in the software architecture of the project.

SMI-1
The user of MICROSAR Safe shall adequately address hardware faults.

The components of MICROSAR Safe can support in the detection and handling of some hardware faults (e.g. using watchdog).
MICROSAR Safe does not provide redundant data storage.
The user of MICROSAR Safe especially shall address faults in volatile random access memory, non-volatile memory, e.g. flash or EEPROM, and the CPU.
MICROSAR Safe relies on the adequate detection of faults in memory and the CPU by other means, e.g. hardware. Thus, Vector recommends using lock-step CPUs together with ECC memory.
See also SMI-14.

SMI-10
The user of MICROSAR Safe shall ensure that the reset or powerless state is a safe state of the system.

Vector uses this assumption in its safety analyses and development process.

SMI-20
The user of MICROSAR Safe shall implement a timing monitoring using e.g. a watchdog.

The components of MICROSAR Safe do not provide mechanisms to monitor their own timing behavior.
The watchdog stack that is part of MICROSAR Safe can be used to fulfill this assumption.
If the safety concept also requires logic monitoring, the watchdog stack that is part of MICROSAR Safe can be used to implement it.
The watchdog is one way to perform timing monitoring. Today the watchdog is the most common approach. In future, there may be different approaches e.g. by monitoring using a different ECU.
The timing protection provided by the operating system can only partially replace the usage of a timing monitoring by a watchdog, because e.g. deadline violations cannot be

directly detected by the OS timing protection.
See also SMI-14.

SMI-98
The user of MICROSAR Safe shall ensure an end-to-end protection for safety-related communication between ECUs.

The communication components of MICROSAR Safe do not assume sending or receiving as a safety requirement, because considered faults can only be detected using additional information like a cycle counter. Vector always assumes that an end-to-end protection or equivalent mechanism is implemented on application level.
Considered faults in communication are:

- Failure of communication peer
- Message masquerading
- Message corruption
- Unintended message repetition
- Insertion of messages
- Re-sequencing
- Message loss
- Message delay

This requirement can be fulfilled by e.g. using the end-to-end protection wrapper for safety related communication.

SMI-11
The user of MICROSAR Safe shall ensure data consistency for its application.

Data consistency is not automatically provided when using MICROSAR Safe. MICROSAR Safe only provides services to support enforcement of data consistency. Their application is in the responsibility of the user of MICROSAR Safe.
To ensure data consistency in an application, critical sections need to be identified and protected.
To identify critical sections in the code, e.g. review or static code analysis can be used.
To protect critical sections, e.g. the services to disable and enable interrupts provided by the MICROSAR Safe operating system can be used.
To verify correctly implemented protection, e.g. stress testing or review can be used.
Note the AUTOSAR specification with respect to nesting and sequence of calls to interrupt enabling and disabling functions.
See also TSR-101876.

1.4.2 Integration of MICROSAR Safe Components

SMI-2

The user of MICROSAR Safe shall adequately select the type definitions to reflect the hardware platform and compiler environment.

The user of MICROSAR Safe is responsible for selecting the correct platform types (PlatformTypes.h) and compiler abstraction (Compiler.h). Especially the size of the predefined types shall match the target environment.
Example: A uint32 shall be mapped to an unsigned integer type with a size of 32 bits.
The user of MICROSAR Safe can use the platform types provided by Vector. Vector has created and verified the platform types mapping according to the information provided by the user of MICROSAR Safe.

SMI-12
The user of MICROSAR Safe shall initialize all components of MICROSAR Safe prior to using them.

This constraint is required by AUTOSAR anyway. Vector uses this assumption in its safety analyses and development process.
Unintended re-initialization may lead to unintended behavior. Thus, the user shall ensure that initialization shall occur only at defined points in time.
Correct initialization can be verified, e.g. during integration testing.

SMI-16
The user of MICROSAR Safe shall only pass valid pointers at all interfaces to MICROSAR Safe components.

Plausibility checks on pointers are performed by MICROSAR Safe (see also SMI-18), but they are limited. MICROSAR Safe components potentially use provided pointers to write to the location in memory.
Also the length and pointer of a buffer provided to a MICROSAR Safe component need to be consistent.
This requirement applies to QM as well as ASIL components. For example an NvM executed in an ASIL context will write to addresses provided by a QM software components. This requirement does not apply to interfaces between MICROSAR Safe components, because that has been verified by Vector already.
This can e.g. be verified using static code analysis tools, reviews and integration testing.

SMI-36039
The user of MICROSAR Safe shall only retain pointers provided by MICROSAR Safe where explicitly stated.

Some MICROSAR Safe components provide call-outs to user or application code. The pointers passed to these call-outs may have limited validity.
The user of MICROSAR Safe shall ensure that pointers are not used neither for reading nor for writing after returning from the call-out. Exceptions of this rule are explicitly documented for the respective call-out.

SMI-494148
The user of MICROSAR Safe shall ensure the correct usage of the provided API from MICROSAR Safe components.

The correct usage of APIs is documented in the respective Technical Reference of the component.

If a component is used incorrectly, it may not fulfill its functionality as specified. Example: If the wrong data is handed over to the communication stack, the communication stack will send this wrong data on the bus.

This requirement is only applicable if using MICROSAR Safe in a system with safety-related availability requirements.

For components of MICROSAR Safe this requirement is only fulfilled if they have the required Technical Safety Requirements assumed (please see respective section of a component if the Technical Safety Requirement is assumed by that component) and are in turn used correctly again.

SMI-18
The user of MICROSAR Safe shall enable plausibility checks for the MICROSAR Safe components.

This setting is necessary to introduce defensive programming and increase robustness at the interfaces as required by ISO 26262.
This setting shall be configured at `/MICROSAR/EcuC/EcucGeneral/EcuCSafeBswChecks` **and** `<Component-specific path>/<Ma>SafeBswChecks` for all components that are intended to be ASIL.
Ma is the Module Abbreviation.
This setting is enforced by an MSSV plug-in.
This setting does not enable error reporting to the DET component.

SMI-1725
The user of MICROSAR Safe shall configure and use the interrupt system correctly.

The user of MICROSAR Safe is responsible for a correct and consistent configuration and usage of the interrupt system.
Especially the following topics shall be verified:

- Consistent configuration of interrupt category, level and priority in OS and MCAL modules
- Correct assignment of logical channels/instances to interrupt vectors in case of MCAL modules with multiple channels/instances
- The interrupt controller is configured in a mode which processes interrupts of the same level sequentially to avoid unbounded interrupt nesting

SMI-36041
The user of MICROSAR Safe shall not use functionality of MICROSAR Safe components marked with a BETA-ESCAN.

Functionality marked with a BETA-ESCAN is not thoroughly tested by Vector.
Vector ensures that functionality marked with a BETA-ESCAN can be completely disabled.

The list of ESCANs (incl. BETA-ESCANs) is provided with a delivery.

1.4.3 Partitioning

SMI-9
The user of MICROSAR Safe shall ensure that for one AUTOSAR functional cluster (e.g. System Services, Operating System, CAN, COM, etc.) only components from Vector are used.

This assumption is required because of dependencies within the development process of Vector.
This assumption does not apply to the MCAL or the EXT cluster.
Vector may have requirements on MCAL or EXT components depending on the upper layers that are used and provided by Vector. For example, the watchdog driver is considered to have safety requirements allocated to its initialization and triggering services. Details are described in the component specific parts of this safety manual.
This assumption does not apply to components that are not provided by Vector at all.
In case the partitioning solution is used, this assumption only partially applies to the System Services cluster. Only the Watchdog Manager and Watchdog Interface need to be used from Vector then, because the Watchdog Manager and Watchdog Interface will be placed in separate memory partitions apart from the other System Services components.

SMI-32
The user of MICROSAR Safe shall provide an argument for coexistence for software that resides in the same partition as components from MICROSAR Safe.

Vector considers an ISO 26262-compliant development process for the software as an argument for coexistence (see [1] Part 9 Clause 6). Vector assumes that especially freedom from interference with respect to memory is provided by an ISO 26262-compliant development process.
Redundant data storage as the only measure by the other software is not considered a sufficient measure.
If ASIL components provided by Vector are used, this requirement is fulfilled.
In general Vector components do not implement methods to interface with other software (e.g. components, hooks, callouts) in other partitions. They assume that all interfacing components reside in the same partition. Interfacing components are described in the respective technical reference.
If an argument for coexistence cannot be provided, other means of separation shall be implemented (e.g. trusted or non-trusted function calls).

SMI-99
The user of MICROSAR Safe shall verify that the memory mapping is consistent with the partitioning concept.
The volatile data of every component shall be placed in the associated memory partition. This can be verified e.g. by review of the linker map file.
The memory sections for each component placed in RAM can be identified <MIP>_START_SEC_VAR[_<xxx>], where <MIP> is the Module Implementation Prefix of the component.
It is assumed that the MPU is configured to not allow write access by a partition with lower integrity to data from a partition with higher integrity.

1.4.4 Resources

SMI-33
The user of MICROSAR Safe shall provide sufficient resources in RAM, ROM, stack and CPU runtime for MICROSAR Safe.

Selection of the microcontroller and memory capacities as well as dimensioning of the stacks is in the responsibility of the user of MICROSAR Safe.
If MICROSAR Safe components have specific requirements, these are documented in the respective Technical Reference document.

SMI-246328
The user of MICROSAR Safe shall perform a worst-case execution time analysis when using MICROSAR Safe in systems with safety-related availability requirements.

In systems with safety-related availability requirements, it is assumed that an upper bound for the execution time of MICROSAR functionality is required.

This requirement is only applicable if using MICROSAR Safe in a system with safety-related availability requirements.

1.4.5 Process

SMI-15
The user of MICROSAR Safe shall follow the instructions of the corresponding Technical Reference of the components.

Especially deviations from AUTOSAR specifications are described in the Technical References.
If there are constraints for the implementation of an exclusive area, these are described in the Technical References.

SMI-5
The user of MICROSAR Safe shall verify all code that is modified during integration of MICROSAR Safe.

Code that is typically modified by the user of MICROSAR Safe during integration comprises generated templates, hooks, callouts, or similar.
This assumption also applies if interfaces between components are looped through user-defined functions.

Vector assumes that this verification also covers ISO 26262:6-9. Vector assumes that modified code that belongs to a Vector component, e.g. EcuM callouts or OS trace hooks can at least coexist with this component, because no separation in memory or time is implemented.

Example: Callouts of the EcuM are executed in the context of the EcuM.
Non-trusted functions provided by the Vector operating system can be used to implement a separation in memory in code modified by the user of MICROSAR Safe.

SMI-30
The user of MICROSAR Safe shall only modify source code of MICROSAR Safe that is explicitly allowed to be changed.

Usually no source code of MICROSAR Safe is allowed to be changed by the user of MICROSAR Safe.
The user of MICROSAR Safe can check if the source code was modified by e.g, comparing it to the original delivery.

SMI-8
The user of MICROSAR Safe shall verify generated functions according to ISO 26262:6-9.

Generated functions can be identified when searching through the generated code.

An example of generated functions is the configured rules of the Basic Software Manager (BswM). Their correctness can only be verified by the user of MICROSAR Safe. Please note, however, that BswM not necessarily assumes a safety requirement.
This requirement does not apply to MICROSAR SafeRTE and the transformers (ComXf, DiagXf, E2eXf, SomeIpXf).

SMI-19
The user of MICROSAR Safe shall execute the MICROSAR Safe Silence Verifier (MSSV).

MSSV is used to detect potential out-of-bounds accesses by Vector's basic software based on inconsistent configuration.
Details on the required command line arguments and integration into the tool chain can be found in [4].
If the report shows “Overall Check Result: Fail", please contact Vector.

SMI-428133
The user of MICROSAR Safe shall execute the Vector Code Analyzer (VCA).

The Vector Code Analyzer is used to detect potential out-of-bounds accesses by Vector's basic software.
This requirement only applies if the Vector Code Analyzer is delivered.
Details on the required command line arguments and integration into the tool chain can be found in [5].
If the report shows “Overall Check Result: Fail", please contact Vector.

SMI-4
The user of MICROSAR Safe shall perform the integration (ISO 26262:6-10) and verification (ISO 26262:6-11) processes as required by ISO 26262.

Especially, the safety mechanisms and safety-related functionality shall be verified in the final target ECU.
As a prerequisite, Vector assumes that the safety concept by the user of MICROSAR Safe allocates safety requirements to the individual components of MICROSAR Safe (see SMI-14).

Vector assumes that by performing the integration and verification processes as required by ISO 26262 the generated configuration data, e.g. data tables, task priorities or PDU handles, are sufficiently checked. An additional review of the configuration data is then considered not necessary.

Integration does not apply to a MICROSAR Safe component that consists of several units. This integration is already performed by Vector.

However, integration of all MICROSAR Safe components in the specific use-case of the user of MICROSAR Safe is the responsibility of the user of MICROSAR Safe. This also includes the hardware-software integration in the context of the target ECU.

Vector assumes that the integration test performed by the user fulfills the exit criteria defined by ISO 26262:6-10 (Structural coverage at the software architectural level) and ISO 26262:6-11.

Vector assumes that in corresponce with the required ASIL for the project of the user of MICROSAR Safe, typical tests methods, like equivalence classes and boundary value analysis are applied. Creating equivalence classes might reduce testing effort with respect to testing all individual instances of a test object.

Vector especially assumes that in correspondence with the required ASIL for the project of the user of MICROSAR Safe, safety mechanisms are tested using fault injection. Example: Restbus simulation of relevant E2E faults for protected signal groups.

SMI-100
The user of MICROSAR Safe shall ensure that a consistent set of generated configuration is used for verification and production.
Make sure that the same generated files are used for testing and production code. E.g. be aware that configuration can be changed in the DaVinci tools without generating the code again.

Make sure that all generated files have the same configuration basis, i.e. always generate the MICROSAR Safe configuration for all components for a relevant release of the ECU software.

The use of post build loadable is supported but not recommended by Vector. Post build loadable increases complexity of verification and validation processes due to increased number of variants. There is no support by Vector to qualify the post build loadable tool chain provided by Vector to create binary data without a qualified compiler.

SMI-176
The user of MICROSAR Safe shall verify the integrity of the delivery by Vector.
Run the SIPModificationChecker.exe and verify that the source code, BSWMD and safety manual files are unchanged.

SMI-31
The user of MICROSAR Safe shall verify the consistency of the binary downloaded into the ECU's flash memory.

This also includes re-programming of flash memory via a diagnostics service. The consistency of the downloaded binary can be checked by the bootloader or the application. MICROSAR Safe assumes a correct program image.

SMI-3

The user of MICROSAR Safe shall evaluate all tools (incl. compiler) that are used by the user of MICROSAR Safe according to ISO 26262:8-11.

Evaluation especially shall be performed for the compiler, linker, debugging and test tools. Vector provides a guideline for the evaluation of the Tool Confidence Level (TCL) for the tools provided by Vector (e.g. DaVinci Configurator PRO).
MSSV and RTE Analyzer can be assumed TCL2 and qualified for this TCL, since a safety-related development process at Vector is applied for those tools. Confirmation that this safety-related development process was applied is part of the safety case.
Vector has only evaluated the tools used by Vector during the development of MICROSAR Safe. Tool evaluation, for tools used by the user of MICROSAR Safe, is in the responsibility of the user of MICROSAR Safe.

2 Safety Manual ComXf

2.1 Assumed safety requirements

SMI-62930
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-108114	ComXf shall provide a service to initialize itself.	TSR-1	ASIL D
CREQ-108117	ComXf shall provide a service to transform a Sender/Receiver communication.	TSR-10,TSR-111108	ASIL D
CREQ-108118	ComXf shall provide a service to retransform a Sender/Receiver communication.	TSR-10,TSR-111108	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory (and timing) with up to ASIL D.

2.2 Configuration constraints

This component does not have configuration constraints.

2.3 Additional verification measures

Please note that the ComXf Generator and RTE Analyzer only implement measures to detect systematic faults by the software. No measures are implemented to detect or mitigate random faults on the computer used for generation.

SMI-62929
The user of MICROSAR Safe shall execute the RTE Analyzer.

The RTE Analyzer performs checks to identify faults in ComXf. Especially out-of-bounds accesses within ComXf are detected. If RTE Analyzer reports a fault, the generated ComXf cannot be used. Moreover it provides the user of MICROSAR Safe with feedback what was generated. This feedback shall be reviewed during integration testing against the intended software design and its configuration.

Please see the Technical Reference of the RTE Analyzer how to execute it.

SMI-62931
The user of MICROSAR Safe shall ensure that the RTE Analyzer does not report unsupported templates.

The generated ComXf code is based on templates. The templates are instantiated by the ComXf Generator in different variants. The RTE Analyzer verifies that the analyzed template variants have been tested during the development of the ComXf Generator according to ISO 26262.

The last section of the RTE Analyzer configuration feedback report provides the information about the template variants.
The report must show that no unsupported templates have been found.

2.3.1 Guided integration testing

Residual faults in the ComXf Generator can only be found during integration testing. Vector assumes that the user of MICROSAR Safe performs an integration testing and verification of software safety requirements according to ISO 26262 Part 6 Clauses 10 and 11 (see also SMI-4). To support this integration testing the RTE Analyzer produces a configuration feedback report.

Please refer to the Technical Reference of the RTE Analyzer for a description of the configuration feedback report.

The following subsections describe the requirements that must be fulfilled during integration testing and verification of software safety requirements.

Each Safety Manual Item (SMI) is structured in the following way:

- The requirement that must be fulfilled
- Explanation of the requirement and a rationale
- Recommended configuration constraints (optional)
- Recommended means of complying with the requirement (optional)
- Details on the information provided by the RTE Analyzer supporting this requirement

SMI-62933
The user of MICROSAR Safe shall ensure that ComXf serializes and deserializes the data in the format that is expected by the network.

Inconsistencies in the communication extract, the configuration of the communication stack, the ComXf configuration and the RTE configuration can lead to missinterpretation of signal group data.

This requires verification of:

- data type
- bit position
- endianess
- length

for every group signal.

Vector requires using E2E protection for safety-related signals.

Verification of the serialization can e.g. be performed by integration testing on vehicle network level with minimum and maximum values for every group signal.
Example:
For a unsigned 3-bit group signal the values 0 and 7 specify the lower and upper limit.
For a signed 3-bit group signal the values -4 and 3 specify the lower and upper limit.

The RTE Analyzer lists the APIs of SWCs that use ComXf to assist in this integration step. The ComXf API names contain the signal group names.

2.4 Safety requirements required from other components

SMI-857104
COMXF requires the following functionality as safety requirements from the RTE:

- Transformer handling (TSR-10)

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

2.5 Dependencies to hardware

This component does not use a direct hardware interface.

3 Safety Manual Crc

3.1 Assumed safety requirements

SMI-344
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-858	Crc shall provide a service to calculate 8-bit SAE-J1850 CRC.	TSR-10	ASIL D
CREQ-859	Crc shall provide a service to calculate 8-bit 0x2F CRC.	TSR-10	ASIL D
CREQ-860	Crc shall provide a service to calculate 16-bit CCITT CRC.	TSR-10	ASIL D
CREQ-861	Crc shall provide a service to calculate 32-bit IEEE-802.3 CRC.	TSR-10	ASIL D
CREQ-862	Crc shall provide a service to calculate 32-bit E2E Profile 4 CRC.	TSR-10	ASIL D
CREQ-117997	Crc shall provide a service to calculate 64-bit ECMA CRC.	TSR-10	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory with up to ASIL D.

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory with up to ASIL D.

3.2 Configuration constraints

This component does not have configuration constraints.

3.3 Additional verification measures

SMI-49
The user of MICROSAR Safe shall verify that the CRC is calculated for the intended data.

This includes the intended buffer and its size (see also SMI-16), start value and if it is the first call to the service.
Verification can be performed by the "magic check" (see AUTOSAR SWS Crc).

If Crc is used by a MICROSAR Safe component (e.g. E2E, NvM), this requirement is fulfilled for the MICROSAR Safe component.

3.4 Safety requirements required from other components

This component has no safety requirements from other components.

3.5 Dependencies to hardware

This component does not use a direct hardware interface.

4 Safety Manual DiagXf

4.1 Assumed safety requirements

This component does not assume safety requirements. This component is developed according to ISO 26262 only to ensure freedom from interference with respect to memory (and timing) with up to ASIL D.

4.2 Configuration constraints

This component does not have configuration constraints.

4.3 Additional verification measures

SMI-105504
The user of MICROSAR Safe shall execute the RTE Analyzer.

The RTE Analyzer performs checks to identify faults in DiagXf. Especially out-of-bounds accesses within DiagXf are detected. If RTE Analyzer reports a fault, the generated DiagXf cannot be used. Moreover it provides the user of MICROSAR Safe with feedback what was generated. This feedback shall be reviewed during integration testing against the intended software design and its configuration.

Please see the Technical Reference of the RTE Analyzer how to execute it.

SMI-105505
The user of MICROSAR Safe shall ensure that the RTE Analyzer does not report unsupported templates.

The generated DiagXf code is based on templates. The templates are instantiated by the DiagXf Generator in different variants. The RTE Analyzer verifies that the analyzed template variants have been tested during the development of the DiagXf Generator according to ISO 26262.

The last section of the RTE Analyzer configuration feedback report provides the information about the template variants.
The report must show that no unsupported templates have been found.

4.3.1 Guided integration testing

Residual faults in the DiagXf Generator can only be found during integration testing. Vector assumes that the user of MICROSAR Safe performs an integration testing and verification of software safety requirements according to ISO 26262 Part 6 Clauses 10 and 11 (see also SMI-4). To support this integration testing the RTE Analyzer produces a configuration feedback report.

Please refer to the Technical Reference of the RTE Analyzer for a description of the configuration feedback report.

The following subsections describe the requirements that must be fulfilled during integration testing and verification of software safety requirements.

Each Safety Manual Item (SMI) is structured in the following way:

- The requirement that must be fulfilled
- Explanation of the requirement and a rationale
- Recommended configuration constraints (optional)
- Recommended means of complying with the requirement (optional)
- Details on the information provided by the RTE Analyzer supporting this requirement

SMI-105506
The user of MICROSAR Safe shall ensure that DiagXf serializes and deserializes the data in the format that is expected by the DCM.

Inconsistencies in the diagnostic extract, the configuration of the communication stack, the DiagXf configuration and the RTE configuration can lead to missinterpretation of record data element data.

This requires verification of:

- data type
- bit position
- endianess
- length

for every record element.

Verification of the serialization can e.g. be performed by integration testing on ECU level with minimum and maximum values for every record element.
Example:
For a unsigned 3-bit record element the values 0 and 7 specify the lower and upper limit.
For a signed 3-bit record element the values -4 and 3 specify the lower and upper limit.

The RTE Analyzer lists the APIs of SWCs that use DiagXf to assist in this integration step. The DiagXf API names contain the SWC, port and data element names.

4.4 Safety requirements required from other components

This component does not require safety requirements from other components.

4.5 Dependencies to hardware

This component does not use a direct hardware interface.

5 Safety Manual E2E

5.1 Assumed safety requirements

SMI-51
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-1086	E2E shall provide services to protect data.	TSR-10	ASIL D
CREQ-1087	E2E shall provide services to initialize the protection state.	TSR-1, TSR-10	ASIL D
CREQ-1088	E2E shall provide services to check protected data.	TSR-10	ASIL D
CREQ-1089	E2E shall provide services to initialize the check state.	TSR-1, TSR-10	ASIL D
CREQ-1175	E2E shall provide a service to map the check result to an adequate state machine status	TSR-10	ASIL D
CREQ-1091	E2E shall provide a service to check data within a reception window.	TSR-10	ASIL D
CREQ-1092	E2E shall provide a service to initialize the state machine.	TSR-1, TSR-10	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory with up to ASIL D.

5.2 Configuration constraints

This component does not have configuration constraints.

5.3 Additional verification measures

SMI-52
The user of MICROSAR Safe shall verify that the E2E is used as intended.

This includes the verification of the parameters for end-to-end protection, e.g. data ID, and the order of calls as described in the Technical Reference for creating and checking the end-to-end protection.

The service to protect shall be called exactly once before sending. The service to check shall be called exactly once for reception.

This requirement is fulfilled for the MICROSAR Safe E2E Protection Wrapper and MICROSAR Safe E2E Transformer if used as specified in their respective safety manual section.

5.4 Safety requirements required from other components

SMI-50
This component requires the CRC calculation functionality as an assumed safety requirement (TSR-10) from CRC component.
For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you to identify if this requirement is fulfilled.

5.5 Dependencies to hardware

This component does not use a direct hardware interface.

6 Safety Manual E2E Protection Wrapper

Term/Abbreviation	Definition
E2EConfig	E2E Protection Wrapper configuration file. The E2EConfig file is a data file containing information about the protected areas within I-PDUs of a given AUTOSAR system that are to be protected by the E2Elib and the E2EPW and how they are to be protected. It serves as input for the E2EPWG.
E2EPWG	E2E Protection Wrapper Generator. It takes an E2EConfig file as input and generates the E2E Protection Wraper code.
node	node name (also SWC)
port or p	port
pde or o	protected data element (PDE)
direc	direction for which a PDE-type is configured (*rx* or *tx*)

The shortcuts *p* and *o* are used for the E2EPW API functions. For other identifiers (like filenames) the notation *port* and *pde* is preferred.

Some identifiers in this document include concatenations of these terms.
E.g., *E2EPW_Write_p_o_direc ()* (i.e., ".. for a combination of some port *p*, some data element *o* and some direction *direc*).
A protected area (PA) is the container of configuration data for a certain protected data element (PDE).

6.1 Safety features

SMI-143
This component provides the following safety features:

- Creation of end-to-end protection for data
- Check of end-to-end protection for data

6.2 Configuration constraints

SMI-124
The user of MICROSAR Safe shall correctly select and configure I-PDUs and their signal groups (protected areas) that are to be protected by the end-to-end protection. That is, the selection and configuration shall be made according to the communication safety requirements of the system. This includes also the selection of the E2E profile.

SMI-125
The user of MICROSAR Safe shall only configure PAs for the same ECU in one E2EConfig.
If source code needs to be generated for different ECUs, then an individual E2EConfig file must be defined for each ECU.

6.3 Additional verification measures

6.3.1 Additional verification of E2EConfig file

SMI-126
The user of MICROSAR Safe shall ensure that the length of the following identifiers can be uniquely identified by the compiler and linker.

The following strings are internal identifiers:

- E2EPW_Marshal_pde_h
- E2EPW_CheckDeserial_pde_h
- E2EPW_node_port_pde_RX_H
- E2EPW_node_port_pde_TX_H

The following strings are external identifiers:

- E2EPW_Marshal_pde
- E2EPW_CheckDeserial_pde
- E2EPW_Init_p_o_rx
- E2EPW_Init_p_o_tx
- E2EPW_Get_SenderState_p_o
- E2EPW_Get_ReceiverState_p_o
- E2EPW_Write_p_o
- E2EPW_Read_p_o

This requirement applies for all PAs in the E2EConfig with *node*, *p(ort)* and *pde/o*. The number of significant initial characters of the compiler and linker can usually be found in their documentation.

SMI-127
The user of MICROSAR Safe shall verify that all PAs in an E2EConfig with the same *PDE_Name* have the same values in the following fields:

- PDE_Type
- Byte_Order_CPU
- Bit_Order
- Bit_Counting
- Unused_Bit_Value

- Category
- Data_Length
- Is_Opaque

SMI-128
The user of MICROSAR Safe shall verify that each signal in a PA is also be defined in all other PAs with same *PDE_Name*, and the signals must also have the same values in the following fields:

- Signal_Name
- Signal_Type
- Signal_Property
- Byte_Order
- Bit_Length
- Bit_Position

SMI-131
The user of MICROSAR Safe shall verify that the attributes of each PA are defined in the E2EConfig as intended.

Attribute	Requirement
PDE_Type	*PDE_Type* shall equal the data type of the corresponding PDE in the AUTOSAR application and the RTE functions that are invoked by the E2EPW code.
Node_Name	*Node_Name* shall equal the name of the SWC. The preprocessor sets *Node_Name* to the SWC's name. If you do not use the preprocessor or manipulate the E2EConfig file, then make sure that the requirement is still met.
Direction	*Direction* shall equal the direction expected for the corresponding PDE in the RTE.
Byte_Order_CPU	*Byte_Order_CPU* shall equal the byte order of the used CPU.
Unused_Bit_Value	*Unused_Bit_Value* shall equal the fillbit configured in the COM layer for the I-PDU representation of the PDE.
Check_DeSerial	*Check_DeSerial* shall be set to YES if it is required that the E2EPW code checks for deserialization errors on the receiver side.
Includes_H	*Includes_H* shall list the header file that defines the data type of the PDE, the header file that defines the type Rte_Instance (if used) and the header file that defines RTE_E_OK.
Includes_C	*Includes_C* shall list the header file(s) that declare(s) the functions Rte_Read_p_o (), Rte_Write_p_o () and Rte_IsUpdated_p_o () (if used). This requirement only applies if the header files are not already included in the field

	"Includes_H".
Category	*Category* shall equal the short name of the E2Elib profile selected for the PDE.
Data_Length	*Data_Length* shall equal the bit-length of the I-PDU representation of the PDE.
Data_ID and *Data_ID_List*	*Data_IDs and _Data_ID_List* shall fulfill the requirements of the respective E2E profile.
Max_Delta_Counter_Init	*Max_Delta_Counter_Init* shall equal the required initial value of MaxDeltaCounter of the E2E library communication state for the PDE.
Data_Id_Mode	*Data_Id_Mode* shall equal the requirements from the respective E2E profile.
Offset	*Offset* shall equal the communication specification by the OEM.
Max_No_New_Or_Repeated_Data	*Max_No_New_Or_Repeated_Data* shall equal the requirements from the respective E2E profile.

SMI-130
The user of MICROSAR Safe shall verify that each signal that is configured in a PA shall be assigned to a signal in the corresponding PDE in the RTE. Vice versa, each signal in the PDE in the RTE shall be assigned to a signal in the corresponding PA.

If a signal in the RTE is not configured for protection, it will not be protected.
Note that the set of signals in the PA represents the corresponding signal group in the PDE.

Attribute	Requirement
Port_Name	*Port_Name* shall equal the Port Prototype name that is used by the RTE for the related data element in the RTE.
VDP_Name	*VDP_Name* shall always be equal to *PDE_Name*. *VDP_Name* shall equal the VDP name that is used by the RTE for the related data element in the RTE.
Is_Opaque	*Is_Opaque* shall always be FALSE. The option *Is_Opaque* is for future versions of the E2EPW. It allows to pass PAs as UINT8-arrays without the necessity of marshaling. The E2EPW then treats the data element as opaque (no marshaling is performed).
Use_Call_By_Ref	*Use_Call_By_Ref* shall be YES if the data element is not a primitive data type or configured for reception. If YES, then a pointer to the data element is passed as an argument to *Rte_Read_p_o()* and *Rte_Write_p_o()*, respectively ("call by reference"). If NO, then the value of data element is passed ("call by value"). Note that for *Rte_Read_p_o()*, *Use_Call_By_Ref* is always YES.
Use_Rte_Update	*Use_Rte_Update* shall be YES if and only if the protection wrapper shall check for new received data elements on the RTE level before calling *Rte_Read_p_o()*. If YES, then *Rte_Read_p_o()* is only called if *Rte_IsUpdated_p_o()* returns TRUE before. If NO, then *Rte_Read_p_o()* is always called. Note that *Rte_IsUpdated_p_o()* is only available in AUTOSAR 4.0 or higher.

Use_Rte_Instance	*Use_Rte_Instance* shall be YES if and only if the *instance* parameter is required as an argument for *Rte_Write_p_o()*, *Rte_Read_p_o()* and *Rte_IsUpdated_p_o()*. Note that the *instance* parameter is currently not interpreted by the E2EPW, hence different instances share the same E2EPW and E2E library code and data. Therefore, *Use_Rte_Instance* may only be YES if each port *p* and data element *o* is not used by more than one instance.

The user of MICROSAR Safe shall verify that the combination of *Port_Name* and *VDP_Name* is unique over all PDEs for the same node.

SMI-132
The user of MICROSAR Safe shall verify that the attributes of each signal in a PA are defined in the E2EConfig as intended.

Attribute	Requirement
Signal_Name	*Signal_Name* shall equal the name of the corresponding signal in the data element in the RTE.
Signal_Type	*Signal_Type* shall equal the type of the corresponding signal in the data element in the RTE.
Byte_Order	*Byte_Order* shall equal the byte order that is defined for the corresponding signal in the I-PDU for the I-PDU mapping in the COM layer.
Bit_Length	*Bit_Length* shall equal the bit length of the corresponding signal in the I-PDU.
Bit_Position	*Bit_Position* shall equal the start-bit position of the corresponding signal relative to the protected signal group (protected area) and depending on the endianess. If the signal is mapped with Little Endian in the I-PDU, then *Bit_Position* is the least significant bit's position in the least significant byte. If the signal is mapped with Big Endian in the I-PDU, then *Bit_Position* is the most significant bit's position in the most significant byte. For a signal of type UINT8N, the *Bit_Position* is the least significant bit's position in the byte with the lowest address.

SMI-133
The user of MICROSAR Safe shall verify that for all instances of a PDE of the corresponding PA in the various E2EConfig files,

- the profile configurations are equal throughout all PA instances (except *Max_Delta_Counter_Init*),
- all signal configurations are equal throughout all PA instances (except *Signal_Name* and *Signal_ID*) and
- the following field values are equal throughout all PA instances:
  - *Bit_Order*
  - *Bit_Counting*
  - *Unused_Bit_Value*

For example, a sender PDE and a receiver PDE on different ECUs must be configured in different E2EConfig files.

6.3.2 Additional verification of generator execution

SMI-134
The user of MICROSAR Safe shall ensure that the output path for the generated E2EPW code (command-line argument "outpath-path") shall be empty before the generator is started.
If the output path is not empty, code from previous generation runs may be accidentally integrated into the system.
This requirement is fulfilled if generation is performed using DaVinci Configurator PRO.

SMI-135
The user of MICROSAR Safe shall inspect the messages of the E2EPW generator execution.
If the generator aborts the generation process with an error message, the (partially) generated output files shall not be used in the system.
If the generator detects an error, a message starting with "ERROR" is displayed on the standard output.
If the generator shows a warning message starting with "WARNING", the user of MICROSAR Safe shall ensure that the cause of the warning does not invalidate the generated output files.
The generator shows a warning message in the following cases:
1. The E2EConfig file ends with some text after the configuration definitions of all protected areas.
2. The E2EConfig file has at least two PDEs defined with the same *PDE_Name*.
It is strongly recommended to analyze these cases.

SMI-142
The user of MICROSAR Safe shall verify that each generated file is complete, i.e. each file must end with an "EOF" comment.

6.3.3 Additional verification for application

SMI-136
The user of MICROSAR Safe shall verify that the E2E library communication states are not modified by the application while an E2EPW API function is running.
If a pointer to the senders/receivers E2E library communication state is required, it shall be queried with *E2EPW_GetProtectState_p_o()* / *E2EPW_GetCheckState_p_o()*.
Make sure that the values of the E2E library communication state are only read and not altered by the application or a module other than the E2EPW or the E2E library unless intended so.
For special purposes, modification of the sequence counter by the application may be useful or required. Be aware that this is done in the responsibility of the user of MICROSAR Safe.

SMI-137
The user of MICROSAR Safe shall verify that, when a data element is passed to *E2EPW_Write_p_o()* or *E2EPW_Read_p_o()* using call by reference, then the data element is not altered while *E2EPW_Write_p_o()* or *E2EPW_Read_p_o()* is running.

SMI-141

The user of MICROSAR Safe shall verify that an E2EPW API function for a certain *port* / *pde* combination is not called while another or the same E2EPW API function is running for the same combination.
This applies to the following functions:

- *E2EPW_WriteInit_p_o()*, Note: *E2EPW_WriteInit_p_o()* is not reentrant
- *E2EPW_Get_ProtectState_p_o()*
- *E2EPW_Write_p_o()*
- *E2EPW_ReadInit_p_o()*, Note: *E2EPW_ReadInit_p_o()* is not reentrant
- *E2EPW_Get_CheckState_p_o()*
- *E2EPW_Read_p_o()*

SMI-138
The user of MICROSAR Safe shall verify that the E2E library communication state is checked for plausibility with *E2EPW_Get[Protect|Check]State_p_o()* after a startup/restart.

SMI-139
The user of MICROSAR Safe shall verify that that *E2EPW_ReadInit_p_o()* and *E2EPW_WriteInit_p_o()* is only called at partition startup/restart.
Note: *E2EPW_ReadInit_p_o()* and *E2EPW_WriteInit_p_o()* have to be called prior to any other E2EPW API function call.
This initializes the communication state on sender and receiver side. The E2E library communication state should be initialized when the application is initialized or reset.

SMI-140
The user of MICROSAR Safe shall verify that the intended parameters are passed. This especially applies to:

- instance IDs. Note: only one instance ID is used for each *p*, *o* combination.
- AppData parameter for *E2EPW_Write_p_o()* and *E2EPW_Read_p_o()*

6.4 Dependencies to other components

6.4.1 Safety features required from other components

SMI-129
This component requires the initialization, protection, mapping and check features from the E2E library as safety features.

6.4.2 Coexistence with other components

SMI-144
This component requires coexistence with the used Rte.
It is assumed that the user of E2EPW has the required ASIL.

6.5 Dependencies to hardware

This component does not use a direct hardware interface.

7 Safety Manual E2EXf

7.1 Assumed safety requirements

SMI-62928
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-131007	E2EXf shall provide a service to initialize itself.	TSR-1	ASIL D
CREQ-131011	E2EXf shall provide a service to protect data.	TSR-10,TSR-111108	ASIL D
CREQ-133278	E2EXf shall provide a service to verify protected data.	TSR-10,TSR-111108	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory (and timing) with up to ASIL D.

7.2 Configuration constraints

This component does not have configuration constraints.

7.3 Additional verification measures

Please note that the E2EXf Generator and RTE Analyzer only implement measures to detect systematic faults by the software. No measures are implemented to detect or mitigate random faults on the computer used for generation.

SMI-62927
The user of MICROSAR Safe shall execute the RTE Analyzer.

The RTE Analyzer performs checks to identify faults in E2EXf. Especially out-of-bounds accesses within E2EXf are detected. If RTE Analyzer reports a fault, the generated E2EXf cannot be used. Moreover it provides the user of MICROSAR Safe with feedback what was generated. This feedback shall be reviewed during integration testing against the intended software design and its configuration.

Please see the Technical Reference of the RTE Analyzer how to execute it.

7.3.1 Guided integration testing

Residual faults in the E2EXf Generator can only be found during integration testing. Vector assumes that the user of MICROSAR Safe performs an integration testing and verification of software safety requirements according to ISO 26262 Part 6 Clauses 10 and 11 (see also SMI-4). To support this integration testing the RTE Analyzer produces a configuration feedback report.

Please refer to the Technical Reference of the RTE Analyzer for a description of the configuration feedback report.

The following subsections describe the requirements that must be fulfilled during integration testing and verification of software safety requirements.

Each Safety Manual Item (SMI) is structured in the following way:

- The requirement that must be fulfilled
- Explanation of the requirement and a rationale
- Recommended configuration constraints (optional)
- Recommended means of complying with the requirement (optional)
- Details on the information provided by the RTE Analyzer supporting this requirement

SMI-62932
The user of MICROSAR Safe shall ensure that the E2E configuration is as expected.

This requires verification of:

- Profile
- E2E header length
- Input header length
- Execute with no data
- Check incoming data
- Window size (4.4 State Machine)
- Window size init (19-11 State Machine)
- Window size invalid (19-11 State Machine)
- Window size valid (19-11 State Machine)
- Clear from valid to invalid (19-11 State Machine)
- Transit to invalid extended (19-11 State Machine)
- State changes
- Offset of counter
- Offset of CRC
- Offset of data ID high byte
- Offset of E2E header

- Data ID
- Data ID nibble mode
- Data length
- Initial max. allowed counter gap
- Max. missing or repeated data
- Initial counter sync data
- Max. allowed counter gap

for every E2E protected data element.

Verification can e.g. be performed by review of the configuration feedback report against the communication extract and the E2ELIB specification (e.g. for the E2E header length). *Input header length* refers to the length of the payload header e.g. SOMEIPXF header length.

Vector assumes integration testing on vehicle network level using fault-injection is used to additionally verify the effectiveness of the E2E protection.

The RTE Analyzer lists the RTE APIs that use E2E transformation to assist in this integration step.
The RTE Analyzer lists the E2E configuration to assist in this integration step.
E2EXF configurations for which the check of incoming data is disabled are not listed.

7.4 Safety requirements required from other components

SMI-66294
E2EXF requires the following functionality as safety requirement from the E2ELIB:

- data protection
- initialization of protection state
- checking of protection state
- mapping of result state to state machine status
- checking of data
- initialization of state machine

This requirement is fulfilled if an ASIL E2ELIB by Vector is used.

SMI-857105
E2EXF requires the following functionality as safety requirements from the RTE:

- Transformer handling (TSR-10)

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

7.5 Dependencies to hardware

This component does not use a direct hardware interface.

8 Safety Manual EcuM

8.1 Assumed safety requirements

This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-470	EcuM shall provide a service to initialize the ECU management.	TSR-1	ASIL D
CREQ-389	EcuM shall provide a service to complete the startup phase.	TSR-1	ASIL D
CREQ-543	EcuM shall perform validation of all postbuild configurable BSW module configuration parameters.	TSR-1	ASIL D
CREQ-525	EcuM shall provide a callout to initialize driver prior the start of the OS.	TSR-1	ASIL D
CREQ-488	EcuM shall provide a callout to determine the Postbuild configuration data.	TSR-1	ASIL D
CREQ-505	EcuM shall provide a callout to initialize drivers prior Postbuild data is available.	TSR-1	ASIL D
CREQ-509	EcuM shall provide a callout to re-initialize drivers during a restart.	TSR-1	ASIL D
CREQ-454	EcuM shall provide ECU initialization on partitions.	TSR-1	ASIL D
CREQ-494	EcuM shall handle ECU phases on MultiCore ECUs.	TSR-100551	ASIL D
CREQ-473	EcuM shall provide a callout to synchronize the master and slave cores during shutdown.	TSR-100551	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory with up to ASIL D.

8.2 Configuration constraints

SMI-823269
The user of MICROSAR Safe shall configure the attribute /MICROSAR/EcuM/EcuMFlexGeneral/EcuMEnableDefBehaviour to FALSE.

This setting is enforced by a MSSV plugin.

SMI-823270
The user of MICROSAR Safe shall not configure any reference in /MICROSAR/EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMWakeupSource/EcuMComMPNCRef to a PNC for a wakeup source.

This setting is enforced by a MSSV plugin.

8.3 Additional verification measures

SMI-39
The user of MICROSAR Safe shall ensure the intended initialization procedure during integration testing.

The user of MICROSAR Safe shall ensure the intended initialization procedure by performing the following tests:

- Start the ECU and verify that the intended initalization routines are called. This needs to be verified for each Postbuild-selectable configuration.
- Corrupt the Postbuild data and the corresponding CRC in non-volatile memory and start the ECU. Then verify that the corruption is detected by EcuM.
- Start the ECU and verify that the intended Postbuild-selectable configuration is used by the EcuM. This needs to be verified for each Postbuild-selectable configuration.

A start of the ECU includes a "cold-start", reset as well as wake-up from sleep if applicable.

This requirement is only applicable if TSR-1 is assumed as a safety requirement.

8.4 Safety requirements required from other components

SMI-42
This component requires the OS service to get the core ID functionality as an assumed safety requirement (TSR-1) from other interfacing components.

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

This requirement is only applicable if TSR-1 is assumed as a safety requirement.

8.5 Dependencies to hardware

This component does not use a direct hardware interface.

9 Safety Manual Os

9.1 Safety requirements

SMI-1259
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-118	OS shall provide different application modes.	TSR-1	ASIL D
CREQ-63	OS shall provide a service to start the OS.	TSR-6,TSR-1	ASIL D
CREQ-162	OS shall provide a service to initialize itself.	TSR-1,TSR-6	ASIL D
CREQ-51	OS shall automatically start a subset of alarms for an application mode.	TSR-6	ASIL D
CREQ-146	OS shall automatically start a subset of tasks for an application mode.	TSR-6	ASIL D
CREQ-72	OS shall automatically start a subset of schedule tables for an application mode.	TSR-6	ASIL D
CREQ-117	OS shall provide a service to get the current application mode.	TSR-6	ASIL D
CREQ-45	OS shall provide a global callback during OS startup.	TSR-6	ASIL D
CREQ-299	Os shall synchronize the startup in multi-core systems.	TSR-6	ASIL D
CREQ-153	OS shall provide a service to shutdown the OS.	TSR-6	ASIL D
CREQ-95	OS shall provide a service to shutdown all cores synchronously.	TSR-6	ASIL D
CREQ-161	OS shall provide a global callback upon shutdown.	TSR-6	ASIL D
CREQ-89	OS shall provide tasks.	TSR-16,TSR-6,TSR-7,TSR-12,TSR-8,TSR-9	ASIL D
CREQ-58	OS shall provide basic tasks.	TSR-6,TSR-7,TSR-12,TSR-16,TSR-9,TSR-8	ASIL D
CREQ-20	OS shall provide extended tasks.	TSR-7,TSR-6,TSR-	ASIL D

		16,TSR-12,TSR-9,TSR-8	
CREQ-71	OS shall provide a global callback directly before a task starts its execution.	TSR-6,TSR-8,TSR-7	ASIL D
CREQ-165	OS shall provide a global callback directly before a task finishes its execution.	TSR-6,TSR-8,TSR-7	ASIL D
CREQ-42	OS shall provide a service to activate a task.	TSR-6,TSR-8,TSR-7,TSR-9	ASIL D
CREQ-28	OS shall handle multiple activations of basic tasks.	TSR-6	ASIL D
CREQ-101	OS shall provide a service to terminate the calling task.	TSR-9,TSR-6	ASIL D
CREQ-121	OS shall provide a service to define the next activated task.	TSR-6	ASIL D
CREQ-126	OS shall provide a service to explicitly invoke the scheduler.	TSR-6	ASIL D
CREQ-80	OS shall provide a service to get the ID of the current task.	TSR-6	ASIL D
CREQ-74	OS shall provide a service to get the state of a given task.	TSR-6	ASIL D
CREQ-135	OS shall provide a service to declare a task.	TSR-6	ASIL D
CREQ-131845	OS shall provide category 0 ISRs.	TSR-6	ASIL D
CREQ-103	OS shall provide category 1 ISRs.	TSR-6	ASIL D
CREQ-355	OS shall provide category 2 ISRs.	TSR-9,TSR-8,TSR-7,TSR-6	ASIL D
CREQ-115	OS shall provide a service to execute a callback in category 2 ISRs.	TSR-6,TSR-9,TSR-7	ASIL D
CREQ-16	OS shall provide a service to get the ID of the currently executing category 2 ISR.	TSR-7,TSR-6	ASIL D
CREQ-24	OS shall handle un-configured interrupt sources.	TSR-6	ASIL D
CREQ-78	OS shall provide a service to determine the interrupt source of a non-configured interrupt upon handling of such interrupt.	TSR-6	ASIL D
CREQ-1200	OS shall allow usage of interrupt services before start and after shutdown of the OS.	TSR-1	ASIL D
CREQ-154	OS shall provide a nestable service to disable all interrupts.	TSR-101876,TSR-6	ASIL D
CREQ-98	OS shall provide a nestable service to enable all	TSR-6,TSR-	ASIL

	interrupts.	101876	D
CREQ-151	OS shall provide a nestable service to disable all category 2 interrupts.	TSR-101876,TSR-6	ASIL D
CREQ-82	OS shall provide a nestable service to enable all category 2 interrupts.	TSR-101876,TSR-6	ASIL D
CREQ-111	OS shall provide a non nestable service to disable all interrupts.	TSR-101876,TSR-6	ASIL D
CREQ-43	OS shall provide a non nestable service to enable all interrupts.	TSR-101876,TSR-6	ASIL D
CREQ-1257	OS shall provide a non nestable service to disable all interrupts callable from kernel mode.	TSR-6,TSR-101876	ASIL D
CREQ-1259	OS shall provide a non nestable service to enable all interrupts callable from kernel mode.	TSR-6,TSR-101876	ASIL D
CREQ-1256	OS shall provide a non nestable service to disable all interrupts callable from user mode.	TSR-6,TSR-101876	ASIL D
CREQ-1258	OS shall provide a non nestable service to enable all interrupts callable from user mode.	TSR-101876,TSR-6	ASIL D
CREQ-108741	OS shall provide a non nestable service to disable all interrupts callable from any mode.	TSR-6,TSR-101876	ASIL D
CREQ-108742	OS shall provide a non nestable service to enable all interrupts callable from any mode.	TSR-101876,TSR-6	ASIL D
CREQ-108744	OS shall provide a non nestable service to disable all category 2 interrupts callable from kernel mode.	TSR-6,TSR-101876	ASIL D
CREQ-108747	OS shall provide a non nestable service to enable all category 2 interrupts callable from kernel mode.	TSR-6,TSR-101876	ASIL D
CREQ-108743	OS shall provide a non nestable service to disable all category 2 interrupts callable from user mode.	TSR-6,TSR-101876	ASIL D
CREQ-108748	OS shall provide a non nestable service to enable all category 2 interrupts callable from user mode.	TSR-101876,TSR-6	ASIL D
CREQ-108745	OS shall provide a non nestable service to disable all category 2 interrupts callable from any mode.	TSR-101876,TSR-6	ASIL D
CREQ-108746	OS shall provide a non nestable service to enable all category 2 interrupts callable from any mode.	TSR-6,TSR-101876	ASIL D
CREQ-106181	OS shall provide a service to disable a specific interrupt source.	TSR-1,TSR-6	ASIL D
CREQ-106182	OS shall provide a service to enable a specific interrupt source.	TSR-6,TSR-1	ASIL D
CREQ-106183	OS shall provide a service to clear pending interrupts.	TSR-1,TSR-6	ASIL D

CREQ-114872	OS shall provide a service to check whether or not the source of the given ISR is enabled.	TSR-6,TSR-1	ASIL D
CREQ-114873	OS shall provide a service to check whether or not the given ISR has been requested.	TSR-1,TSR-6	ASIL D
CREQ-183488	OS shall provide a service to enable multiple interrupt sources at once.	TSR-6	ASIL D
CREQ-106	OS shall provide events.	TSR-6	ASIL D
CREQ-68	OS shall provide a service to wait for the occurrence of events.	TSR-6	ASIL D
CREQ-155	OS shall provide a service to signal the occurrence of events to a task.	TSR-6	ASIL D
CREQ-53	OS shall provide a service to acknowledge the occurrence of events.	TSR-6	ASIL D
CREQ-129	OS shall provide a service to get the event states of a given task.	TSR-6	ASIL D
CREQ-133	OS shall provide a service to declare an event.	TSR-6	ASIL D
CREQ-140	OS shall provide counters.	TSR-111109,TSR-6	ASIL D
CREQ-22	OS shall provide a service to increment a counter.	TSR-6	ASIL D
CREQ-156	OS shall provide a service to get the current value of a counter.	TSR-6,TSR-111109	ASIL D
CREQ-39	OS shall provide a service to get the difference between a given and the current counter value.	TSR-111109,TSR-6	ASIL D
CREQ-44	OS shall provide a service for each hardware counter to translate a given period of time into number of ticks.	TSR-6	ASIL D
CREQ-297	OS shall provide a service for each hardware counter to translate number of counter ticks into a period of time.	TSR-6	ASIL D
CREQ-1260	OS shall provide a service to get the maximum possible value of a counter.	TSR-6	ASIL D
CREQ-1261	OS shall provide a service to get the number of underlying driver ticks required to reach a specific unit.	TSR-6	ASIL D
CREQ-1262	OS shall provide a service to get the minimum allowed number of ticks for a cyclic alarm of a counter.	TSR-6	ASIL D
CREQ-138	OS shall provide alarms.	TSR-6	ASIL D
CREQ-116	OS shall provide a service to set a relative alarm.	TSR-6	ASIL D
CREQ-29	OS shall provide a service to set an absolute alarm.	TSR-6	ASIL D
CREQ-93	OS shall provide a service to get an alarm.	TSR-6	ASIL

			D
CREQ-164	OS shall provide a service to cancel an alarm.	TSR-6	ASIL D
CREQ-19	OS shall provide a service to get the alarm base.	TSR-6	ASIL D
CREQ-142	OS shall provide alarm callbacks.	TSR-6	ASIL D
CREQ-32	OS shall provide a service to declare an alarm.	TSR-6	ASIL D
CREQ-99	OS shall provide schedule tables.	TSR-6	ASIL D
CREQ-298	OS shall provide expiry points.	TSR-6	ASIL D
CREQ-14	OS shall provide single-shot schedule tables.	TSR-6	ASIL D
CREQ-67	OS shall provide repeating schedule tables.	TSR-6	ASIL D
CREQ-69	OS shall provide explicitly synchronized schedule tables.	TSR-6,TSR-111109	ASIL D
CREQ-81	OS shall provide implicitly synchronized schedule tables.	TSR-6	ASIL D
CREQ-61	OS shall provide a service to start a schedule table at a relative value.	TSR-6	ASIL D
CREQ-136	OS shall provide a service to start a schedule table at an absolute value.	TSR-6	ASIL D
CREQ-96	OS shall provide a service to stop the processing of a schedule table.	TSR-6	ASIL D
CREQ-112	OS shall provide a service to switch the processing between different schedule tables.	TSR-6	ASIL D
CREQ-100	OS shall provide a service to start an explicitly synchronized schedule table synchronously.	TSR-6	ASIL D
CREQ-152	OS shall provide a service to synchronize a schedule table with a synchronization counter.	TSR-111109,TSR-6	ASIL D
CREQ-25	OS shall provide a service to stop synchronization of a schedule table.	TSR-6	ASIL D
CREQ-108	OS shall provide a service to query the state of a schedule table.	TSR-111109,TSR-6	ASIL D
CREQ-36	OS shall provide a mechanism to coordinate concurrent access to shared resources.	TSR-6,TSR-8	ASIL D
CREQ-157	OS shall provide linked resources.	TSR-8,TSR-6	ASIL D
CREQ-66	OS shall provide internal resources.	TSR-8,TSR-6	ASIL D

CREQ-56	OS shall provide a service to acquire a resource.	TSR-101876,TSR-8,TSR-6	ASIL D
CREQ-107	OS shall provide a service to release a resource.	TSR-8,TSR-101876,TSR-6	ASIL D
CREQ-163	OS shall provide a service to declare a resource.	TSR-6	ASIL D
CREQ-15	OS shall provide spinlocks.	TSR-8	ASIL D
CREQ-17	OS shall provide a service to acquire a spinlock.	TSR-8,TSR-101876,TSR-6	ASIL D
CREQ-139	OS shall provide a service to asynchronously acquire a spinlock.	TSR-101876,TSR-6,TSR-8	ASIL D
CREQ-167	OS shall provide a service to release a spinlock.	TSR-8,TSR-101876,TSR-6	ASIL D
CREQ-208524	OS shall provide a service to synchronize participating tasks at a referenced barrier.	TSR-6	ASIL D
CREQ-354	OS shall provide applications.	TSR-7,TSR-9,TSR-8	ASIL D
CREQ-172	OS shall provide a service to determine the application ID to which the current execution context was configured.	TSR-9,TSR-7,TSR-6,TSR-8	ASIL D
CREQ-60	OS shall provide a service to determine the application ID in which the current execution context is executed.	TSR-6,TSR-9,TSR-7,TSR-8	ASIL D
CREQ-114	OS shall provide a service to make an application accessible.	TSR-9,TSR-8,TSR-6,TSR-7	ASIL D
CREQ-109	OS shall provide a service to identify accessibility of OS objects .	TSR-9,TSR-8,TSR-7,TSR-6	ASIL D
CREQ-18	OS shall provide a service to identify object ownership.	TSR-8,TSR-7,TSR-6,TSR-9	ASIL D
CREQ-110	OS shall provide a service to terminate an application.	TSR-6,TSR-9,TSR-7,TSR-8	ASIL D
CREQ-170	OS shall provide restart tasks.	TSR-7,TSR-6,TSR-9,TSR-8	ASIL D
CREQ-104	OS shall provide a service to get the state of a given application.	TSR-7,TSR-6,TSR-8,TSR-9	ASIL D

CREQ-34	OS shall provide a service to call exported services from trusted applications.	TSR-7,TSR-8,TSR-6,TSR-9	ASIL D
CREQ-115372	OS shall allow usage of exported services from trusted applications before start of the OS.	TSR-7	ASIL D
CREQ-105586	OS shall provide a service to call exported services from non-trusted applications.	TSR-7,TSR-6,TSR-8,TSR-9	ASIL D
CREQ-105587	OS shall allow usage of exported services from non-trusted applications before start of the OS.	TSR-7	ASIL D
CREQ-253949	OS shall provide a service to call lightweight exported services from trusted applications.	TSR-7,TSR-9,TSR-6,TSR-8	ASIL D
CREQ-253950	OS shall allow usage of lightweight exported services from trusted applications before start of the OS.	TSR-7	ASIL D
CREQ-48	OS shall provide an application specific callback during OS startup.	TSR-6,TSR-1	ASIL D
CREQ-76	OS shall provide an application specific callback during OS shutdown.	TSR-6	ASIL D
CREQ-54	OS shall provide an application specific callback if an error occurs.	TSR-6	ASIL D
CREQ-844	OS shall provide global memory protection configuration.	TSR-7	ASIL D
CREQ-41	OS shall provide memory protection configuration for applications.	TSR-7	ASIL D
CREQ-62	OS shall provide memory protection configuration for tasks and category 2 ISRs.	TSR-7	ASIL D
CREQ-73	OS shall provide a service to return the access rights of a memory access of a task.	TSR-6,TSR-7	ASIL D
CREQ-13	OS shall provide a service to return the access rights of a memory access of a category 2 ISR.	TSR-7,TSR-6	ASIL D
CREQ-49	OS shall provide execution time protection.	TSR-8	ASIL D
CREQ-85	OS shall provide inter-arrival time protection.	TSR-8	ASIL D
CREQ-31	OS shall provide locking time protection.	TSR-8	ASIL D
CREQ-845	OS shall monitor execution times.	TSR-8	ASIL D
CREQ-846	OS shall monitor inter arrival time frames.	TSR-8	ASIL D
CREQ-847	OS shall monitor locking times.	TSR-8	ASIL D
CREQ-40	OS shall prevent OS-Applications from corrupting the OS through service calls.	TSR-6,TSR-7	ASIL D
CREQ-131	OS shall provide peripheral regions.	TSR-7	ASIL

			D
CREQ-35	OS shall provide a service to modify a value in a peripheral region.	TSR-7,TSR-6	ASIL D
CREQ-79	OS shall provide a service to read a value from a peripheral region.	TSR-6,TSR-7	ASIL D
CREQ-145	OS shall provide a service to write a value in a peripheral region.	TSR-6,TSR-7	ASIL D
CREQ-115373	OS shall allow usage of services for peripheral regions before start of the OS.	TSR-7	ASIL D
CREQ-88	OS shall provide a stack monitoring mechanism.	TSR-7,TSR-6	ASIL D
CREQ-26	OS shall be able to call a global callback function if an error occurs.	TSR-6	ASIL D
CREQ-38	OS shall be able to call a global callback function if a fatal error occurs.	TSR-9,TSR-8,TSR-6	ASIL D
CREQ-133061	OS shall be able to call a global callback function if an internal error occurs.	TSR-6	ASIL D
CREQ-97	OS shall provide a service to all configured error callbacks, which return the parameters of the system service which triggered error handling.	TSR-6	ASIL D
CREQ-23	OS shall provide a service to all configured error callbacks, which returns the service identifier where the error has been risen.	TSR-101876,TSR-6	ASIL D
CREQ-102	OS shall provide information to determine the service and the cause of a reported error.	TSR-101876,TSR-6	ASIL D
CREQ-129663	OS shall provide a service, which writes the context of the thread to which the system returns after error handling.	TSR-6	ASIL D
CREQ-129664	OS shall provide a service, which returns the context of the thread which triggered a fatal error.	TSR-6	ASIL D
CREQ-247263	OS shall provide a service, which returns the address of the latest instruction that caused a fatal error.	TSR-6	ASIL D
CREQ-70	OS shall provide a service to forcibly terminate a task.	TSR-9,TSR-6	ASIL D
CREQ-21	OS shall provide a service to forcibly terminate a category 2 ISR.	TSR-9,TSR-6	ASIL D
CREQ-168	OS shall provide a service to select the idle mode action.	TSR-6	ASIL D
CREQ-75	OS shall provide inter-application communication.	TSR-12,TSR-7	ASIL D
CREQ-848	OS IOC shall provide queued communication.	TSR-12,TSR-6,TSR-7	ASIL D
CREQ-849	OS IOC shall provide unqueued communication.	TSR-7,TSR-12	ASIL D

CREQ-850	OS IOC shall provide 1:1 communication.	TSR-12,TSR-7	ASIL D
CREQ-852	OS IOC shall provide N:1 communication.	TSR-7,TSR-12	ASIL D
CREQ-851	OS IOC shall provide N:M communication.	TSR-7,TSR-12	ASIL D
CREQ-150	OS shall provide a service to write data to an unqueued IOC channel.	TSR-6,TSR-7,TSR-12	ASIL D
CREQ-55	OS shall provide a service to read data from a unqueued IOC channel.	TSR-7,TSR-6,TSR-12	ASIL D
CREQ-91	OS shall provide a service to send data to a queued IOC channel.	TSR-7,TSR-6,TSR-12	ASIL D
CREQ-160	OS shall provide a service to receive data from a queued IOC channel.	TSR-12,TSR-6,TSR-7	ASIL D
CREQ-90	OS shall provide a service to write multiple data to an unqueued IOC channel.	TSR-7,TSR-6,TSR-12	ASIL D
CREQ-147	OS shall provide a service to read multiple data from an unqueued IOC channel.	TSR-7,TSR-6,TSR-12	ASIL D
CREQ-119	OS shall provide a service to send multiple data to a queued IOC channel.	TSR-6,TSR-7,TSR-12	ASIL D
CREQ-113	OS shall service a method to receive multiple data from a queued IOC channel.	TSR-12,TSR-7,TSR-6	ASIL D
CREQ-128	OS shall provide a service to clear all data from a queued IOC channel.	TSR-7,TSR-12,TSR-6	ASIL D
CREQ-141	OS shall be able to call a callback function upon IOC data reception.	TSR-12,TSR-7,TSR-6	ASIL D
CREQ-124	OS shall schedule tasks based on priorities.	TSR-6,TSR-8	ASIL D
CREQ-86	OS shall provide full preemptive task scheduling.	TSR-8,TSR-6	ASIL D
CREQ-30	OS shall provide non preemptive task scheduling.	TSR-8,TSR-6	ASIL D
CREQ-229342	OS shall provide round robin scheduling.	TSR-6	ASIL D
CREQ-149	OS shall provide a service to identify the local core.	TSR-6	ASIL D
CREQ-148	OS shall provide a service to get the number of cores controlled by OS.	TSR-6	ASIL D
CREQ-37	OS shall provide a service to start a core for usage of AUTOSAR OS software.	TSR-1,TSR-7,TSR-8,TSR-9,TSR-12,TSR-6	ASIL D

CREQ-120	OS shall provide a service to start a core for usage of non AUTOSAR OS software.	TSR-1,TSR-6	ASIL D
CREQ-115996	OS shall be able to initialize itself and the hardware on any of the available cores.	TSR-1	ASIL D
CREQ-213720	OS shall provide a service to get the start state of a core.	TSR-6	ASIL D
CREQ-130	OS shall allow to share stacks between tasks which do not preempt each other.	TSR-7	ASIL D
CREQ-115010	OS shall provide a callback for signalling a task activation.	TSR-6	ASIL D
CREQ-115028	OS shall provide a callback for signalling the setting of an event.	TSR-6	ASIL D
CREQ-115029	OS shall provide a callback for signalling a thread switch.	TSR-6	ASIL D
CREQ-115030	OS shall provide a callback for signalling forcible termination of a thread.	TSR-6	ASIL D
CREQ-115031	OS shall provide a callback for signalling the acquirement of a resource.	TSR-6	ASIL D
CREQ-115032	OS shall provide a callback for signalling the release of a resource.	TSR-6	ASIL D
CREQ-115033	OS shall provide a callback for signalling the attempt to acquire a spinlock.	TSR-6	ASIL D
CREQ-115034	OS shall provide a callback for signalling the acquirement of a spinlock.	TSR-6	ASIL D
CREQ-115035	OS shall provide a callback for signalling the release of a spinlock.	TSR-6	ASIL D
CREQ-115036	OS shall provide a callback for signalling the attempt to internally acquire a spinlock.	TSR-6	ASIL D
CREQ-115037	OS shall provide a callback for signalling the internal acquirement of a spinlock.	TSR-6	ASIL D
CREQ-115038	OS shall provide a callback for signalling the internal release of a spinlock.	TSR-6	ASIL D
CREQ-115039	OS shall provide a callback for signalling the locking of interrupts.	TSR-6	ASIL D
CREQ-115040	OS shall provide a callback for signalling the release of an interrupt lock.	TSR-6	ASIL D
CREQ-140268	OS shall provide a callback for signalling failed task activation because the number of activations exceeds the limit.	TSR-6	ASIL D
CREQ-140269	OS shall provide a callback for signalling that event is already set when WaitEvent is called.	TSR-6	ASIL D
CREQ-Os-ArtiServiceCall	OS shall provide ARTI callbacks for the class OS_SERVICECALLS.	TSR-6,TSR-7	ASIL D
CREQ-Os-ArtiTask	OS shall provide ARTI callbacks for the class OSARTI_TASK.	TSR-6,TSR-7	ASIL D

CREQ-Os-ArtiCat2Isr	OS shall provide ARTI callbacks for the class OS_CAT2ISR.	TSR-6,TSR-7	ASIL D
CREQ-Os-ArtiSpinlock	OS shall provide ARTI callbacks for the class OS_SPINLOCK.	TSR-6,TSR-7	ASIL D
CREQ-Os-ArtiApplication	OS shall provide ARTI callbacks for the class OS_APPLICATION.	TSR-6,TSR-7	ASIL D
CREQ-Os-ArtiProtectionHook	OS shall provide ARTI callbacks for the class OS_PROTECTIONHOOK.	TSR-6,TSR-7	ASIL D
CREQ-300	OS shall provide the state READY for tasks.	TSR-8,TSR-6	ASIL D
CREQ-302	OS shall provide the state RUNNING for tasks.	TSR-8,TSR-6	ASIL D
CREQ-303	OS shall provide the state SUSPENDED for tasks.	TSR-8,TSR-6	ASIL D
CREQ-301	OS shall provide the state WAITING for Tasks.	TSR-8,TSR-6	ASIL D
CREQ-305	OS shall provide the state SCHEDULETABLE_NEXT for schedule tables.	TSR-8	ASIL D
CREQ-308	OS shall provide the state SCHEDULETABLE_STOPPED for schedule tables.	TSR-8	ASIL D
CREQ-309	OS shall provide the state SCHEDULETABLE_WAITING for schedule tables.	TSR-8	ASIL D
CREQ-310	OS shall provide the state SCHEDULETABLE_RUNNING for schedule tables.	TSR-8	ASIL D
CREQ-311	OS shall provide the state SCHEDULETABLE_RUNNING_AND_SYNCHRONOUS for schedule tables.	TSR-8	ASIL D
CREQ-1263	OS shall provide the state APPLICATION_ACCESSIBLE for applications.	TSR-8,TSR-7,TSR-6	ASIL D
CREQ-1264	OS shall provide the state APPLICATION_RESTARTING for applications.	TSR-9,TSR-7,TSR-8,TSR-6	ASIL D
CREQ-1265	OS shall provide the state APPLICATION_TERMINATED for applications.	TSR-9,TSR-7,TSR-6,TSR-8	ASIL D
CREQ-1201	OS shall be executable on any combination of cores provided by the hardware.	TSR-7,TSR-12,TSR-1,TSR-4,TSR-8,TSR-9	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory and timing with up to ASIL D.

9.2 Configuration constraints

SMI-378

The user of MICROSAR Safe shall configure and verify the extended OS status of APIs.

The attribute */MICROSAR/Os_Core/Os/OsOs/OsStatus* shall be equal to *EXTENDED*.

The OS safety measures rely on the validity checks defined for EXTENDED status of OS API calls. Without these checks invalid calls might destroy the system integrity and violate safety requirements. Ensuring the validity of API calls and arguments in STANDARD status for any caller (which e.g. might be QM software) is considered to be infeasible.

The user does not need to check this setting explicitly, as the MICROSAR Safe Silence Verifier (MSSV) assures that this setting is correct.

SMI-377
The user of MICROSAR Safe shall configure and verify the service protection.

The attribute */MICROSAR/Os_Core/Os/OsOs/OsServiceProtection* shall be equal to *TRUE*.

The OS safety measures rely on the validity checks defined for OsServiceProtection. Without these checks, invalid API calls might destroy the system integrity and violate safety requirements. Ensuring the validity of API calls for each caller (which might be QM software) is not considered feasible if OsServiceProtection is disabled.

The user does not need to check this setting explicitly, as the MICROSAR Safe Silence Verifier (MSSV) assures that this setting is correct.

SMI-379
The user of MICROSAR Safe shall configure and verify the scalability class 3 or 4.

The attribute */MICROSAR/Os_Core/Os/OsOs/OsScalabilityClass* shall be equal to *SC3* or *SC4*.

The OS safety measures rely on memory protection and service protection provided by the scalability classes SC3 and SC4. Without memory protection, all software parts (even QM parts) would have to ensure freedom from interference regarding memory (including absence of stack overflow). Without service protection, all software parts (even QM parts) would have to ensure only calls with valid access rights.

The user does not need to check this setting explicitly, as the MICROSAR Safe Silence Verifier (MSSV) assures that this setting is correct.

SMI-385
The user of MICROSAR Safe shall not use ISRs of category 1 if timing protection is configured.

If a thread is killed because of timing protection, ISRs of category 1 might be aborted.

A possible workaround is to use ISRs of category 2 instead of category 1.

9.3 Additional verification measures

SMI-380
The user of MICROSAR Safe shall ensure the correct usage of the OS regarding program flow. The correct program flow is only ensured if all OS API functions fulfill the expectations of the user application designer and are used correctly according to the AUTOSAR OS specification and the technical reference.

This requirement is fulfilled by all MICROSAR Safe components. It only applies if APIs of the operating system are used directly by the user of MICROSAR Safe.

This may be checked by monitoring the program flow at development time. Another measure may be, to check whether the API functions have the expected effects by double checking the technical reference.

SMI-3732
The user of MICROSAR Safe shall ensure the correct usage of the hardware peripherals. It is assumed that user software uses the microcontroller exactly as specified in the vendors hardware documentation. Hardware peripheral access may be performed using the MICROSAR OS Peripheral Access APIs.

SMI-383
The user of MICROSAR Safe shall not call OS hook functions directly. The OS hook functions shall be called by the OS only. This applies to the following hook functions:

- *StartupHook*
- *ShutdownHook*
- *ProtectionHook*
- *PreTaskHook*
- *PostTaskHook*
- *ErrorHook*
- *PanicHook*

The OS makes assumptions which are valid if these hook functions are called by the OS. These assumptions might be violated if hook functions are called directly by the user. As a hook may expect that it is called from a specific context, hooks shall not be called from user code directly.

SMI-1047
The user of MICROSAR Safe shall verify that the context described in the Hardware Software Interface (HSI) of the used platform is sufficient for the requirements of his application. The context is derived from the EABI of the platform as well as the calling conventions of the compiler. Discrepancies may occur in case a coprocessor is used, assembler code uses additional registers or precompiled libraries use different registers.

SMI-100816
The user of MICROSAR Safe shall assure that the application initializes the user controlled processor state bits before Os_Init(). The user controlled processor state bits are transfered into every new context as a copy from the previous context. Changing the user controlled processor state bits during runtime may lead to unexpected behavior of the system.

The technical reference of the platform describes all processor state bits controlled by the OS. All other bits are either under user or hardware control. Refer to the hardware manual for a detailed description.

SMI-592
The user of MICROSAR Safe shall verify that software, which implements safety functionality, shall not rely on functionality provided by the following APIs:

- *Os_GetVersionInfo*
- Stack Usage API (*Os_Get*StackUsage*)
- *Os_InitInterruptOnly*

9.3.1 Interrupt handling

SMI-381
The user of MICROSAR Safe shall ensure the correct usage of the OS regarding interrupt disabling. Unintended disabling of interrupts may lead to timing inconsistencies because pending interrupts might be delayed.

The following interrupt disabling API functions shall be used correctly according to the AUTOSAR OS specification:

- *DisableAllInterrupts*
- *SuspendAllInterrupts*
- *SuspendOSInterrupts*

The user shall select the function that is in accordance to the system design expectations.

SMI-382
The user of MICROSAR Safe shall ensure the correct usage of the OS regarding interrupt enabling. Unintended enabling of interrupts may lead to timing inconsistencies (because interrupts, which should be disabled, might occure) and data inconsistencies (see also SMI-11).

The following interrupt enabling API functions shall be used correctly according to the AUTOSAR OS specification:

- *EnableAllInterrupts*

- *ResumeAllInterrupts*
- *ResumeOSInterrupts*

The user shall select the function that is in accordance to the system design expectations.

SMI-482
The user of MICROSAR Safe shall verify that the following API functions:

- *Os_DisableLevel[KM|UM|AM]*
- *Os_EnableLevel[KM|UM|AM]*
- *Os_DisableGlobal[KM|UM|AM]*
- *Os_EnableGlobal[KM|UM|AM]*

are never called in the following cases:

- within critical sections
- nested within other interrupt APIs
- within a section where an interrupt resource is locked
- within interrupt locking spinlocks
- within ISRs, Hook functions, non-trusted functions, trusted functions and alarm callbacks

Furthermore, the disable functions shall only be called when the interrupts are enabled.

SMI-488
The user of MICROSAR Safe shall verify that the following API functions are only called from privileged mode:

- *Os_DisableLevelKM*
- *Os_EnableLevelKM*
- *Os_DisableGlobalKM*
- *Os_EnableGlobalKM*

SMI-489
The user of MICROSAR Safe shall verify that the API functions *Os_EnableGlobal[KM|UM|AM]* are only called if interrupts were disabled previously by calling the corresponding *Os_DisableGlobal[KM|UM|AM]* function.

SMI-490
The user of MICROSAR Safe shall verify that the API functions *Os_EnableLevel[KM|UM|AM]* are only called if interrupts were disabled previously by calling the corresponding *Os_DisableLevel[KM|UM|AM]* function.

SMI-44677
The user of MICROSAR Safe shall verify that all APIs called in ISRs of category 0 are allowed to be called in this context by the technical reference.

The call context „inside category 0 ISR“ cannot be checked by the API functions. Erroneous calls are not detected.

SMI-590
The user of MICROSAR Safe shall verify that all APIs called in ISRs of category 1 are allowed to be called in this context by the AUTOSAR specification and the technical reference.

The call context „inside category 1 ISR“ cannot be checked by the API functions. Erroneous calls are not detected.

SMI-44676
The user of MICROSAR Safe shall verify that all ISRs of category 0 are implemented transparent. Transparent means, that the execution of a category 0 interrupt can not be detected by the OS. Thus, the user has to ensure, that the context described in the HSI of the platform remains unchanged or is restored correctly after the execution of a category 0 ISR.

SMI-541
The user of MICROSAR Safe shall verify that all ISRs of category 1 are implemented transparent. Transparent means, that the execution of a category 1 interrupt can not be detected by the OS. Thus, the user has to ensure, that the context described in the HSI of the platform remains unchanged or is restored correctly after the execution of a category 1 ISR.

SMI-491
The user of MICROSAR Safe shall verify the functionality of each configured ISR.

This includes the call of the correct ISR handler with the expected priority. In case that the following APIs are used, the user has to check that the correct ISR is influenced.

- Os_InitialEnableInterruptSources
- Os_EnableInterruptSource
- Os_DisableInterruptSource
- Os_ClearPendingInterrupt

The aim of the check is to make sure, that no interrupt sources or vectors are mixed up. Therefore, it is sufficient to perform the check with one of the mentioned functions for each ISR.

SMI-44675
The user of MICROSAR Safe shall avoid forcible termination by means of the protection reaction of the *ProtectionHook* in case category 0 ISRs are configured.

If a category 0 ISR causes the *ProtectionHook* call, the interrupted Task or ISR would be terminated together with the category 0 ISR.
This situation should be handled safely in the application design.

SMI-44678
The user of MICROSAR Safe shall avoid forcible termination by means of the protection reaction of the *ProtectionHook* in case category 1 ISRs are configured.

If a category 1 ISR causes the *ProtectionHook* call, the interrupted Task or ISR would be terminated together with the category 1 ISR.
This situation should be handled safely in the application design.

SMI-44681
The user of MICROSAR Safe shall be aware that category 0 ISRs cannot be disabled or suspended by the OS interrupt APIs.
The following APIs have no effect on category 0 ISRs:

- *DisableAllInterrupts*
- *EnableAllInterrupts*
- *SuspendAllInterrupts*
- *ResumeAllInterrupts*
- *SuspendOSInterrupts*
- *ResumeOSInterrupts*
- *Os_DisableLevelAM*
- *Os_EnableLevelAM*
- *Os_DisableLevelKM*
- *Os_EnableLevelKM*
- *Os_DisableLevelUM*
- *Os_EnableLevelUM*
- *Os_DisableGlobalAM*
- *Os_EnableGlobalAM*
- *Os_DisableGlobalKM*
- *Os_EnableGlobalKM*
- *Os_DisableGlobalUM*
- *Os_EnableGlobalUM*

The OS provides no measures to ensure data consistency for variables shared between category 0 ISRs and other Tasks or ISRs. Such measures need to be implemented by the user.

SMI-334973
The user of MICROSAR Safe shall verify that the time between two high resolution timer interrupts is sufficiently small.

In case of a timer of at least 32 bits, the time must not be bigger than 2,147,483,647 hardware ticks.

In case of a timer with less then 32 bits, the time must not be bigger than 2^(n-1) - 1 hardware ticks, where n is the register bit width of the hardware timer module.

Timer interrupts can be delayed by disabling or suspending the interrupt handling or by other interrupts with higher priority.

High resolution timer interrupts are used as Timing Protection interrupts. Additionally, they are used for OS Counters which have the high resolution timer flag enabled.

9.3.2 Memory mapping and linking

SMI-340
The user of MICROSAR Safe shall verify that the complete range specified by each *Os_PeripheralConfigType* object in *Os_Peripheral_Lcfg.c* is intended to be accessed by the (non-trusted) callers of the peripheral API. The first writable address is denoted as *AddressStart* and the last writable address is denoted as *AddressEnd*.

SMI-494
The user of MICROSAR Safe shall verify for IOC functions, that the configured access rights and linker configuration allow only valid callers to write the corresponding IOC data.

Valid callers are the configured sender applications for the corresponding IOC send data structure and the configured receiver applications for the corresponding IOC data structure.

In order to optimize IOC functionality, the OS links the IOC data structures to the sender/receiver application in many cases. In these cases, the user has to ensure, that no other application has write access to this memory section.
In case the IOC data structures are linked to the OS data section, the OS guarantees that only valid senders/receivers can write to the IOC data structure.

The user may find all IOC data structures in the generated file Os_Ioc_Lcfg.c as dynamic objects.

SMI-495
The user of MICROSAR Safe shall ensure by linkage for each optimized spinlock that only intended tasks and ISRs have write access to the corresponding spinlock data (or at least only tasks and ISRs of partitions with the same ASIL levels).

The user of MICROSAR Safe shall verify that no unintended task or ISR has access to data of optimized spinlocks.

SMI-539
The user of MICROSAR Safe shall verify that no configured MPU region grants write access to OS variables from non-trusted software.

SMI-549
The user of MICROSAR Safe shall verify that none of the configured MPU regions grants write access to a memory section where OS stacks are linked to. To fulfill this requirement, the generated map file may be helpful.

SMI-1045
The user of MICROSAR Safe shall verify that the linkage scheme includes a stack safety gap linked adjacent to the stack section (in dependency of the stack growth direction, see technical reference). No software parts shall have write access to the stack safety gap.

This measure allows the MPU to detect stack overflows even if the owner of the stack has write access to data linked adjacent to the stack section.

SMI-564
The user of MICROSAR Safe shall verify the configuration of access rights to sections. A precondition to this is, that a concept exists, which describes the partitioning of the address range and the access rights to the partitions for different parts of the software.

This can be done by stopping the execution and checking the MPU register values. Dependent on the granularity of the access right configuration, this should be done globally (static MPU regions), for each OS Application or for each Task and each ISR.

SMI-109809
The user of MICROSAR Safe shall ensure that the whole OS code is linked within OS start and end code labels as described in the technical reference. The start and end labels are generated into the array *OsCfg_OsCode_Section<x>*. This array may be found in Os_Error_Lcfg.c.

Exceptions which occur in this address range will lead to the safe state of the OS, while other exceptions cause a call of the *ProtectionHook*.

9.3.3 Stack

SMI-565
The user of MICROSAR Safe shall ensure that sufficient stack is available for call/execution of StartOS.

StartOS performs some initializations before switching to an internal stack and enabling the memory protection. The active stack at call of StartOS shall provide sufficient space to execute this code. Because the stack consumption depends on the compiler and the compiler options, it is recommended to switch to a stack provided by the OS before calling StartOS and to use the stack usage measurement API of the OS to determine the necessary stack size.

SMI-4452
If the attribute */MICROSAR/Os/OsOS/OsGenerateMemMap* is equal to *USERCODE_AND_STACKS_GROUPED_PER_CORE*, the user of MICROSAR Safe shall ensure that all configured stack sizes match the MPU granularity and alignment. Otherwise stack protection cannot be ensured.

9.3.4 Multicore systems with mixed diagnostic coverage capability

SMI-483
The user of MICROSAR Safe shall ensure that frequent cross core API calls from cores with lower diagnostic coverage to cores with higher diagnostic coverage do not interfere with the application design.
Excessive runtime consumption of cores with lower diagnostic coverage shall not prevent cores with higher diagnostic coverage from keeping the timing constraints.

One possible measure is to use timing protection for the X-Signal ISRs. Furthermore, X-Signal Receiver ISRs may be split among different sender cores and provided API functions may be restricted.

SMI-484
The user of MICROSAR Safe shall ensure that synchronous cross core API calls from a core with higher diagnostic coverage to a core with lower diagnostic coverage do not violate the safety goals in case the API calls never return.

Synchronous calls block the caller until the return result is received. If, for any reason, a core with lower diagnostic coverage does not return the result or does not return the result in time, the caller has to deal with this situation.

One possible measure is to use asynchronous X-Signal calls.

SMI-485
The user of MICROSAR Safe shall call *ShutdownAllCores* only on cores with the highest diagnostic coverage.

SMI-486
The user of MICROSAR Safe shall note that the *ShutdownHook* might not be called during Shutdown on multicore systems with mixed diagnostic coverage capability.

Errors caused by cores with lower diagnostic coverage capabilities (data overwrite) might prevent the call of the *ShutdownHook* by cores with higher diagnostic coverage.

Note that the AUTOSAR OS specification requires a synchronization barrier before the *ShutdownHook*. If one core does not reach the barrier, no core will reach the *ShutdownHook*.

SMI-487
The user of MICROSAR Safe shall configure and verify that the core with the highest diagnostic coverage initializes the peripheral moduls used by the OS (e.g. MPU, Interrupt Controller).
The attribute */MICROSAR/Os/OsOS/OsHardwareInitCore* shall be set to the core

reference with the highest diagnostic coverage.
The user of MICROSAR Safe shall ensure, that, if a core with lower diagnostic coverage initializes peripherals or hardware components (like e.g. a system MPU), the core with higher diagnostic coverage does not rely on this initialization.

SMI-493
The user of MICROSAR Safe shall verify that the configuration of cross core API calls prevents cores with lower diagnostic coverage from shutting down cores with higher diagnostic coverage.

Mind that X-Signal Receiver ISRs may be split among different sender cores and provided API functions may be restricted.

SMI-25766
The user of MICROSAR Safe shall ensure that the receiver core of cross core API calls is able to handle unintended calls of APIs. This applies only to APIs which are allowed to be called between two cores by configuration.

9.3.5 (Non-)Trusted Functions

SMI-497
The user of MICROSAR Safe shall verify that a trusted, non-trusted or fast trusted function checks the validity of passed arguments before usage.
The caller of CallTrustedFunction, Os_CallNonTrustedFunction or Os_CallFastTrustedFunction might be non-trusted. Therefore, the passed data might also not be trustworthy.

SMI-542
The user of MICROSAR Safe shall verify that each caller of a trusted, non-trusted or fast trusted function is allowed to call the function, or the function validates the caller before performing its functionality to prevent from any violation of safety goals.

SMI-95699
The user of MICROSAR Safe shall verify that the caller Task/ISR of each trusted, non-trusted or fast trusted function which uses the FPU, is configured to use the FPU context.

9.3.6 Miscellaneous

SMI-480
The user of MICROSAR Safe shall not rely on the error parameter macros.
These are named: OSError_<API-name>_<parameter-name>().
They are not assumed as safety requirements by Vector.

SMI-481
The user of MICROSAR Safe shall notify that the *PanicHook* might not be called if the active thread is not allowed to modify the interrupt enable/disable state.

Before calling the *PanicHook*, the OS disables all interrupts. If this fails, the *ProtectionHook* might be called, caused by the illegal access (depending on hardware platform).

SMI-492
The user of MICROSAR Safe shall verify for cross core API calls that for each pair of sender/receiver cores at least one API call is tested and verified across these cores. The test shall be performed for both directions.

SMI-496
The user of MICROSAR Safe shall verify that calls of the optimized spinlock API don't violate any of the spinlock API constraints (e.g. the order of locking, no simultaneous locks from the same core, allowed caller). The optimized spinlock API skips any checks and therefore does not detect wrong calls.
Mind that the order of GetSpinlock calls also depends on preemption of tasks and ISRs.

SMI-591
The user of MICROSAR Safe shall verify the correct usage of IOC API functions. These functions don't call the ErrorHook. It is recommended to check the return code of these functions:

- IocSend/IocWrite
- IocSendGroup/IocWriteGroup
- IocReceive/IocRead
- IocReceiveGroup/IocReadGroup

SMI-109842
The user of MICROSAR Safe shall be aware that spinlocks occupied by an IOC channel are not released when the communicating task or ISR is terminated via protection hook. The operating system does not guarantee that calls to the IOC API for channels, which have blocked spinlocks, will return.
Spinlocks in IOC APIs are, for example, used when

- An IOC channel is non-queued or
- Multiple senders are configured for the same queued channel

SMI-540
The user of MICROSAR Safe shall verify that the user software does not contain system call instructions.

Any system call instruction will result in an OS API or in an OS Error. If the user code uses a system call instruction directly, the triggered OS API will most likely not work as expected. Instead, system calls shall only be used by OS APIs.

A possible verification method might be reviewing the code for (inline) assembler statements, pragmas or intrinsic functions containing system call instructions.

System calls used by the OS are described in the technical reference.

SMI-2900

The user of MICROSAR Safe shall verify that the array OsCfg_CorePhysicalRefs contains all physical cores. The array may be found in Os_Hal_Kernel_Lcfg.c.
For each existing physical hardware core identifier there shall be one corresponding entry inside the array. This entry is indexed by the physical hardware core identifier which is provided directly by the hardware registers.

SMI-39288
The user of MICROSAR Safe shall verify that the array OsCfg_Hal_Context_ExceptionContextRef contains all physical cores. The array may be found in Os_Hal_Context_Lcfg.c.
For each existing physical hardware core identifier, which is also an Autosar core, there shall be one corresponding entry (not NULL_PTR) inside the array. This entry is indexed by the physical hardware core identifier which is provided directly by the hardware registers.

SMI-487587
The user of MICROSAR Safe shall verify the expected behavior of the system if the following OS APIs are used:

- Os_GetExceptionContext
- Os_SetExceptionContext

The OS is not able to guarantee the correctness of the context structure for each possible situation. Thus, the user of MICROSAR Safe needs to assure the functionality for his use case on his own. For this, the values of the members of the structure Os_ExceptionContextType need to be checked for correctness. In case of Os_SetExceptionContext usage, the register values need to be checked against the values in the context structure after returning. Os_GetExceptionContext usage can be checked by preparing a context, triggering an exception and checking the returned values by a runtime test or with the help of a debugger. For Os_SetExceptionContext a breakpoint may be set to the return address in order to check the restored register values against the values in the context structure.

SMI-44342
The user of MICROSAR Safe shall verify that peripheral APIs

- Os_ReadPeripheral8Legacy
- Os_ReadPeripheral16Legacy
- Os_ReadPeripheral32Legacy
- Os_WritePeripheral8Legacy
- Os_WritePeripheral16Legacy
- Os_WritePeripheral32Legacy
- Os_ModifyPeripheral8Legacy

- Os_ModifyPeripheral16Legacy
- Os_ModifyPeripheral32Legacy

are not used on platforms which address width is greater than 32 bits.

SMI-44679
The user shall ensure real time behavior of the system, even in case of delayed calls of the *ProtectionHook*.
The *ProtectionHook* may be delayed by the execution of Cat 0 ISRs. So Cat 0 ISRs should either not be used or assure a sufficiently short runtime.

SMI-109810
The user of MICROSAR Safe shall be aware that in case that MICROSAR OS detects a potentially internal inconsistency, MICROSAR OS enters the PanicHook. The PanicHook is executed with all interrupts disabled. If the PanicHook returns, the OS enters an endless loop with all interrupts disabled. This is assumed to be the safe state of the ECU.

SMI-429122
The user of MICROSAR Safe shall verify that the type Os_AppAccessMaskType is generated in accordance to the number of configured OS Applications and that OS_APPMASK_BITWIDTH is defined in accordance to the processors bit width.

The definition for OS_APPMASK_BITWIDTH may be found in Os_Application_Lcfg.h:
#define OS_APPMASK_BITWIDTH (<BitWidth>u)

The type definition for AppAccessMaskType may be found in Os_Types_Lcfg.h.

If OS_APPID_COUNT is larger than <BitWidth>:

typedef P2CONST(uint<BitWidth>, TYPEDEF, OS_CONST) Os_AppAccessMaskType;

Otherwise:

typedef uint<BitWidth> Os_AppAccessMaskType;

Where <BitWidth> is the bit width of the processor (32 or 64).

9.3.7 Tracing

SMI-69754
The user of MICROSAR Safe shall be aware that user timing or ARTI hook implementation influences the runtime behaviour of the system.

SMI-69755
The user of MICROSAR Safe shall not use any OS API within timing or ARTI hooks.

1.3.8 Barriers

SMI-309405
The user of MICROSAR Safe shall ensure, that the number of calls to the API *Os_BarrierSynchronize* is identical for all participating tasks of the referenced barrier.

Otherwise, a participating task may block an entire core, which results in unpredictable behaviour.

9.3.9 Inter OS Application Communication (IOC)

This chapter describes the review of the correctness of the locking mode generation in file Os_Ioc_Lcfg.c for all configured senders, writers, receivers and readers of IOC communications. In order to keep each safety manual item sufficiently simple, the review has been split into several items. However, it may make sense, to scan through the file for any sender, writer, receiver or reader configuration structures and to perform the respective review on them. Typically two safety manual items will describe the complete review for a specific sender, writer, receiver or reader. For the receivers of queued IOC communications, only one safety manual item needs to be considered.

SMI-889322
The user of MICROSAR Safe shall verify the consistency of the LockMode and the Spinlock pointer for all generated configuration structures of readers on IOC communications. The configuration data structure of readers in Os_Ioc_Lcfg.c looks like:

```
            CONST(Os_IocReadConfigType, OS_CONST)
OsCfg_IocRecv_<NameOfIocCommunication> =


                {


                  /* .Ioc      = */


                  {


                      /* .Spinlock              = */ <SpinlockPointer>,


                      /* .LockMode              = */ <LockModeValue>,


                      ...
```

```
                        /* .AccessingApplications = */
OS_APPID2MASK(<NameOfOsApplication>)



<|OS_APPID2MASK(<NameOfOsApplication>)>*


                    },


                    ...


                };
```

Where <NameOfIocCommunication> is the configured short name of a container with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication and <NameOfOsApplication> is the short name of a configuration container, reffered to by the parameter: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocReceiverProperties/OsIocReceivingOsApplicationRef of the Ioc communication. The values at <SpinlockPointer> and <LockModeValue> shall be reviewed against the table for consistency:

<LockModeValue>	<SpinlockPointer>
OS_IOCLOCKMODE_NONE	NULL_PTR
OS_IOCLOCKMODE_INTERRUPTS	NULL_PTR
OS_IOCLOCKMODE_SPINLOCKS	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>
OS_IOCLOCKMODE_ALL	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>

SMI-889323
The user of MICROSAR Safe shall verify the consistency of the LockMode and the Spinlock pointer for all generated configuration structures of writers on IOC communications. The configuration structure of writers in Os_Ioc_Lcfg.c looks like:

```
            CONST(Os_IocWriteConfigType, OS_CONST)
```

```
    OsCfg_IocSend_<NameOfIocCommunication><_<OsIocSenderId>>? =

{

  /* .Ioc       = */

  {

      /* .Spinlock              = */ <SpinlockPointer>,

      /* .LockMode              = */ <LockModeValue>,

      ...

      /* .AccessingApplications = */
OS_APPID2MASK(<NameOfOsApplication>)

  },

  ...

};
```

Where <NameOfIocCommunication> is the configured short name of a container with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication, <NameOfOsApplication> is the short name of a configuration container, reffered to by the parameter: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocSenderProperties/OsIocSendingOsApplicationRef and <_<OsIocSenderId>>? is the value of a parameter with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocSenderProperties/OsIocSenderId of that IOC communication. This part of the name may not be available in 1:1 communication as the parameter needs not be configured then. The values at <SpinlockPointer> and <LockModeValue> shall be reviewed against the table for consistency:

<LockModeValue>	<SpinlockPointer>
OS_IOCLOCKMODE_NONE	NULL_PTR
OS_IOCLOCKMODE_INTERRUPTS	NULL_PTR
OS_IOCLOCKMODE_SPINLOCKS	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>
OS_IOCLOCKMODE_ALL	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>

SMI-889324
The user of MICROSAR Safe shall verify the correct generation of the LockMode for all readers and writers of a specific IOC communication. See SMI-889322 and SMI-889323 for the relevant configuration structures. The value for <LockMode> shall be identical for all readers and writers and be in accordance to the table:

Data can be read/written atomically	Core local and/or Cross-Core communication	<LockMode>
yes	don't care	OS_IOCLOCKMODE_NONE
no	All writer and reader applications are located on the same core.	OS_IOCLOCKMODE_INTERRUPTS
no	All writer and reader applications are located on different cores.	OS_IOCLOCKMODE_SPINLOCKS
no	Some writer and/or reader applications are located on the same core, while others are located on different cores.	OS_IOCLOCKMODE_ALL

SMI-889325
The user of MICROSAR Safe shall verify the consistency of the LockMode and the Spinlock pointer for all generated configuration structures of receivers on IOC communications. The configuration data structure of receivers in Os_Ioc_Lcfg.c looks like:

```
CONST(Os_Ioc<Bytes>ReceiveConfigType, OS_CONST)
```

```
OsCfg_IocRecv_<NameOfIocCommunication> =

{

  /* .Ioc      = */

  {

    /* .Base     = */

    {

      /* .Spinlock              = */ <SpinlockPointer>,

      /* .LockMode              = */ <LockModeValue>,

      ...

      /* .AccessingApplications = */
OS_APPID2MASK(<NameOfOsApplication>)

    },
```

```
    ...


  },


  ...


};
```

Where <NameOfIocCommunication> is the configured short name of a container with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication. The value of <Bytes> is one of: { 08, 16, 32, Ref }. The value <NameOfOsApplication> is the short name of a configuration container, reffered to by the parameter: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocReceiverProperties/OsIocReceivingOsApplicationRef of the Ioc communication. The values at <SpinlockPointer> and <LockModeValue> shall be reviewed against the table for consistency:

<LockModeValue>	<SpinlockPointer>
OS_IOCLOCKMODE_NONE	NULL_PTR
OS_IOCLOCKMODE_INTERRUPTS	NULL_PTR
OS_IOCLOCKMODE_SPINLOCKS	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>
OS_IOCLOCKMODE_ALL	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>

SMI-889326
The user of MICROSAR Safe shall verify the consistency of the LockMode and the Spinlock pointer for all generated configuration structures of senders on IOC communications. The configuration structure of senders in Os_Ioc_Lcfg.c looks like:

```
CONST(Os_Ioc<Bytes>SendConfigType, OS_CONST)


OsCfg_IocSend_<NameOfIocCommunication><_<OsIocSenderId>>? =
```

```
                {

                  /* .Ioc       = */

                  {

                    /* .Base     = */

                    {

                      /* .Spinlock              = */ <SpinlockPointer>,

                      /* .LockMode              = */ <LockModeValue>,

                      ...

                      /* .AccessingApplications = */
OS_APPID2MASK(<NameOfOsApplication>)

                    },

                    ...
```

```
  },

  /* .FifoWrite = */

  {

    ...

    /* .Data     = */ <DataBufferPointer>,

    ...

  }

};
```

Where <NameOfIocCommunication> is the configured short name of a container with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication and <_<OsIocSenderId>>? is the value of a parameter with definition ref: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocSenderProperties/OsIocSenderId of that IOC communication. This part of the name may not be available in 1:1 communication as the parameter needs not be configured then. The value of <Bytes> is one of: { 08, 16, 32, Ref }. The value of <NameOfOsApplication> is the short name of a configuration container, reffered to by the parameter: /MICROSAR/Os/OsIoc/OsIocCommunication/OsIocReceiverProperties/OsIocReceivingOsApplicationRef of the Ioc communication. The value of <DataBufferPointer> will be OsCfg_Ioc_<NameOfIocCommunication><_<OsIocSenderId>>?_Buffer_Dyn, where the sender ID may not be identical to the one used for the name of the configuration structure.

The values at <SpinlockPointer> and <LockModeValue> shall be reviewed against the table for consistency:

<LockModeValue>	<SpinlockPointer>
OS_IOCLOCKMODE_NONE	NULL_PTR
OS_IOCLOCKMODE_INTERRUPTS	NULL_PTR
OS_IOCLOCKMODE_SPINLOCKS	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>
OS_IOCLOCKMODE_ALL	&OsCfg_Spinlock_IocSpinlock_<NameOfIocCommunication>

SMI-889327
The user of MICROSAR Safe shall verify the correct generation of the LockMode for all senders of a specific IOC communication. See SMI-889326 for the relevant configuration structures. The value for <LockMode> shall be identical for all senders and be in accordance to the table:

More than one sender	<DataBufferPointer>	Core local and/or Cross-Core communication	<LockModeValue>
no	not relevant	not relevant	OS_IOCLOCKMODE_NONE
yes	different for all senders	not relevant	OS_IOCLOCKMODE_NONE
yes	Identical for all senders (typically named with lowest sender ID)	All sender applications are located on the same core.	OS_IOCLOCKMODE_INTERRUPTS
yes	Identical for all senders (typically named with lowest sender ID)	All sender applications are located on diferent cores.	OS_IOCLOCKMODE_SPINLOCKS
yes	Identical for all senders (typically named with lowest sender ID)	Some sender applications are located on the same core while others are located on different core.	OS_IOCLOCKMODE_ALL

Mind that MICROSAR-OS uses the same data pointer for all sender applications when a receiver callback is configured.

9.4 Safety requirements required from other components

This component does not require safety requirements from other components.

9.5 Dependencies to hardware

The dependencies of this component to hardware is described in the platform specific part of the Safety Manual.

10 Safety Manual Os (ARM CortexM)

10.1 Assumed safety requirements

No additional safety requirements are provided.

10.2 Configuration constraints

This component does not have configuration constraints.

10.3 Additional verification measures

10.3.1 General

SMI-44709
The user of MICROSAR Safe shall verify the generated value of the define *OS_HAL_MP_STACK_REGION_NUMBER* in *Os_Hal_MemoryProtection_Cfg.h*. The value must be a valid region number provided by the hardware.

SMI-394251
The user of MICROSAR Safe shall verify that the interrupt priority of any Category 2 interrupt (including CounterIsr) that can cause scheduling is lower than all other Category 2 interrupts.

10.3.2 ATSAMV7x

SMI-30843
The user of MICROSAR Safe shall verify that the array Core2Thread in the object OsCfg_Hal_Core2Thread contains all physical cores.
For each existing physical hardware core identifier there shall be one corresponding entry inside the array which is indexed by the physical hardware core identifier provided directly by the hardware registers.

10.3.3 S32K14x

SMI-333241
The user of MICROSAR Safe shall verify that the following configuration struct is generated into Os_Hal_Timer_Lcfg.c if the FTM timer is used by the OS:
CONST(Os_Hal_TimerFrtConfigType, OS_CONST)
OsCfg_Hal_TimerFrt_<CounterName>
The following struct members must be checked (where <X> (0, 2, 4, 6) is the channel ID) and <y> is the timer module):
ModuleControlRegisterAddress = OS_HAL_TIMER_FTM<Y>_BASE
ModuleCountRegisterAddress = OS_HAL_TIMER_FTM<Y>_BASE + OS_HAL_TIMER_FTM_COUNTREGISTER_OFFSET
ChannelControlRegisterAddress = OS_HAL_TIMER_FTM<Y>_BASE + OS_HAL_TIMER_FTM_CHANNEL<X>_OFFSET
ChannelCompareRegisterAddress = OS_HAL_TIMER_FTM<Y>_BASE +

OS_HAL_TIMER_FTM_CHANNEL<X>_OFFSET +
OS_HAL_TIMER_FTM_CHANNEL_CMP_OFFSET

10.3.4 S32G274A, S32K3XX and S32R45

SMI-431712
The user of MICROSAR Safe shall verify for all non-autostarted Autosar cores that the generated value of CoreStartAddress has to be the base address of the interrupt vector table of the corresponding core.
The struct "OsCfg_Hal_Core_OsCoreX" shall be checked (where <X> is the logical core ID of the checked core)
CONST(Os_Hal_CoreConfigType, OS_CONST) OsCfg_Hal_Core_OsCoreX = {
/* .CoreId = */ X,
/* .CoreStartAddress = */ (uint32)(&OsCfg_Hal_Core_OsCoreX_VectorTable)
};

10.3.5 Traveo II - CYTx

SMI-568210
The user of MICROSAR Safe shall verify that the following configuration struct is generated into Os_Hal_Timer_Lcfg.c if the TCPWM0 timer is used by the OS:
CONST(Os_Hal_TimerFrtConfigType, OS_CONST)
OsCfg_Hal_TimerFrt_<CounterName>
The following struct members must be checked (where <X> is the channel ID):
TimerBaseAddr = OS_HAL_TIMER_FRT_TCPWM0_GROUP2_BASE
CounterIndex = OS_HAL_TCPWM_COUNTER_X

10.4 Safety requirements required from other components

This component does not require safety requirements from other components.

10.5 Dependencies to hardware

SMI-116969
This component requires exclusive access to the hardware registers of the unit it is intended to control. See the technical reference for the hardware register names and used hardware manuals.

10.5.1 S32K14x

SMI-116970
The following hardware safety manual has been taken into consideration:
File Name: S32K14XSM.pdf
S32K14x Series Safety Manual
Doc. No. S32K14XSM

This OS does not implement any requirement from that safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.2 DRA7x

SMI-147939
The following hardware safety manual has been taken into consideration:
File Name: SPRUII9.pdf
Safety Manual for TDA2x, TDA2Px, and DRA7x Applications Processors
Literature. No. SPRUII9

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.3 STA1385

SMI-431926
The following hardware safety manual has been taken into consideration:
File Name: ST_Telemaco3P_STA1385_SM.pdf
Funtional Safety Manual for Telemaco3P(STA1385) Rev1.3
Literature. No. ST_Telemaco3P_STA1385_SM

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.4 IMX8x

SMI-431927
The following hardware safety manual has been taken into consideration:
File Name: iMX8M8XSM.pdf
Funtional i.MX 8/8X Applications Processor Safety
Manual. Document Number: iMX8M8XSM Rev. 0, 1 April 2019

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.5 S32G274A

SMI-431713
The following hardware safety manual has been taken into consideration:
File Name: S32G274ASM.pdf, including Assumptions.xlsx, Safety_Mechanisms.xlsx and Module_Classification.xlsx
Funtional S32G274A Safety Manual
Document identifier: S32G274ASM, Rev. 1 Draft C,30 Aug 2019

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.6 CYT2Bx

SMI-568209
The following hardware safety manual has been taken into consideration:
File Name: SM-002-20312_0E_V.pdf
Traveo™ II Automotive Body Controller Entry Safety Manual Doc. No. 002-20312 Rev. *E

This OS implements safety requirement MXS40-319. Any other requirement from the safety manual is not implemented by this OS.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.7 CYT3Dx/CYT4Dx

SMI-672568
The following hardware safety manual has been taken into consideration:
File Name: CYT3D_4D_SafetyManual_2021_03_002-31415_0A.pdf
Traveo II Automotive Cluster 2D Safety Manual
Document Number: 002-31415 Rev. *A

This OS implements safety requirement MXS40-319. Any other requirement from the safety manual is not implemented by this OS.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.8 TDA3x

SMI-793100
The following hardware safety manual has been taken into consideration:
File Name: TDA3x_SafetyManual_SPRUIG2_September2017.pdf, including TDA3_PPAP_FMEDA_customerVersion(6).xlsm
Safety Manual for TDA3x Automotive Vision safety-critical Applications Processors
Document identifier: SPRUIG2 , September 2017

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.9 S32K3XX

SMI-844633
The following hardware safety manual has been taken into consideration:
File Name: S32K3XXSM.pdf, including Addendum_S32K3xx.xlsx
Funtional S32K3XX Safety Manual
Document identifier: S32K3XXSM, Rev. 1, Sep 2021

File Name: S32K396 Preliminary Safety Manual.pdf, including Addendum_S32K396.xlsx
Funtional S32K396 Safety Manual
Document identifier: S32K396SM, Rev. 1, Nov 2021

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

10.5.10 S32R45

SMI-844634

The following hardware safety manual has been taken into consideration:
File Name: S32R45SM.pdf, including Addendum_S32R45.xlsx
Funtional S32R45 Safety Manual
Document identifier: S32R45SM, Rev. 1, Aug 2021

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

SMI-850672

10.5.11 CYT3Bx/CYT4Bx

The following hardware safety manual has been taken into consideration:
File Name: CYT4BB-BF_3BB_BH_SM_2021-08-25_002-25937_0E_V.pdf
TRAVEO™ II Automotive Body Controller High Safety Manual
Document Number: 002-25937 Rev. *E

MICROSAR OS implements safety requirement MXS40-319. Any other requirement from the safety manual is not implemented by this OS.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

SMI-853203

10.5.12 BCM89107

The following hardware safety manual has been taken into consideration:
File Name: BCM89107_SM_89107-RM200l.pdf
BCM89107 Safety Manual, Preliminary Reference Manual
Document identifier: 89107-RM200 February 1, 2018

This OS does not implement any requirement from the safety manuals.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

SMI-886602

10.5.13 CYT2Cx

The following hardware safety manual has been taken into consideration:
File Name: CYT2_B6_B7_B9_BL_CL_SM__002-20312_0G_V.pdf
Traveo™ T2G Automotive Body/Cluster Controller Entry Safety Manual
Document No. 002-20312 Rev. *G, 2022-02-21

This OS implements safety requirement MXS40-319. Any other requirement from the safety manual is not implemented by this OS.
It is assumed that the requirements related to OS functionality are related to latent faults only and the aimed ASIL is still achievable without implementation by the OS.

11 Safety Manual Rte

11.1 Assumed safety requirements

SMI-323
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-1024	Rte shall provide a service to initiate the transmission of data elements with last-is-best semantic for explicit S/R communication.	TSR-16,TSR-12,TSR-111108,TSR-10	ASIL D
CREQ-1021	Rte shall provide a service to copy the received data element to a buffer with last-is-best semantic for explicit S/R communication.	TSR-16,TSR-111108,TSR-10,TSR-12	ASIL D
CREQ-1022	Rte shall provide a service to get the value of the received data element with last-is-best semantic for explicit S/R communication.	TSR-16,TSR-12	ASIL D
CREQ-1031	Rte shall provide a service to read a data element for implicit S/R communication.	TSR-12,TSR-16	ASIL D
CREQ-1029	Rte shall provide a service to write a data element for implicit S/R communication.	TSR-16,TSR-12	ASIL D
CREQ-1041	Rte shall provide a service to get the reference of a data element to be written for implicit S/R communication.	TSR-16,TSR-12	ASIL D
CREQ-1037	Rte shall provide a service to get the status of a data element for implicit S/R communication.	TSR-16,TSR-12	ASIL D
CREQ-1033	Rte shall provide a service to access the update flag for a data element for explicit S/R communication.	TSR-12,TSR-16	ASIL D
CREQ-1036	Rte shall provide a "Never-received" status of a data element for S/R communication.	TSR-16,TSR-12	ASIL D
CREQ-1023	Rte shall provide a service to initiate the transmission of a data element with queued semantic for explicit S/R communication.	TSR-111108,TSR-12,TSR-10,TSR-16	ASIL D
CREQ-1025	Rte shall provide a service to initiate the reception of a data element with queued semantic for explicit S/R communication.	TSR-111108,TSR-12,TSR-16,TSR-10	ASIL D
CREQ-1042	Rte shall provide a service to initiate a client-server communication.	TSR-16,TSR-12	ASIL D
CREQ-1043	Rte shall provide a service to get the result of an asynchronous client-server call.	TSR-12,TSR-16	ASIL D
CREQ-1151	Rte shall provide the optional usage of one or several data transformers.	TSR-10,TSR-104422	ASIL D
CREQ-1109	Rte shall provide mode management.	TSR-100551	ASIL D
CREQ-1055	Rte shall provide a service to get the currently active mode.	TSR-100551	ASIL D

CREQ-1052	Rte shall provide a service to get the currently active, previous and next mode.	TSR-100551	ASIL D
CREQ-1053	Rte shall provide a service to initiate a mode switch.	TSR-100551	ASIL D
CREQ-1299	Rte shall provide Nv data communication.	TSR-5,TSR-4	ASIL D
CREQ-1150	Rte shall provide a callback to copy data from a NVM buffer to RTE.	TSR-5	ASIL D
CREQ-1148	Rte shall provide a callback to copy data from RTE to a NVM buffer.	TSR-4	ASIL D
CREQ-1144	Rte shall provide a callback to get notified about a finished NVM job.	TSR-4,TSR-5	ASIL D
CREQ-1147	Rte shall provide a callback to get notified about a requested mirror initialization.	TSR-5,TSR-4	ASIL D
CREQ-1046	Rte shall provide a service to read Inter-Runnable Variables with explicit behavior.	TSR-16	ASIL D
CREQ-1048	Rte shall provide a service to write Inter-Runnable Variables with explicit behavior.	TSR-16	ASIL D
CREQ-1047	Rte shall provide a service to read Inter-Runnable Variables with implicit behavior.	TSR-16	ASIL D
CREQ-1044	Rte shall provide a service to write Inter-Runnable Variables with implicit behavior.	TSR-16	ASIL D
CREQ-149683	Rte shall provide a service to get the reference of an Inter-Runnable Variable with implicit behavior for write purposes.	TSR-16	ASIL D
CREQ-174803	Rte shall provide a service to initiate the triggering of runnable entities over internal trigger communication.	TSR-6	ASIL D
CREQ-1045	Rte shall provide a service to access per-instance memory.	TSR-16	ASIL D
CREQ-1051	Rte shall provide a service to enter an exclusive area.	TSR-101876	ASIL D
CREQ-1050	Rte shall provide a service to leave an exclusive area.	TSR-101876	ASIL D
CREQ-1056	Rte's Basic Software Scheduler shall provide a service to enter an exclusive area of a BSW Module.	TSR-101876	ASIL D
CREQ-1049	Rte's Basic Software Scheduler shall provide a service to leave an exclusive area of a BSW Module.	TSR-101876	ASIL D
CREQ-1068	Rte shall provide a service to access internal calibration parameters.	TSR-5	ASIL D
CREQ-1075	Rte shall provide a service to access calibration parameters accessible via ports.	TSR-5	ASIL D
CREQ-1059	Rte shall provide a service to initialize itself.	TSR-1	ASIL D
CREQ-184330	Rte shall provide a service to activate the triggers for cyclic and background runnables.	TSR-1	ASIL D

CREQ-1073	Rte's Basic Software Scheduler shall provide a service to initialize itself.	TSR-1	ASIL D
CREQ-1161	Rte shall provide a service to trigger executable entities.	TSR-6	ASIL D
CREQ-1165	Rte shall use schedule points to invoke the scheduler of the OS.	TSR-6	ASIL D
CREQ-1129	Rte shall provide the event handling of TimingEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1112	Rte shall provide the event handling of SwcModeSwitchEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1124	Rte shall provide the event handling of AsynchronousServerCallReturnsEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1126	Rte shall provide the event handling of OperationInvokedEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1118	Rte shall provide the event handling of DataReceivedEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1132	Rte shall provide the event handling of ModeSwitchedAckEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1114	Rte shall provide the event handling of the InitEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-174850	Rte shall provide the event handling of InternalTriggerOccuredEvents to trigger a runnable.	TSR-6	ASIL D
CREQ-1295	Rte shall provide the event handling of TimingEvents to trigger a schedulable entity.	TSR-6	ASIL D
CREQ-1152	Rte shall provide a "RTE_AND_SCHM_UNINIT" state.	TSR-6	ASIL D
CREQ-1164	Rte shall provide a "RTE_UNINT_SCHM_INIT" state.	TSR-6	ASIL D
CREQ-1166	Rte shall provide a "INIT" state.	TSR-6	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory (and timing) with up to ASIL D.

11.2 Configuration constraints

SMI-2066

The user of MICROSAR Safe shall disable online calibration and measurement during series production.

The RTE can be generated with online calibration and measurement enabled for series production, but they shall be made inoperable for normal operation. Vector's XCP can e.g. be disabled safely during runtime by ASIL software.

SMI-530545

The user of MICROSAR Safe shall not use RTE Implementation Plugins for ASIL software.

RTE implementation plugins can be used to partially replace RTE APIs with custom implementations.
The user of MICROSAR Safe shall ensure that the safety related parts of the RTE do not use RTE implementation plugins.
The RTE Analyzer lists used RTE implementation plugins to assist in this integration step.

SMI-764244
The user of MICROSAR Safe shall disable Logical Execution Time Support for ASIL software
No Rte_LetFrame_<frame>_Release and Rte_LetFrame_<frame>_Terminate functions shall exist in the code.
The RTE Analyzer reports not supported template variants in case it detects usage of LET.

SMI-1102567
The user of MICROSAR Safe shall disable ArrayImplPolicy PayloadAsPointerToArray for safety related SWCs
The RTE Analyzer reports not supported template variants in case it detects usage of arrays with ArrayImplPolicy set to PayloadAsPointerToArray.

11.3 Additional verification measures

Please note that the RTE Generator and RTE Analyzer only implement measures to detect systematic faults by the software. No measures are implemented to detect or mitigate random faults on the computer used for generation.

SMI-322
The user of MICROSAR Safe shall execute the RTE Analyzer.

The RTE Analyzer performs checks to identify faults in the generated RTE. Especially out-of-bounds accesses within the RTE are detected. If the RTE Analyzer reports a fault, the generated RTE cannot be used. Moreover it provides the user of MICROSAR Safe with feedback what was generated. This feedback shall be reviewed during integration testing against the intended software design and its configuration.

For details on how to execute the RTE Analyzer, please refer to the Technical Reference of the RTE Analyzer.

SMI-860026
The user of MICROSAR Safe shall execute the RTE Analyzer on Windows.
MICROSAR Classic RTE Analyzer for Linux is not yet qualified for series production.

SMI-466082
The user of MICROSAR Safe shall execute the RTE Analyzer with extended configuration feedback commandline switch for fail-operational systems.

The RTE Analyzer performs checks to identify faults in the generated RTE. By default, the RTE Analyzer configuration feedback is only printed for ASIL partitions. An MPU can protect the ASIL partition from memory corruption that is caused by the QM partition. However, the QM partition can also have an impact on the scheduling of the ASIL partition that needs to be considered for fail-operational systems.

This is especially relevant when there are connections between ASIL and QM SWCs. When RTE Analyzer is started with the commandline switch for the extended configuration feedback, the configuration feedback is also printed for QM partitions.

For details on how to execute the RTE Analyzer, please refer to the Technical Reference of the RTE Analyzer.

SMI-36067
The user of MICROSAR Safe shall ensure that the RTE Analyzer does not report unsupported templates.

The generated RTE code is based on templates. The templates are instantiated by the RTE Generator in different variants. The RTE Analyzer verifies that the analyzed template variants have been tested during the development of the RTE Generator according to ISO 26262.

The last section of the RTE Analyzer configuration feedback report provides information about the template variants.
The report must show that no unsupported templates have been found.

SMI-763706
The user of MICROSAR Safe shall ensure that the variables Rte_InitState<Core> are mapped to memory sections with the highest ASIL level.

The RTE uses RTE_START_SEC_VAR_<Partition>_ZERO_INIT_8BIT to map the Rte_InitState variables.
In fail-safe systems with the Partitioning approach where BSW (Com, LdCom, NvM) is QM, the initialization of the Rte can be done from the partition that contains EcuM (QM Partition after StartOS).
Also Rte_InitState can be mapped to this QM partition as long as the execution of the safety-related entities is supervised by additional safety mechanisms (e.g. a watchdog).
When Rte_InitState is mapped to an ASIL partition, QM ECUM needs to call the lifecycle APIs within trusted functions.

This requirement is necessary to ensure freedom from interference with respect to memory by this component.

11.3.1 Guided integration testing

Residual faults in the RTE Generator can only be found during integration testing. Vector assumes that the user of MICROSAR Safe performs an integration testing and verification of software safety requirements according to ISO 26262 Part 6 Clauses 10 and 11 (see also SMI-4). To support this integration testing, the RTE Analyzer produces a configuration feedback report.

Please refer to the Technical Reference of the RTE Analyzer for a description of the configuration feedback report.

The following subsections describe the requirements that must be fulfilled during integration testing and verification of software safety requirements.

Each Safety Manual Item (SMI) is structured in the following way:

- The requirement that must be fulfilled
- Explanation of the requirement and a rationale
- Recommended configuration constraints (optional)
- Recommended means of complying with the requirement (optional)
- Details on the information provided by the RTE Analyzer supporting this requirement

11.3.1.1 BSW configuration

SMI-2124
The user of MICROSAR Safe shall ensure that the RTE and the operating system assume the same scheduling properties.

The scheduling properties of the RTE tasks depend on the configuration of the operating system. The scheduling properties are e.g. preemptability, core assignment or task priority.

The RTE Analyzer lists the scheduling properties in the configuration feedback report to assist in this integration step. The scheduling properties listed in the feedback report shall be verified.

SMI-2129
The user of MICROSAR Safe shall ensure that the assumptions of the operating system and the RTE are the same with regards to the locking behavior of the spinlocks.

The RTE generator uses spinlocks from the operating system to protect inter-core communication. Spinlocks must not be called concurrently on the same core. The operating system optionally provides spinlocks that can prevent these concurrent accesses on the same core. If this protection by the operating system is not used, the RTE generator has to prevent concurrent calls to the spinlock APIs on the same core.

Verification can e.g. be performed by review of the configuration feedback report.

The RTE Analyzer lists spinlocks that are not protected by the RTE to assist in this integration step.

SMI-684
The user of MICROSAR Safe shall ensure that the configuration of the COM and the RTE are consistent.

The interfaces to the COM that are used for signal reception use *void* pointers as parameter. Inconsistencies between the configuration of the COM and the RTE might lead to memory corruption by the COM. During integration, the size assumptions between the COM and the RTE shall be compared.

Verification can be performed by review of the generated configuration and/or static code analysis.

The RTE Analyzer lists relevant calls to assist in this integration step.
The RTE Analyzer listing includes the number of written bytes for the MICROSAR COM.
The RTE Analyzer lists the signals that are assigned to the individual partitions with callbacks.
The RTE Analyzer lists the signals that are assigned to the individual partitions without callbacks.

SMI-685
The user of MICROSAR Safe shall ensure that the configuration of the NVM and the RTE are consistent.

The interfaces to the NVM that are used to handle NV Block SWCs use *void* pointers as parameters.
Inconsistencies between the configuration of the NVM and the RTE might lead to memory corruption by the RTE.
During integration, the size assumptions between the NVM and the RTE shall be compared and it shall be checked that the intended block is accessed and has the expected layout.
Moreover it has to be assured that the NVM calls the Rte_GetMirror and Rte_SetMirror callbacks to store and restore the NVM data at the point in time that is expected by the application.

The Rte_GetMirror and Rte_SetMirror functions, the block size and whether the block is read or written during read and write all can typically be found in the generated data structures of NVM in NvM_Cfg.c.
The value that is used in the checks in the Rte_GetMirror and Rte_SetMirror callbacks needs to be the same as the block size.
if (size <= <BlockSize>U)

Verification can e.g. be performed by review of the generated configuration, static code analysis and/or integration testing.

The RTE Analyzer lists relevant calls to assist in this integration step.
For MICROSAR Classic NVM, the RTE Analyzer also checks that the written data does not exceed the maximum internal NVM buffer length.
The RTE Analyzer lists the detected partition that contains NVM.

11.3.1.2 Executable Entity Scheduling

SMI-2063
The user of MICROSAR Safe shall ensure that all safety-related executable entities are triggered with their correct conditions.

These conditions are:

- cylic triggers with cycle time and offset

- init triggers
- background triggers
- triggers fired by RTE APIs

Safety related executable entities include server runnables that are called with direct function calls from the ASIL partition.
If triggers have a dependency on modes, the scheduling has to be verified for all modes. Modes can be switched with the *Rte_Switch* API.
Moreover, mode switching can also happen in the task that contains the mode switch triggers or mode disablings. When mode receivers are distributed on several cores, the RTE synchronizes the mode machines in these tasks. Thus mode triggered runnables on other cores can block the scheduling on the core that initiated the mode switch. Please note that, if these tasks are tasks with lower safety level and freedom from interference with regards to memory is not ensured, also other code in the same partition can corrupt the mode disablings and lead to unintended blocking of the tasks.
Moreover queues and APIs with waitpoints can lead to blocking of ASIL tasks. See SMI-2125 and SMI-2064.
This needs to be considered when designing fail-operational systems.
All partitions need to be analyzed for fail-operational systems, not only the ASIL partitions.
Triggers can be decoupled by the minimum start interval functionality and the data reception filter functionality.
The scheduling of the executable entities also depends on the configuration of the operating system, the used controller, other running tasks and interrupt service routines and the resource usage of the entities that are implemented by the user.

Vector requires not using the minimum start interval for safety-related runnables.
Vector requires not using the data reception filter functionality for safety-related runnables.
Vector requires not using activation reasons for safety-related runnables.
Vector recommends not using background triggers for safety-related functionality.
Vector recommends not distributing mode receivers on different cores.
Vector requires not using QM mode receivers on different cores for fail-operational systems.
Vector recommends using cyclic scheduling without mode dependencies and using of a watchdog as safety mechanism for safety-related entities where possible.

Cyclic triggers are e.g. scheduled deterministically. Thus, an integration test verifying that safety-related functionality is scheduled at the expected times may be sufficient.

The RTE Analyzer lists the executable entities of SWCs and the tasks in which they are executed to assist in this integration step.
The RTE Analyzer lists recursive calls to assist in this integration step.

SMI-2128
The user of MICROSAR Safe shall ensure that reentrant runnables are reentrant.

Runnables can be called reentrantly from multiple tasks. Their implementation needs to support this use case.

Verification can e.g. be performed by review, static code analysis and/or integration testing.

The RTE Analyzer lists all runnables of SWCs that are called from concurrent tasks to assist in this integration step.
Implicit exclusive areas and nonpreemptive tasks can be configured to prevent concurrent execution.

SMI-2064
The user of MICROSAR Safe shall ensure that the timeouts configured for blocking and non-blocking APIs that are used in safety-related executable entities are adequately addressed.

If a timeout for an API is used as a safety mechanism (E.g. no checkpoint with deadline monitoring in the task), the user of MICROSAR Safe shall also ensure that the timeout value is adequate.

Relevant timeouts are:

- timeouts of *Rte_Call* APIs
- timeouts of *Rte_Result* APIs
- timeouts of *Rte_Receive* APIs
- timeouts of *Rte_Feedback* APIs
- timeouts of *Rte_SwitchAck* APIs

The timeouts also depend on the configuration of the operating system, the used controller, other running tasks and interrupt service routines and the resource usage of entities that are implemented by the user.

For blocking APIs:
Vector recommends not using blocking APIs in safety-related entities except for cross partition client-/server communication.
Vector strongly recommends not using blocking APIs without timeout.
Vector requires not using blocking APIs without timeout for fail-operational systems.

This is necessary because otherwise the APIs will wait forever after a communication loss and cannot be used for additional requests until the RTE is restarted.

For non-blocking APIs:
Vector requires not using non-blocking APIs with timeout in safety-related entities.

A review may be sufficient to verify that timeout handling is implemented properly by the SWC.
If no other safety mechanism is in place, a test that the timeout is notified at the expected time by the RTE can be used as means of verification.

The RTE Analyzer lists the blocking and non-blocking APIs of SWCs with and without timeout to assist in this integration step.

SMI-2122
The user of MICROSAR Safe shall ensure that the correct implementation method has been chosen for every exclusive area.

Exclusive areas can be used to ensure data consistency (see SMI-11).
The implementation depends on the requirements of the application and on other factors like the expected duration of the exclusive area. Interrupt locks are typically faster than resources but can only be used for short sequences. OS_INTERRUPT_BLOCKING only blocks the interrupts of the operating system whereas ALL_INTERRUPT_BLOCKING blocks all interrupts.

Verification can e.g. be performed by review, static code analysis and/or integration testing.

The RTE Analyzer lists the exclusive areas and their implementation method to assist in this integration step.

11.3.1.3 SWC Communication

SMI-41492
The user of MICROSAR SafeBRE shall provide the RTE APIs for systems without RTE.

MICROSAR Classic Basic Runtime Environment (BRE) only provides the BSW scheduler functionality of the RTE and does not support application SWCs. The implementation of the interface from the application to BSW modules must be developed according to ISO 26262. The Technical References of the BSW modules as well as the AUTOSAR standard define the semantics and APIs that will have to be implemented while integrating the BRE.

The RTE Analyzer does not assist in this integration step.

SMI-324, SMI-2065 and SMI-2123 do not apply to MICROSAR SafeBRE.

SMI-324
The user of MICROSAR Safe shall ensure that the connections between SWCs are as intended.

Many types of faults can lead to a mix of connections between SWCs. These are unlikely and usually already addressed by straight-forward integration testing.
The list of senders needs to be correct for every receiver and the subset of the received data needs to be correct. For implicit communication also the buffering in the RTE needs to be considered.

Vector recommends the following RTE subset for safety-related SWCs:

- use only 1:1 or 1:N connections.
- use the same datatype on both sides of a connection

- avoid data conversion

Information that is used from non-safety-related SWCs has to be checked for plausibility. If such a data path is found during integration, this is an indicator that your safety analysis has to be reconsidered. Please note that also other code in the same partition as the non-safety-related SWCs can corrupt the communication if freedom from interference with regards to memory is not ensured.
Moreover a mismatch between the SWC implementation and the configuration that is seen by the RTE generator when it comes to the argument direction of client-server arguments may lead to wrong input and output values.

Verification can be performed by review and/or an integration test testing the normal operation.

The RTE Analyzer list the connections between SWCs to assist in this integration step.

SMI-2065
The user of MICROSAR Safe shall ensure that inter-ECU sender-/receiver and inter-ECU client-/server communication work as expected.

This requires verification of:

- data needs to be routed to the correct ECU by the underlying communication stack. This includes 1:1, 1:N, N:1 and reception of partial signal data.

- both ECUs need to use the same data representation (datatypes, endianness, serialization)

Vector requires using E2E protection for safety-related signals.
Only evaluating the status information from the COM (e.g. update, alive timeout, invalidation status) is no sufficient safety menchanism.
Vector requires not using Inter-ECU client-server communication for safety-related signals.
Vector requires not using Inter-ECU trigger communication for safety-related signals.
Vector requires not using implicit communication for safety-related signals.
Vector requires not using blocking queued communication for safety-related signals.
Vector requires not using LDCOM trigger transmit for safety-related signals.
Vector requires not using queued communication with TP signals for safety-related signals.
Vector requires not using invalidation for safety-related signals.
Vector requires not using scaling for safety-related signals.
Vector requires not using read by value for safety-related signals.
Vector requires that other software does not corrupt RTE variables in the BSW partition when client or server use E2E protection and are not located in the BSW partition. This is necessary because the transformers including the E2E transformer need to be called in the BSW partition in order to route signals to the requesting component.
Vector recommends using 1:1 connections.
Vector recommends always sending and receiving complete data elements.

Integration testing on vehicle network level using fault-injection can be used. Vector assumes that this is normally done to verify the effectiveness of the E2E protection.
Testing needs to be performed for all postbuild variants.

The RTE Analyzer lists the APIs of SWCs with inter-ECU communication to assist in this integration step.

SMI-2123
The user of MICROSAR Safe shall ensure that all connected SWCs expect the same converted data.

The RTE offers conversions that can be applied to specific connections.

Vector recommends not using data conversion for safety-related connections.

Verification can e.g. be performed by review or integration testing of all data conversions.

The RTE Analyzer lists all APIs of SWCs that perform data conversion to assist in this integration step.

11.3.1.4 Usage of RTE Headers

SMI-2067
The user of MICROSAR Safe shall ensure that the *defines* and *typedefs* that are generated by the RTE match the expectations of the SWCs that use them.

Inconsistencies may lead to e.g, memory corruption when a runnable uses an RTE array datatype within its implementation and writes beyond the bounds of this array.
Moreover, different SWCs may have different assumptions with regards to the meaning of communicated values, e.g. if one SWC uses the symbolic name, another SWC the integral value of an enumerated type.

The following code is provided:

- Configured Application Error Defines for Client-/Server Communication
- Configured AUTOSAR Datatypes
- Configured Upper and Lower Limit Defines for Primitive Application Data Types
- Configured Init Value Defines for Sender-/Receiver Communication
- Configured InvalidValue Defines for Sender-/Receiver Communication
- Configured Enumeration Defines for CompuMethods
- Configured Mask Defines for CompuMethods
- Configured Mode Defines for Mode Communication
- Configured ActivationReasons
- Configured Rte_IsAvailable and Rte_SetAvailable defines for records with optional elements

The defines and typedefs are part of the *Rte_Type.h*, *Rte_*<SWC>.*h* and *Rte_*<SWC>_Type.*h* headers.

Vector recommends using the headers generated by the RTE when a static code analysis is performed on the application code.
Vector recommends only using the *defines* instead of the defined values in the code.
Vector recommends using the *defines* only when needed (Mode, Application Error Defines).
Vector recommends reviewing the used *defines* for safety-related SWCs.
Vector recommends not using union types in the SWCs.

Verification for correct usage of datatypes may be performed by review and/or static code analysis. Consistent usage of *defines* can be verified by review and/or integration testing with all used values.

The RTE Analyzer verifies that all accesses within the RTE do not lead to memory corruption.

SMI-2071
The user of MICROSAR Safe shall ensure that the indirect API is used consistently.

Indirect API functionality consists of the APIs:

- *Rte_Port*
- *Rte_Ports*
- *Rte_NPorts*

The indirect API makes it possible to call different APIs through an array access. The indirect API functionality can be enabled individually per port.
A wrong configuration switch can easily lead to a call outside of the array returned by the Rte_Ports API.

Vector recommends not using the indirect API.

Verification can e.g. be performed by review that the intended APIs are returned.

The RTE Analyzer does not assist in this integration step.

11.3.1.5 Usage of RTE APIs

SMI-2072
The user of MICROSAR Safe shall ensure that the RTE and all of its users have the same assumptions with regards to the sizes of the datatypes.

The RTE supports the configuration of custom datatypes for its APIs. The RTE specification mandates that arrays are passed as pointer to the array base type.
The RTE does not enforce that both sides of a connection use arrays of the same size.

No NULL pointers or invalid pointers shall be passed to RTE APIs.
As an exception NULL pointers can be passed to IN arguments of client-server operations that use a data type of category data reference.
The object to which the pointer points needs to have at least the size of the pointer base type.
No variables from ASIL partitions shall be passed as out parameters to RTE APIs in QM partitions.

RTE APIs returning a pointer/reference need special care:

- Use correct data-type/data-size for reading or writing.
- Use the pointer/reference only during execution of the runnable and within its scope.
- Do not globally store retrieved pointers, they might not be valid after runnable execution.

Vector recommends using the headers generated by the RTE when static code analysis is performed on the application code.
Vector recommends using the same datatypes on both sides of a connection.
Arrays and void pointers on interfaces where the called function writes to them are considered especially relevant.

Verification can e.g. be performed by review and/or static code analysis.

The RTE Analyzer lists APIs and runnables that use such parameters to assist in this integration step.

SMI-2073
The user of MICROSAR Safe shall ensure that RTE APIs are only called from their configured contexts.

Fast response times are crucial in embedded systems. Therefore, the RTE generator analyzes the call contexts of all APIs in order to optimize away unneeded interrupt locks. When the application calls the APIs from a different context than the RTE assumes, data consistency problems may arise.

In systems with ASIL partitions, the RTE generator uses a conservative locking strategy unless this is explicitly disabled with the parameter OptimizeLocksInASILPartitions. Locks are only optimized away if all accesses are done within the same task.

Verification can e.g. be performed by review and/or static code analysis.

The RTE Analyzer lists the optimized APIs of SWCs to assist in this integration step.
The RTE Analyzer lists APIs that must not be called by the application because they are considered unreachable due to the RTE configuration.

SMI-859695
The user of MICROSAR Safe shall ensure that implicit accesses with RteExclusiveAccessOptimization are really exclusive.

RteExclusiveAccessOptimization containers can be configured to mark implicit accesses to NV block SWCs as exclusive. As the RTE generator does not implement any data consistency mechanisms for such accesses, RteExclusvieAccessOptimization must only be configured in case the RAM mirror is not accessed concurrently in the target system.

Verification can e.g. be performed by review.

The RTE Analyzer lists concurrent accesses to the RAM mirror of NvBlockSWCs as if RteExclusiveAccessOptimization was not configured.

11.3.1.6 Configuration of RTE APIs

SMI-2074
The user of MICROSAR Safe shall ensure that the receivers can handle the initial value provided by the RTE if no write or calibration occurred.

For implicit accesses the user of MICROSAR Safe shall assure that the correct initial value is sent when *Rte_IWrite* or *Rte_IWriteRef* were not called in the runnable.

Initial values can be configured for:

- non-queued sender-/receiver communication
- inter-runnable variables
- mode ports
- calibration parameters
- per-instance memories

The initial value is returned when no sending API was called before the first read or no calibration was performed before the first read. The initial values depend on the connected components.

Vector recommends using the same initial value on all port data elements that are connected with each other.

The RTE Analyzer lists all APIs of SWCs that may provide an initial value to assist in this integration step. If possible, the RTE Analyzer reports the initial value generated by the RTE into the configuration feedback report.

SMI-2075
The user of MICROSAR Safe shall ensure that the alive timeout by the COM is not used for safety-related inter-ECU sender/receiver communication.

Safety-related communication must be protected by E2E protection. The decision if new data is available (alive) can only be made by the E2E mechanism. Data is not interpreted by the COM. For example the sending ECU might repeat old data. This is only detected by the cycle counter that is part of the E2E protection.

The RTE Analyzer lists the reading APIs that provide the alive timeout status to assist in this integration step.

SMI-2126
The user of MICROSAR Safe shall ensure that SWCs handle the *RTE_E_INVALID* return code properly.

The RTE offers a functionality to invalidate signals.

Vector requires using end-to-end protection for safety-related inter-ECU communication. Relying on the invalidation mechanism for safety-related signals is not an option. The user of MICROSAR Safe shall not use invalidation for inter-ECU communication.

Verification can e.g. be performed by review and/or integration testing.

The RTE Analyzer lists RTE APIs that return the *RTE_E_INVALID* return code to assist in this integration step.

SMI-2127
The user of MICROSAR Safe shall ensure that SWCs handle the *RTE_E_NEVER_RECEIVED* return code properly.

The RTE offers a functionality to report if a signal was received after the ECU was started.

Vector requires using end-to-end protection for safety-related signals. Relying on the never received mechanism for safety-related signals is not an option. Especially, when using the E2E transformer, its return value shall be evaluated.

Verification can e.g. be performed by review or integration testing.

The RTE Analyzer lists RTE APIs that return the *RTE_E_NEVER_RECEIVED* return code to assist in this integration step.

SMI-2125
The user of MICROSAR Safe shall ensure that the queue sizes that were chosen during the configuration are sufficient for the integrated system.

The RTE uses queues for mode communication, sender-/receiver communication and mapped client-/server communication. The queue sizes depend on the scheduling of entities and the call sequences of the APIs.
The queue handling code for server runnables and mode switches is executed in a loop in the task.
Adding new data while the queue is processed leads to additional loop iterations.
The RTE uses the configured queue sizes to limit the number of queue entries that are processed at once when the server runnable is triggered so that other entities in the same task and tasks with higher priority can be scheduled in between.
Still, entities in tasks with lower and equal priority may be blocked until the queue is empty. This needs to be considered when designing fail-operational systems. All partitions need to be analyzed for fail-operational systems, not only the ASIL partitions.

Vector recommends not using APIs with queues for safety-related functionality.
Vector requires not using queues with more than 1000000 elements for safety-related functionality.

Verification can e.g. be performed by stress testing.

The RTE Analyzer lists the queue sizes to assist in this integration step.

SMI-36068
The user of MICROSAR Safe shall ensure that all connections with end-to-end (E2E) protection are generated.

The RTE Analyzer lists RTE APIs that read or write end-to-end protected data (see SMI-98).

SMI-1093171
The user of MICROSAR Safe shall ensure that the generated code with activated illegal invocation detection is not used in ASIL Software.

The RTE offers a functionality to detect RTE API Calls called by a Runnable which should not call the RTE API according to the Runnables contract and report those to the DET. This functionality is intended to be only used as a checking mechanism to ensure that the RTE APIs are used as configured.

Using this functionality in production software could have negative impact in the overall time behaviour of the system, as a check is performed for every call to one of the RTE APIs, that are configured to be checked.

Vector recommends not using the illegal invocation detection for safety-related functionality.

11.4 Safety requirements required from other components

SMI-2121
The RTE requires the following functionality as safety requirements from the operating system:

- Interrupt enabling/disabling (TSR-101876)
- Resource handling (TSR-101876)
- Inter OS Application Communicator (IOC) sending and receiving functionality (TSR-12)
- Spin-lock functionality (TSR-101876)
- Alarm handling (TSR-6)
- Core handling (TSR-6)
- Counter handling (TSR-6)

- Schedule table handling (TSR-6)
- Activation of tasks (TSR-6)
- Event handling (TSR-6)

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

SMI-2978
The RTE requires the following functionality as safety requirements from the NVM:

- Reading blocks (TSR-5)
- Writing blocks (TSR-4)

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.
This requirement only applies if TSR-4 and TSR-5 are considered a safety requirement.

SMI-661323
This component requires sending and receiving functionality as an assumed safety requirement (TSR-111108) from other interfacing components.

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

This requirement is only applicable if TSR-111108 is assumed as a safety requirement.

11.5 Dependencies to hardware

This component does not use a direct hardware interface.

12 Safety Manual VStdLib

12.1 Assumed safety requirements

This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-VStdLib-MemoryInitializationZero	VStdLib_GenericAsr shall provide a service to initialize memory to zero.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-MemoryInitializationPattern	VStdLib_GenericAsr shall provide a service to initialize memory to a specified pattern.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-MemoryCopy	VStdLib_GenericAsr shall provide a service to copy data from one memory location to another.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-MemoryCopyEndianConversion	VStdLib_GenericAsr shall provide a service to copy data from one memory location to another with a converted endianness.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-MemoryComparison	VStdLib_GenericAsr shall provide a service to compare two memory areas.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876,	ASIL D

		TSR-111108, TSR-111109	
CREQ-VStdLib-MemoryComparisonPattern	VStdLib_GenericAsr shall provide a service to check that a memory area contains only a specified pattern.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-BitSearch	VStdLib_GenericAsr shall provide a service to search for the highest/lowest set/unset bit in different sized unsigned integers.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-Serialization	VStdLib_GenericAsr shall provide a service to serialize differently sized integers to a byte stream.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-Deserialization	VStdLib_GenericAsr shall provide a service to deserialize a byte stream to differently sized integers.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-EndianSwap	VStdLib_GenericAsr shall provide a service to swap the endianness of differently sized integers.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D
CREQ-VStdLib-EndianConversion	VStdLib_GenericAsr shall provide a service to convert differently sized integers from the platform endianness to	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14,	ASIL D

	big/little endian and vice versa.	TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	
CREQ-VStdLib-BitCopy	VStdLib_GenericAsr shall provide a service to copy data from one memory location to another where the amount of data to be copied is specified in bits.	TSR-1, TSR-4, TSR-5, TSR-6, TSR-7, TSR-8, TSR-9, TSR-10, TSR-12, TSR-13, TSR-14, TSR-15, TSR-16, TSR-17, TSR-18, TSR-100551, TSR-101876, TSR-111108, TSR-111109	ASIL D

The above listed TSRs apply to the Vector implementation (functions and macros) and not to externally defined implementations like, e.g., compiler intrinsics or user implementations.

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory and timing with up to ASIL D.

12.2 Configuration constraints

This component does not have configuration constraints.

12.3 Additional verification measures

This component does not require additional verification measures.

12.4 Safety requirements required from other components

This component does not require safety requirements from other components.

12.5 Dependencies to hardware

This component does not use a direct hardware interface.

13 Safety Manual Wdg

13.1 Safety requirements

SMI-279
This component provides the following safety requirements:

ID	Safety requirements	Linked TSR
CREQ-105559	Wdg shall provide a service to initialize itself and the watchdog hardware.	TSR-13,TSR-14,TSR-15
CREQ-105560	Wdg shall provide a service to set the watchdog mode.	TSR-13,TSR-14,TSR-15
CREQ-105561	Wdg shall provide a service to set trigger condition.	TSR-13,TSR-14,TSR-15
CREQ-105563	Wdg shall provide a service that triggers the watchdog hardware if trigger condition is valid.	TSR-13,TSR-14,TSR-15
CREQ-106194	Wdg shall provide an OFF mode.	TSR-13,TSR-14,TSR-15
CREQ-106195	Wdg shall provide a SLOW mode.	TSR-13,TSR-14,TSR-15
CREQ-106196	Wdg shall provide a FAST mode.	TSR-13,TSR-14,TSR-15

13.2 Configuration constraints

This component does not have configuration constraints.

13.3 Additional verification measures

SMI-253
The user of MICROSAR Safe shall verify that the Wdg driver is only initialized at intended points in time, e.g. during initialization.
Unintended re-initialization may lead to a disabled watchdog.

SMI-254
The user of MICROSAR Safe shall verify that the Wdg driver services are only called by the watchdog interface.
This requierment does not apply to the service for initalization and the interface to Gpt (general purpose timer).
This requirement is fulfilled for all components of MICROSAR.
If e.g. the trigger condition is extended by another component, this may lead to unintended triggering of the watchdog.

13.4 Safety requirements required from other components

13.4.1 Safety features required from other components

SMI-255
The Wdg interface shall call the service to set the mode and the trigger condition as expected by the Wdg driver.
This requirement is fulfilled if the watchdog interface and manager by Vector is used.
If the watchdog is not properly set up, it may not provide the expected protection.

13.4.2 Coexistence with other components

SMI-280
This component requires coexistence with Det, Dem, Dio, Spi, Sbc and Gpt components if the interface for those components is configured.
This requirement is fulfilled if components from Vector are used.

13.5 Dependencies to hardware

The hardware interface is described in the Safety Manual of the low-level part of the Wdg driver.

14 Safety Manual Wdg (DrvWd_XTle4278gEAsr)

14.1 Safety features

No additional safety features are provided for DrvWd_XTle4278gEAsr.

14.2 Configuration constraints

No additional configuration constraints are required.

14.3 Additional verification measures

No additional verification measures are required.

14.4 Dependencies to other components

14.4.1 Safety features required from other components

SMI-2676
The Dio driver shall provide the DIO channel read service and the DIO channel write service as a safety feature.
This requirement is fulfilled if components from Vector are used.

14.4.2 Coexistence with other components

No additional coexistence with other components is required.

14.5 Dependencies to hardware

This component does not use a direct hardware interface. The Dio driver is used to access the hardware watchdog unit.

15 Safety Manual WdgIf

15.1 Assumed safety requirements

SMI-519
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-107414	WdgIf shall provide a service to set the mode of a watchdog device	TSR-13,TSR-15,TSR-14	ASIL D
CREQ-107415	WdgIf shall provide a service to set the trigger condition for a watchdog device	TSR-13,TSR-15,TSR-14	ASIL D
CREQ-107416	WdgIf shall support a mechanism to combine statuses of different cores and handle one watchdog for different cores	TSR-13,TSR-14,TSR-15	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory with up to ASIL D.

Hint: This component does not offer an init function.

15.2 Configuration constraints

SMI-522
If a state combiner is used, the user of MICROSAR Safe shall configure

- *WdgIfStateCombinerOSSpinlock* or *WdgIfStateCombinerSpinlockID* and
- *WdgIfStateCombinerStartUpSyncCycles*

for each core that is used by the state combiner.

SMI-523
If a state combiner is used and *WdgIfStateCombinerStartUpSyncCycles* is set to a value s, the user of MICROSAR Safe shall consider that for the first s SupervisionCycles of the master, the master does not monitor the slave triggers.
However, reset requests from a slave within the first s SupervisionCycles, are escalated by the master with the next call of the master's *WdgM_MainFunction()*.

15.3 Additional verification measures

SMI-525
The user of MICROSAR Safe shall verify that the output path of the generator is empty before the generator is started.
The output path can be defined by the command line argument *OUTPUT-DIRECTORY*.

SMI-526
The user of MICROSAR Safe shall inspect the messages of the generator execution.
If the generator aborts the generation process with an error message, the (partially)

generated output files shall not be used in the system.
If the generator detects an error, a message starting with "ERROR" is displayed on the standard output.
If the generator shows a warning message starting with "WARNING", the user of MICROSAR Safe shall ensure that the cause of the warning does not invalidate the generated output files.

SMI-527
The user of MICROSAR Safe shall verify that the following preprocessor directives are defined with the correct value independent from whether a state combiner is configured or not:

Preprocessor Directive	Value
WDGIF_VERSION_INFO_API	*STD_ON* if *WdgIfVersionInfoApi* is *TRUE*, otherwise *STD_OFF*.
WDGIF_DEV_ERROR_DETECT	*STD_ON* if *WdgIfDevErrorDetect* is *TRUE*, otherwise *STD_OFF*.
WDGIF_USE_STATECOMBINER	*STD_ON* if *WdgIfUseStateCombiner* is *TRUE*, otherwise *STD_OFF*.

The defines can be found in *WdgIf_Cfg_Features.h*.

SMI-528
The user of MICROSAR Safe shall verify that the following preprocessor directives are defined with the correct value only if a state combiner is configured:

Preprocessor Directive	Value
WDGIF_STATECOMBINER_USE_OS_SPIN_LOCK	*STD_ON* if *WdgIfStateCombinerUseOsSpinlock* is *TRUE*, otherwise *STD_OFF*.

This define can be found in *WdgIf_Cfg_Features.h*.

SMI-529
The user of MICROSAR Safe shall verify that the following preprocessor directives are defined with the correct value independent from whether a state combiner is configured or not:

Preprocessor Directive	Value
WDGIF_NUMBER_OF_WDGIFDEVICES	The number of configured WD Interface Devices.

This define can be found in *WdgIf_LCfg.h*.

SMI-530
The user of MICROSAR Safe shall verify that the following preprocessor directives are defined with the correct value only if a state combiner is configured:

Preprocessor Directive	Value
WDGIF_NUMBER_OF_SLAVES	The configured number of slave cores. Cores that are not attached to a State Combiner (i.e. they run independent from other cores) do not count as slave.

This define can be found in *WdgIf_LCfg.h*.

SMI-531
The user of MICROSAR Safe shall verify that the C-struct *const WdgIf_InterfaceType WdgIf_Interface* is defined in *WdgIf_LCfg.c*.

WdgIf_Interface shall contain the following fields:

Field	Value	Description
1st	*WDGIF_NUMBER_OF_WDGIFDEVICES*	The number of WD Interface Devices.
2nd	*WdgIf_FunctionsPerWdg*	A reference to the list of WD Interface Devices.

If a state combiner is configured, *WdgIf_Interface* shall also contain the following field:

Field	Value	Description
3rd	*&wdgif_statecombiner_config*	A reference to the state combiner data structure.

SMI-532
The user of MICROSAR Safe shall verify that the array *static const WdgIf_InterfaceFunctionsPerWdgDeviceType WdgIf_FunctionsPerWdg* [WDGIF_NUMBER_OF_WDGIFDEVICES] is defined in *WdgIf_LCfg.c*.
WdgIf_FunctionsPerWdg shall refer all – and only - the WD Interface Devices that are configured in the EDF.
WdgIf_FunctionsPerWdg[i] shall refer the underlying WD Interface Device in the EDF with *WdgIfDeviceIndex = i*.
The fields in an array element in *WdgIf_FunctionsPerWdg* shall be set as follows:

Field	Value	Description
1st	*&device_functions*	If the underlying WD Interface Device is directly linked to a WD driver for device, then the field refers to the C-struct device_functions.
1st	NULL_PTR	If the underlying WD Interface Device shares the WD driver with other WD Interface Devices using a state combiner, then the linked WD device is referred in *wdgif_statecombiner_config*.

If a state combiner is configured, an array element in *WdgIf_FunctionsPerWdg* shall also contain the following field:

Field	Value	Description
2nd	*WdgInstance*	A number that uniquely identifies the WD Interface Device instance for the underlying platform. All instances on the same platform are numbered consecutively starting with 0. Example: One platform has instances A and B, another platform has instances C and D. Then A, B, C, and D have *WdgInstance* set (in this order) as: 0,1,0,1.

SMI-533
If a state combiner is configured, the user of MICROSAR Safe shall verify that the array *static const WdgIf_StateCombinerConfigType wdgif_statecombiner_config* is defined in

WdgIf_LCfg.c.
The fields in *wdgif_statecombiner_config* shall be set as follows:

Field	Value	Description
1st	*WDGIF_NUMBER_OF_SLAVES*	The number of configured slaves.
2nd	*WdgIfStateCombinerSpinlockID*	Depending on configuration: Either the referenced OSSpinlock (*WdgIfStateCombinerOSSpinlock*) or the value of field *WdgIfStateCombinerSpinlockID* in the EDF.
3rd	*WdgIfStateCombinerStartUpSyncCycles*	The value of field *WdgIfStateCombinerStartUpSyncCycles* in the EDF.
4th	*&device_functions*	A reference to the WD driver API functions for device.
5th	*wdgif_statecombiner_shared_memory*	A reference to the shared memory for the state combiners of the Watchdog Interfaces.
6th	*wdgif_statecombiner_slave_trigger_pattern*	A reference to the array of the state combiners trigger pattern of each slave.

SMI-534
If a state combiner is configured, the user of MICROSAR Safe shall verify that the array *WdgIf_StateCombinerSharedMemory wdgif_statecombiner_shared_memory* [WDGIF_NUMBER_OF_SLAVES] is defined in *WdgIf_LCfg.c*.
The WdgIf writes to this array, hence it is not *const*.
wdgif_statecombiner_shared_memory shall contain one array element for each configured slave and the fields in the element shall be set as shown in the following table:

Field	Value	Description
1st	*0u*	Initial value for the slave's counter.
2nd	*~((uint32) 0u)*	Inverse initial value for the slave's counter.

SMI-536
If the state combiner is configured, the user of MICROSAR Safe shall verify that for a configured slave with ID the C-struct *static const WdgIf_StateCombinerSlaveTriggerPatternType wdgif_statecombiner_config_slave<ID>* is defined in *WdgIf_LCfg.c*.
wdgif_statecombiner_config_slave<ID> shall be set as follows:

Field	Value	Description
1st	*WdgIfStateCombinerReferenceCycle*	The value of field *WdgIfStateCombinerReferenceCycle* in the EDF.
2nd	*WdgIfStateCombinerSlaveIncrementsMin*	The value of field *WdgIfStateCombinerSlaveIncrementsMin* in

		the EDF.
3rd	*WdgIfStateCombinerSlaveIncrementsMax*	The value of field *WdgIfStateCombinerSlaveIncrementsMax* in the EDF.

SMI-537
The user of MICROSAR Safe shall verify that for each configured platform the C-struct *static const WdgIf_InterfaceFunctionsType <WdgIfDevice>_functions* is defined in WdgIf_LCfg.c.
The fields for each C-struct <WdgIfDevice>_functions shall be set as follows:

Field	Value	Description
1st	*Wdg_*<infix>_SetMode_	A reference to the WD driver's API function to set the mode of the device referred to by infix.
2nd	*Wdg_*<infix>_SetTriggerCondition_	A reference to the WD driver's API function to set the trigger condition of the device referred to by infix.

15.4 Safety requirements required from other components

SMI-520
This component requires the triggering of the watchdog and setting the triggering mode as a safety requirement from the watchdog driver.
This requirement is fulfilled if a watchdog driver by Vector is used.

SMI-3085
The WdgIf shall be used in order to call the services of underlying Watchdog driver(/s) to set the mode and the trigger condition as expected by the drivers.
The user of MICROSAR Safe shall verify that the WdgIf services are only called by the WdgM.
This requirement is fulfilled for all components of MICROSAR (if Vector's WdgM is used).
If e.g. the trigger condition is called by another component, this may lead to unintended triggering of the watchdog.
If the watchdog stack is not properly set up, it may not provide the expected protection.

15.5 Dependencies to hardware

This component does not use a direct hardware interface.

16 Safety Manual WdgM

16.1 Assumed safety requirements

SMI-373
This component assumes the following safety requirements:

ID	Safety Requirement	Associated TSR	ASIL
CREQ-102743	WdgM shall provide a service to initialize itself.	TSR-1	ASIL D
CREQ-102747	WdgM shall provide Supervised Entities.	TSR-14,TSR-15,TSR-13	ASIL D
CREQ-102748	WdgM shall provide Checkpoints.	TSR-15,TSR-14,TSR-13	ASIL D
CREQ-102746	WdgM shall provide a mechanism for Alive Supervision.	TSR-14	ASIL D
CREQ-102745	WdgM shall provide a mechanism for Deadline Supervision.	TSR-15	ASIL D
CREQ-102744	WdgM shall provide a mechanism for Logical Supervision.	TSR-13	ASIL D
CREQ-102749	WdgM shall provide a service to trigger a checkpoint.	TSR-13,TSR-15,TSR-14	ASIL D
CREQ-102754	WdgM shall provide a service to activate the supervision of a Supervised Entity.	TSR-13,TSR-15,TSR-14	ASIL D
CREQ-102755	WdgM shall provide a service to deactivate the supervision of a Supervised Entity.	TSR-14,TSR-13,TSR-15	ASIL D
CREQ-102763	WdgM shall provide a service to cyclically update the global supervision status.	TSR-14,TSR-13,TSR-15	ASIL D
CREQ-102756	WdgM shall provide modes.	TSR-15,TSR-14,TSR-13	ASIL D
CREQ-102758	WdgM shall provide a service to set a new trigger mode.	TSR-13,TSR-14,TSR-15	ASIL D
CREQ-102764	WdgM shall provide a global status DEACTIVATED.	TSR-14,TSR-13,TSR-15	ASIL D
CREQ-102765	WdgM shall provide a global status OK.	TSR-14,TSR-15,TSR-13	ASIL D
CREQ-102766	WdgM shall provide a global status FAILED.	TSR-15,TSR-14,TSR-13	ASIL D
CREQ-102767	WdgM shall provide a global status EXPIRED.	TSR-15,TSR-14,TSR-13	ASIL D
CREQ-102768	WdgM shall provide a global status STOPPED.	TSR-14,TSR-13,TSR-15	ASIL D
CREQ-102769	WdgM shall provide a supervised entity status DEACTIVATED.	TSR-14,TSR-13,TSR-15	ASIL D
CREQ-	WdgM shall provide a supervised entity status OK.	TSR-14,TSR-	ASIL

102770		15,TSR-13	D
CREQ-102771	WdgM shall provide a supervised entity status FAILED.	TSR-15,TSR-13,TSR-14	ASIL D
CREQ-102772	WdgM shall provide a supervised entity status EXPIRED.	TSR-13,TSR-14,TSR-15	ASIL D

This component in addition to the assumed safety requirements ensures freedom from interference with respect to memory and timing with up to ASIL D.

The watchdog manager is able to detect program flow violations, alive counter violations and deadline violations.
The following types of faults can be detected:

- omission of an operation (program flow, alive counter),
- unrequested execution of an operation (program flow, alive counter),
- operation executed too early (alive counter, deadline),
- operation executed too late (alive counter, deadline), and
- operations executed in the wrong sequence (program flow).

16.2 Configuration constraints

SMI-501
The user of MICROSAR Safe shall use the WdgM only on 32-bit microcontroller platforms.
This requirement is necessary to ensure usage in a released configuration.

SMI-499
The user of MICROSAR Safe shall configure and verify alive supervision for a supervised entity.
If no alive supervision is configured for a supervised entity, WdgM cannot detect if the corresponding checkpoint is reached at least once.
Please note that for non-periodic supervised entities alive supervision is not possible.
This requirement is necessary to ensure usage in a released configuration.

A value of a pre-compile configuration parameter is valid for every core. A different value cannot be set for the same pre-compile parameter and different cores.

SMI-498
The user of MICROSAR Safe shall set the value of *WdgMTimebaseSource* according to the required source of time ticks:

WdgMTimebaseSource	Description
WDGM_INTERNAL_SOFTWARE_TICK	An internal time source for Deadline Monitoring is selected. The tick counter is incremented each time the *WdgM_MainFunction()* is invoked.

WDGM_OS_COUNTER_TICK	An OsCounter is selected for Deadline Monitoring as timebase. The user of MICROSAR Safe is responsible for configuring the OsCounter accurately.
WDGM_EXTERNAL_TICK	An external time source for Deadline Monitoring is selected. The tick counter is incremented each time the *WdgM_UpdateTickCount()* function is invoked. The function is implemented in the WdgM. The user of MICROSAR Safe is responsible for calling WdgM_UpdateTickCount() accurately.

This requirement is necessary to ensure usage in a released configuration.

SMI-560
The user of MICROSAR Safe shall use the functions *WdgM_ActivateSupervisionEntity()* and *WdgM_DeactivateSupervisionEntity()* to activate and deactivate the supervision of supervised entities.

Activation and deactivation shall only be performed by a software component that is developed according to the highest ASIL that is allocated to the ECU.

The functions are only available if *WdgMEntityDeactivationEnabled* is set to *TRUE*.
Vector recommends setting *WdgMEntityDeactivationEnabled* to *FALSE* to prevent that faults are not detected.
This requirement is necessary to ensure TSR-13, TSR-14 and TSR-15.

16.3 Additional verification measures

SMI-3072
The user of MICROSAR Safe shall verify that the WdgM is initialized only at intended points in time, e.g. during initialization.
Unintended re-initialization may lead to a incorrect monitoring.

SMI-524
The user of MICROSAR Safe shall inspect the messages of the generator execution.
If the generator aborts the generation process with an error message, the (partially) generated output files shall not be used in the system.
If the generator detects an error, a message starting with "ERROR" is displayed on the standard output.
If the generator shows a warning message starting with "WARNING", the user of MICROSAR Safe shall ensure that the cause of the warning does not invalidate the generated output files.

SMI-126207
If WdgMStatusReportingMechanism is configured to be WDGM_USE_MODE_SWITCH_PORTS or WdgMStatusReportingMechanism is configured to be WDGM_USE_NOTIFICATIONS and WdgMUseRte is enabled an additional file with executable code is generated for each used core. Another precondition for the existence of the file is that there must be at least one parameter WdgMReportStatusViaRte of a WdgMMode or WdgMSupervisedEntity enabled (of the corresponding core).

The file is named WdgM_StatusNotifications_Core<CoreId>.c.

The user of MICROSAR Safe shall inspect the generated executable code. Notification functions are generated for each WdgMMode and WdgMSupervisedEntity if WdgMReportStatusViaRte is enabled. Only valid functions from Rte must be called within that notifications.

SMI-605940
If *WdgMTimebaseSource* is configured to be WDGM_OS_COUNTER_TICK the user of MICROSAR Safe shall verify that the *WdgMOsCounterMaxAllowedValue* is correctly generated according to the referenced Os counter.

16.3.1 Additional verification using WdgM Verifier

SMI-503
The user of MICROSAR Safe shall execute the supplied WdgM Verifier.
Instructions can be found in the technical reference of WdgM on how to run the WdgM Verifier.

SMI-504
The user of MICROSAR Safe shall verify that the report of the WdgM Verifier

- comprises "All tests passed" in the last line. and
- that all tests in the *Summary* of the report are marked with *PASSED*.

SMI-512
The user of MICROSAR Safe shall verify the generated local transitions in *wdgm_verifier_info.c*.

Generated local transitions are defined by the C-struct array with the name *local_transitions*.
The array holds all local transitions of all supervised entities.
Each local transition lt contains the names of:

- the source entity (SE) of lt,
- the source checkpoint of lt,
- the destination entity (SE) of lt and
- the destination checkpoint of lt.

Verification shall include that

- each local transition is defined as stated in the System Specification, and
- no local transition in the System Specification is missing.

SMI-513
The user of MICROSAR Safe shall verify the generated global transitions in *wdgm_verifier_info.c*.

Generated global transitions are defined by the C-struct array with the name *global_transitions*.
The array holds all global transitions of all supervised entities.
Each global transition gt contains the names of:

- the source entity (SE) of gt,
- the source checkpoint of gt,
- the destination entity (SE) of gt, and
- the destination checkpoint of gt.

Verification shall include that

- each global transition is defined as stated in the System Specification, and
- no global transition in the System Specification is missing.

SMI-514
The user of MICROSAR Safe shall verify the checkpoints in *wdgm_verifier_info.c*.

Supervised entities named se are defined by a C-struct array with the name *se_*<se>_cp_list_.
The array *se_*<se>_cp_list_ holds information about all checkpoints configured for se.
Each array item contains information about one checkpoint cp of the supervised entity se:

- the supervised entity's ID (for this cp of se),
- the checkpoint's ID (for this cp of se),
- the supervised entity name (for this cp of se), and
- the checkpoint name (for this cp of se).

Verification shall include that

- each checkpoint is configured as stated in the System Specification, and
- no checkpoint for the actual supervised entity is missing.

SMI-515
The user of MICROSAR Safe shall verify the supervised entities in *wdgm_verifier_info.c*.

Supervised entities named se are defined by a C-struct array with the name *entities*.
The array *entities* holds information about a configured supervised entity.
The fields in an array item for a supervised entity se shall have the following values (in this order):

Value	Source Line Comment
The entity ID (of se).	enitity id

The entity name (of se).	entity name
The number of checkpoints configured for se.	number of checkpoints
A reference *se_*<se>*_cp_list_*, which refers to the list of CPs for se	this entity's checkpoints
A reference to the callback function for se as configured in *WdgMLocalStateChangeCbk* if WdgMStatusReportingMechanism is configured to be WDGM_USE_NOTIFICATIONS (otherwise NULL_PTR).	*WdgMLocalStateChangeCbk*
A reference to rte function for se if WdgMStatusReportingMechanism is configured to be WDGM_USE_MODE_SWITCH_PORTS (otherwise NULL_PTR).	*WdgM_StatusReportToRte*
false	autosar_3_1_x_compatibility
The application task for se as configured in field *WdgMAppTaskRef*	*WdgMAppTaskRef*

Verification shall include that

- each supervised entity is configured as stated in the System Specification, and
- no supervised entity is missing.

SMI-516
The user of MICROSAR Safe shall verify the deadline supervisions in *wdgm_verifier_info.c*.

Deadline supervisions are defined by a C-struct array with the name *deadline_supervisions*.
The array *deadline_supervisions* holds information about all transitions with deadline supervision.
Each deadline supervision dl contains the following values:

- the source entity name (SE) of dl,
- the source checkpoint name of dl,
- the destination entity name (SE) of dl,
- the destination checkpoint name of dl,
- the minimum deadline for dl (in seconds), and
- the maximum deadline of dl (in seconds).

Verification shall include that

- each deadline supervision is configured as stated in the System Specification, and
- no deadline supervision is missing.

SMI-517
The user of MICROSAR Safe shall verify the alive supervisions in *wdgm_verifier_info.c*.

Alive supervisions are defined by a C-struct array with the name *alive_supervisions*.
The array *alive_supervisions* holds information about all transitions with alive supervision.
Each alive supervision al contains the following values:

- the supervised entity name (SE) of al,
- the checkpoint name of that se of al,
- the number of expected alive indications per reference cycle of al,
- the minimum margin for alive indications of al,
- the maximum margin for alive indications of al, and
- the number of supervision reference cycle of al.

Verification shall include that

- each alive supervision is configured as stated in the System Specification, and
- no alive supervision is missing.

SMI-518
The user of MICROSAR Safe shall verify the configured cores in *wdgm_verifier_info.c*.

The configured cores are defined in the *main()* function.
For each configured core with ID, the following line shall be present:

```
result += verify (&WdgMConfig_Modem_core<ID>, &verifier_info);
```

where ID is the core ID and m is the ID of the WdgM mode.

Verification shall include that for each core there is a corresponding line in the file.

16.3.2 Additional verification of generator execution

SMI-506
The user of MICROSAR Safe shall verify that for the following arrays in *WdgM_Cfg.c*, the array length matches the number of items in the array:

- *WdgMTransition*
- *WdgMGlobalTransition*
- all arrays named *StartsGlobalTransition*<se>_<cp>_<i>_ (for a supervised entity se, a checkpoint cp and an integer i)
- *WdgMCheckPoint*
- *WdgMSupervisedEntity*
- all arrays named *WdgMTriggerMode_core<ID>* (for each core with ID)
- *WdgMWatchdogDevice<ID>* (for each core with ID)
- *WdgMAllowedCallers*

Some of these arrays have preprocessor defines for their size, e.g. *WdgMCheckPoint [WDGM_NR_OF_CHECKPOINTS]*. These defines can be found in *WdgM_Cfg.h*.

SMI-507
The user of MICROSAR Safe shall verify that each item in the array *WdgMSupervisedEntity* follows the rule:

- *WdgMCheckpointRef* has a value of the form _&WdgMCheckPoint[i] with i < WDGM_NR_OF_CHECKPOINTS, and

The array can be found in *WdgM_Cfg.c*.

SMI-27923
The user of MICROSAR Safe shall verify that each WdgM_StatusReportToRte member of struct WdgM_ConfigType and each WdgM_StatusReportToRte member of struct WdgM_SupervisedEntityType has a valid entry if WDGM_STATUS_REPORTING_MECHANISM is set to WDGM_USE_MODE_SWITCH_PORTS.
The functions must be implemented by the Rte and must have the following signature: Std_ReturnType (*WdgM_StatusReportToRte) (WdgMMode).
Both structs can be found in WdgM_Cfg.c.

SMI-508
The user of MICROSAR Safe shall verify that for each core with ID *WDGM_NR_OF_WATCHDOGS_CORE<ID>* matches the actual number of configured WD devices.
The define can be found in *WdgM_Cfg.h*.

SMI-509
The user of MICROSAR Safe shall verify that for each core with ID *WDGM_NR_OF_TRIGGER_MODES_CORE<ID>* matches the actual number of configured Watchdog Manager Trigger Modes.
The define can be found in *WdgM_Cfg.h*.

SMI-510

The user of MICROSAR Safe shall verify that *WDGM_NR_OF_ALLOWED_CALLERS* matches the number of modules that call function *WdgM_SetMode()*.
The define can be found in *WdgM_Cfg.h*.

SMI-511
The user of MICROSAR Safe shall verify that in *WdgMConfig_Mode<m>_core<ID>* (for each core with ID and every mode m), the field *WdgMCallersRef* points to *WdgMAllowedCallers* and *WdgMAllowedCallers* is an array of type *WdgM_CallersType* with a length of *WDGM_NR_OF_ALLOWED_CALLERS*.
The variable can be found in *WdgM_Cfg.h*.

SMI-550
The user of MICROSAR Safe shall verify the types used by WdgM.

If the configuration parameter *WDGM_USE_RTE* is set to *STD_ON*, the types from *Rte_Type.h* are used:

Type	Allowed Value
WdgM_SupervisedEntityIdType	*uint16*
WdgM_CheckpointIdType	*uint16*
WdgM_ModeType	*uint8*
WdgM_LocalStatusType	*uint8*
WdgM_GlobalStatusType	*uint8*

The WdgM includes *WdgM_Rte_Includes.h* if and only if *WDGM_USE_RTE* is set to *STD_ON*.

If the configuration parameter *WDGM_USE_RTE* is set to *STD_OFF*, the types from WdgM are used:

Type	Allowed Value
WdgM_SupervisedEntityIdType	*uint16*
WdgM_CheckpointIdType	*uint16*
WdgM_ModeType	*uint8*
WdgM_LocalStatusType	*uint8*
WdgM_GlobalStatusType	*uint8*

SMI-551
The user of MICROSAR Safe shall verify the definitions used by WdgM.

If the configuration parameter *WDGM_USE_RTE* is set to *STD_ON*, the definitions from *Rte_WdgM_Type.h* are used.
If the configuration parameter *WDGM_USE_RTE* is set to *STD_OFF*, the definitions from WdgM are used.

Definition	Value
WDGM_LOCAL_STATUS_OK	0

WDGM_LOCAL_STATUS_FAILED	1
WDGM_LOCAL_STATUS_EXPIRED	2
WDGM_LOCAL_STATUS_DEACTIVATED	4
WDGM_GLOBAL_STATUS_OK	0
WDGM_GLOBAL_STATUS_FAILED	1
WDGM_GLOBAL_STATUS_EXPIRED	2
WDGM_GLOBAL_STATUS_STOPPED	3
WDGM_GLOBAL_STATUS_DEACTIVATED	4

The WdgM includes *Rte_WdgM_Type.h* if and only if *WDGM_USE_RTE* is set to *STD_ON*.

16.4 Safety requirements required from other components

SMI-372
This component requires setting a trigger condition and setting the triggering mode as an assumed safety requirement (TSR-13, TSR-14, TSR-15) from WdgIf component.

For components provided as part of MICROSAR Safe, the respective section of the Safety Manual for that component allows you identifying if this requirement is fulfilled.

SMI-3414
The user of MICROSAR Safe shall call the service to set the mode as expected by the Wdg stack.
If the watchdog is not properly set up, it may not provide the expected protection.

16.5 Dependencies to hardware

This component does not use a direct hardware interface.

17 Glossary and Abbreviations

17.1 Glossary

Term	Definition
User of MICROSAR Safe	Integrator and user of components from MICROSAR Safe provided by Vector.
MICROSAR Safe	MICROSAR Safe comprises MICROSAR SafeBSW and MICROSAR SafeRTE as Safety Element out of Context. MICROSAR SafeBSW is a set of components, that are developed according to ISO 26262 [1], and are provided by Vector in the context of this delivery. The list of MICROSAR Safe components in this delivery can be taken from the documentation of the delivery.
Critical section	A section of code that needs to be protected from concurrent access. A critical section may be protected by using the AUTOSAR exclusive area concept.
Configuration data	Data that is used to adapt the MICROSAR Safe component to the specific use-case of the user of MICROSAR Safe. Configuration data typically comprises among others: feature selection, routing tables, channel tables, task priorities, memory block descriptions.
Generated code	Source code that is generated as a result of the configuration in DaVinci Configurator Pro
Partition	A set of memory regions that is accessible by tasks and ISRs. Synonym to OSApplication.

17.2 Abbreviations

Abbreviation	Description
ASIL	Automotive Safety Integrity Level
BSWMD	Basic Software Module Description
CPU	Central Processing Unit
CREQ	Component Requirement
EEPROM	Electronically Erasable and Programmable Read-only Memory
ECC	Error Correcting Code
ECU	Electronic Control Unit
EXT	Driver for an external hardware unit
ISO	International Standardization Organization
MCAL	Microcontroller Abstraction
MIP	Module Implementation Prefix
MSSV	MICROSAR Safe Silence Verifier
OS	Operating System
PDU	Protocol Data Unit
QM	Quality Management

RAM	Random Access Memory
SMI	Safety Manual Item
TCL	Tool Confidence Level

18 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com